MI VENC API

Version 3.5

REVISION HISTORY

Revision None.	Description	Date
3.0	• Initial release	12/04/2020
3.1	• Modified MI_VENC_PackInfo_t, added u8FrameQP, s32PocNum and u32Gradient for every frame	01/19/2021
3.2	• Added u32MaxISize and u32MaxPSize for H26x VENC_ParamH26XVbr_t and MI_VENC_ParamH26XCbr_t	01/26/2021
	• Added procfs introduction	08/25/2021
3.3	• Modified MI_VENC_H265eNaulType_e to MI_VENC_H265eNaluType_e	11/17/2021
3.4	• Modified MI_VENC_InputSrcBufferMode_e, adding member variable description for E_MI_VENC_INPUT_MODE_RING_UNIFIED_DMA	02/18/2022
	• Add usage note of MI_VENC_SetInputSourceConfig for venc sub stream	03/04/2022
3.5	• Add Maruko platform introduction	03/24/2022
	• Add description for venc scaling down function	03/25/2022
	• Add description for alignment parameters of the input buffer	04/22/2022

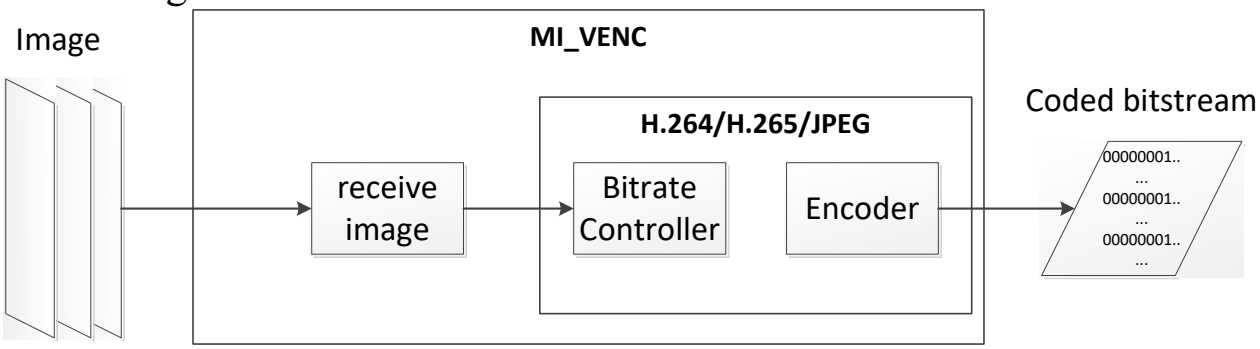
TABLE OF CONTENTS

1. OVERVIEW

1.1. Module Description

The video coding module mainly provides the functions of creating and destroying a video coding channel, resetting a video coding channel, starting and stopping receiving images, setting and acquiring coding channel attributes, and acquiring and releasing a code stream. This module supports multi-channel real-time coding, each channel is independent, and different coding protocols and profiles can be set.

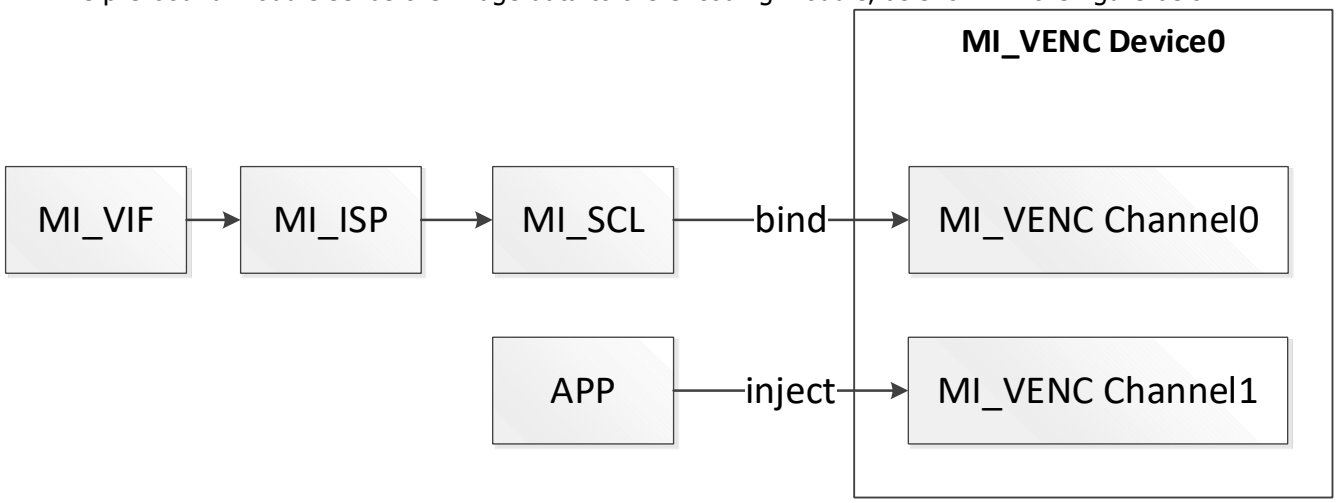
1.2. Encoding Process



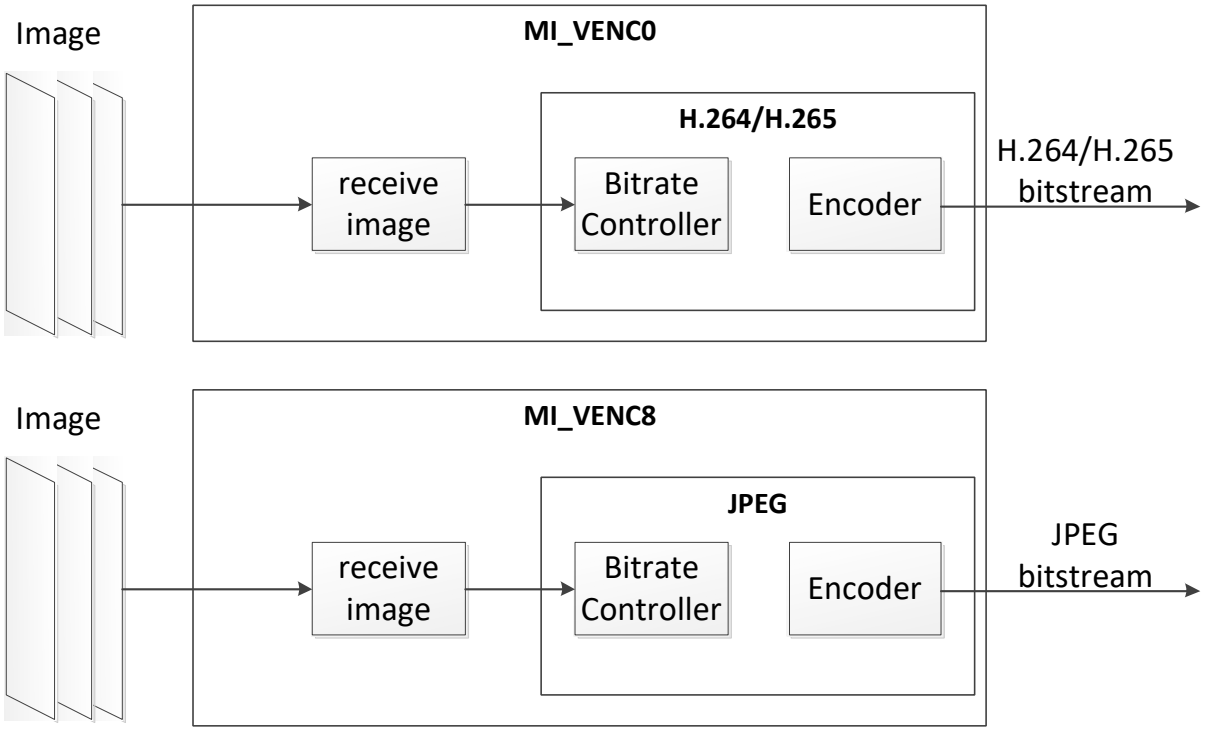
The encoding process includes receiving the input image, coding the input image, and outputting the encoded stream. H.264/H.265 input image YUV format only supports NV12, JPEG input image YUV format supports NV12 or YUYV422.

The input sources of this module include the following two types:

1. The user's app directly sends image data to the encoding module.
2. The pre-bound module sends the image data to the encoding module, as shown in the figure below:

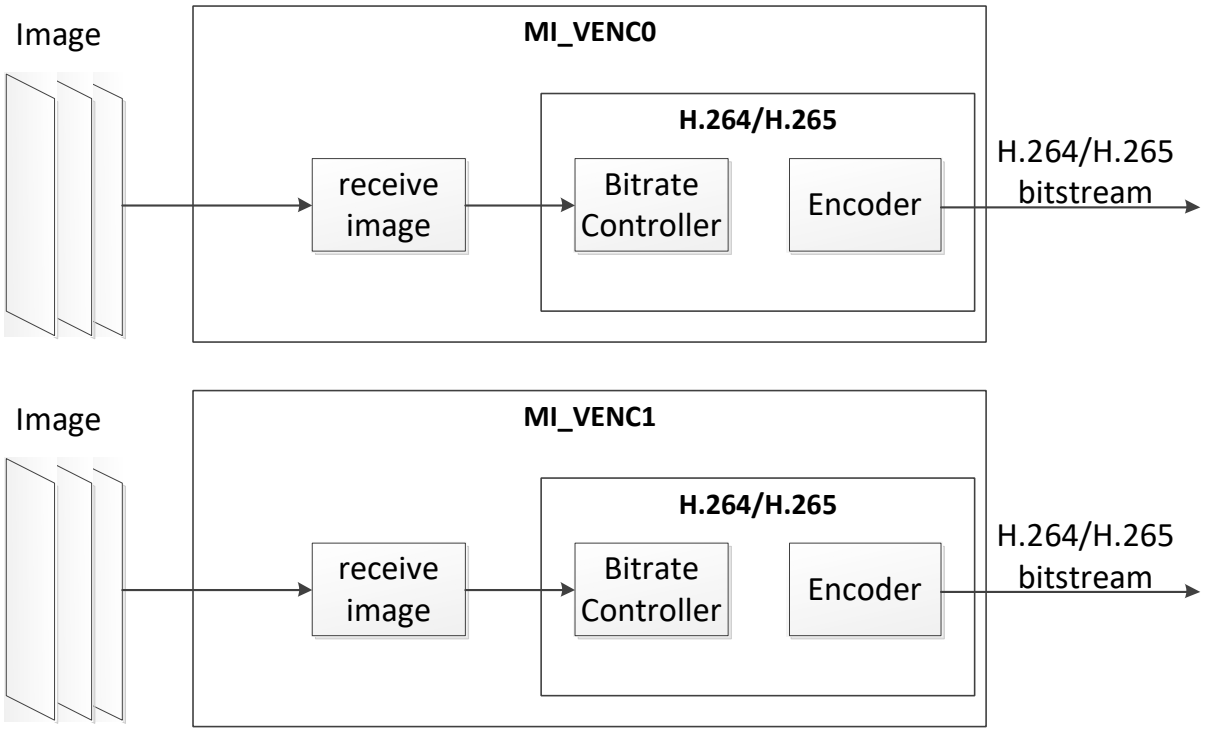


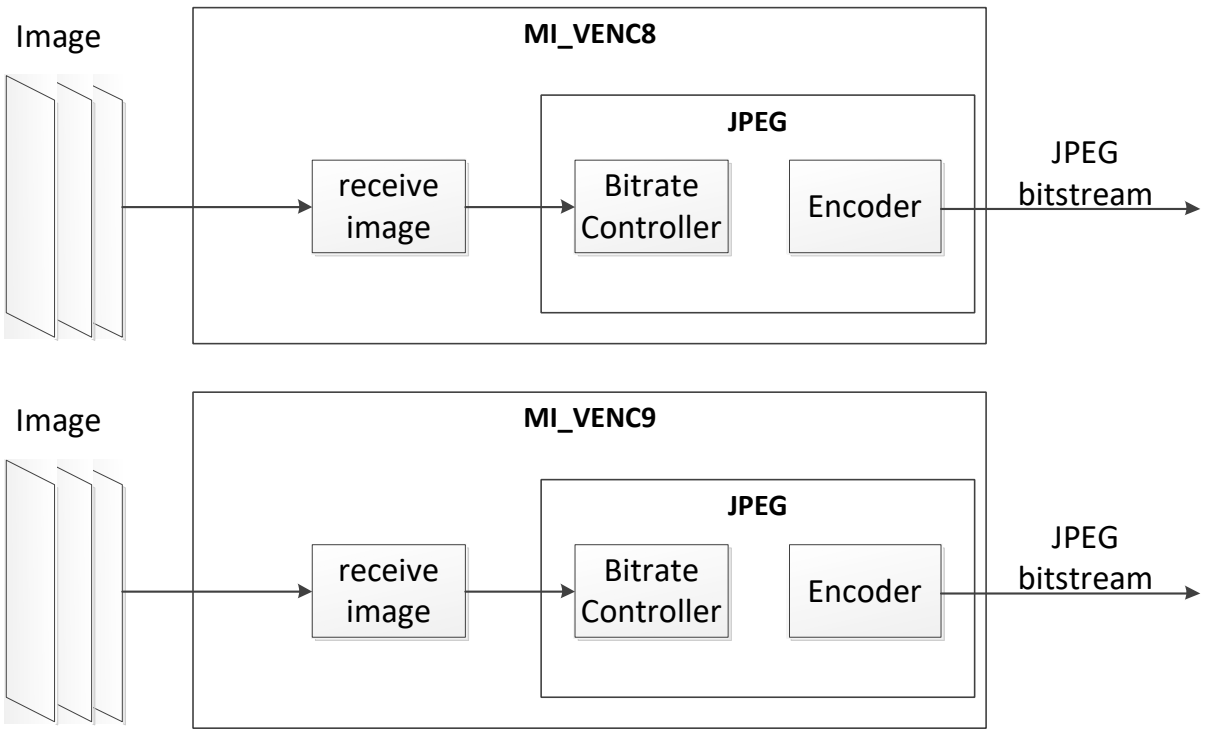
1.2.1 Tiramisu Encoding Process



The device id used to encode H.264/H.265 is MI_VENC_DEV_ID_H264_H265_0.
The device id used to encode JPEG is MI_VENC_DEV_ID_JPEG_0.

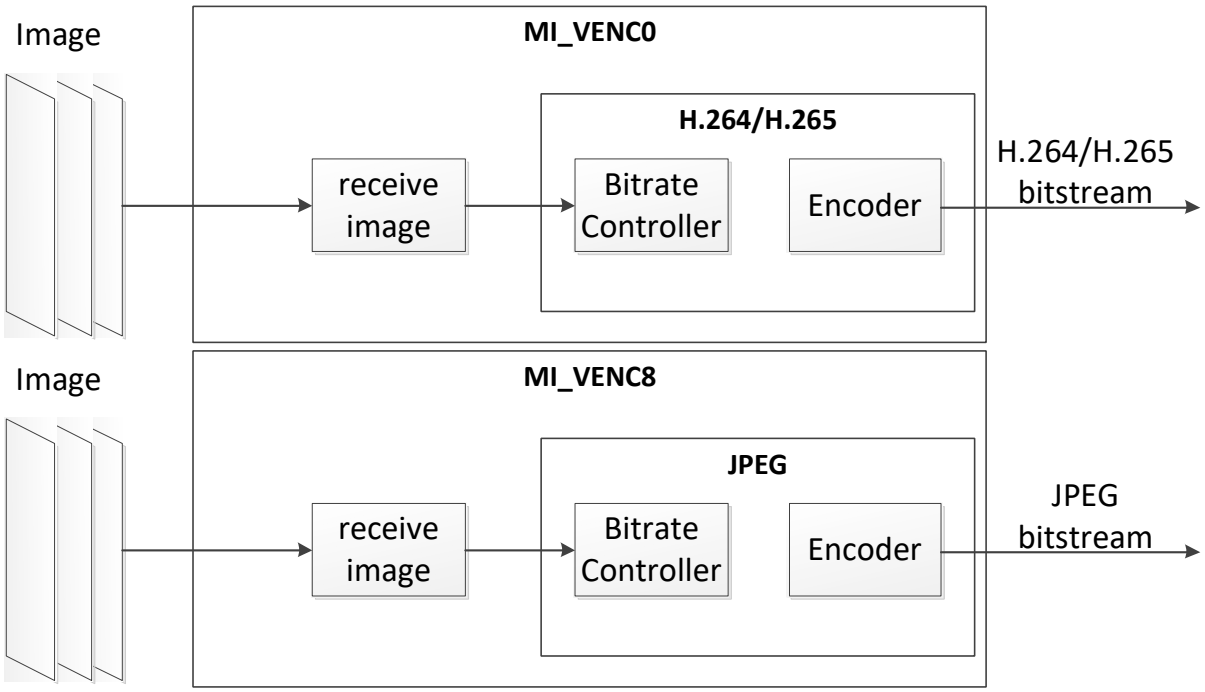
1.2.2 Muffin Encoding Process

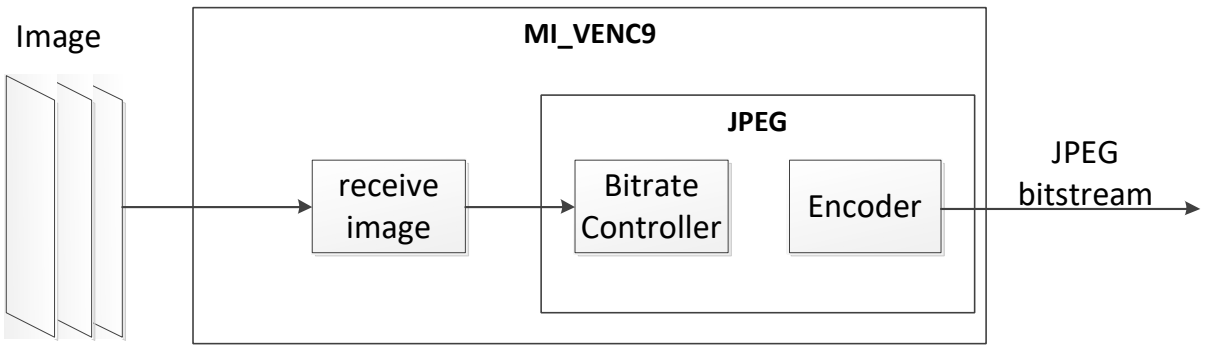




The id of the device used to encode H.264/H.265 is MI_VENC_DEV_ID_H264_H265_0 or MI_VENC_DEV_ID_H264_H265_1;
The id of the device used to encode JPEG is MI_VENC_DEV_ID_JPEG_0 or MI_VENC_DEV_ID_JPEG_1;

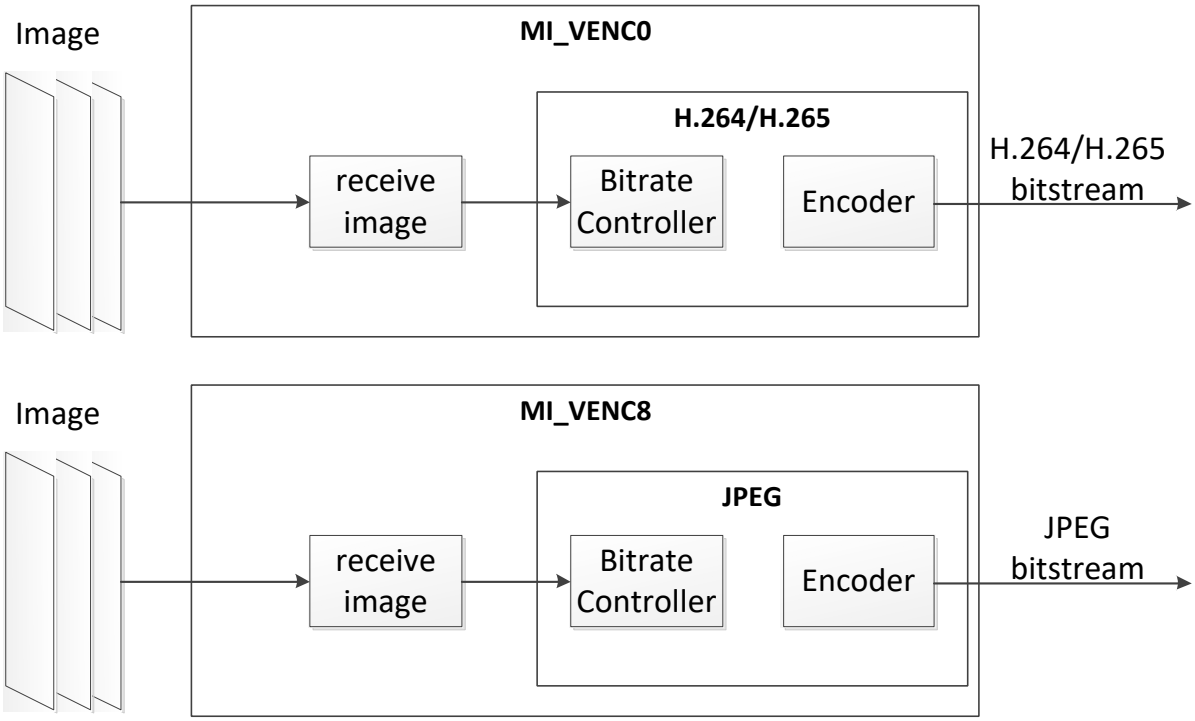
1.2.3 Mochi Encoding Process





The id of the device used to encode H.264/H.265 is MI_VENC_DEV_ID_H264_H265_0;
The id of the device used to encode JPEG is MI_VENC_DEV_ID_JPEG_0 or MI_VENC_DEV_ID_JPEG_1;

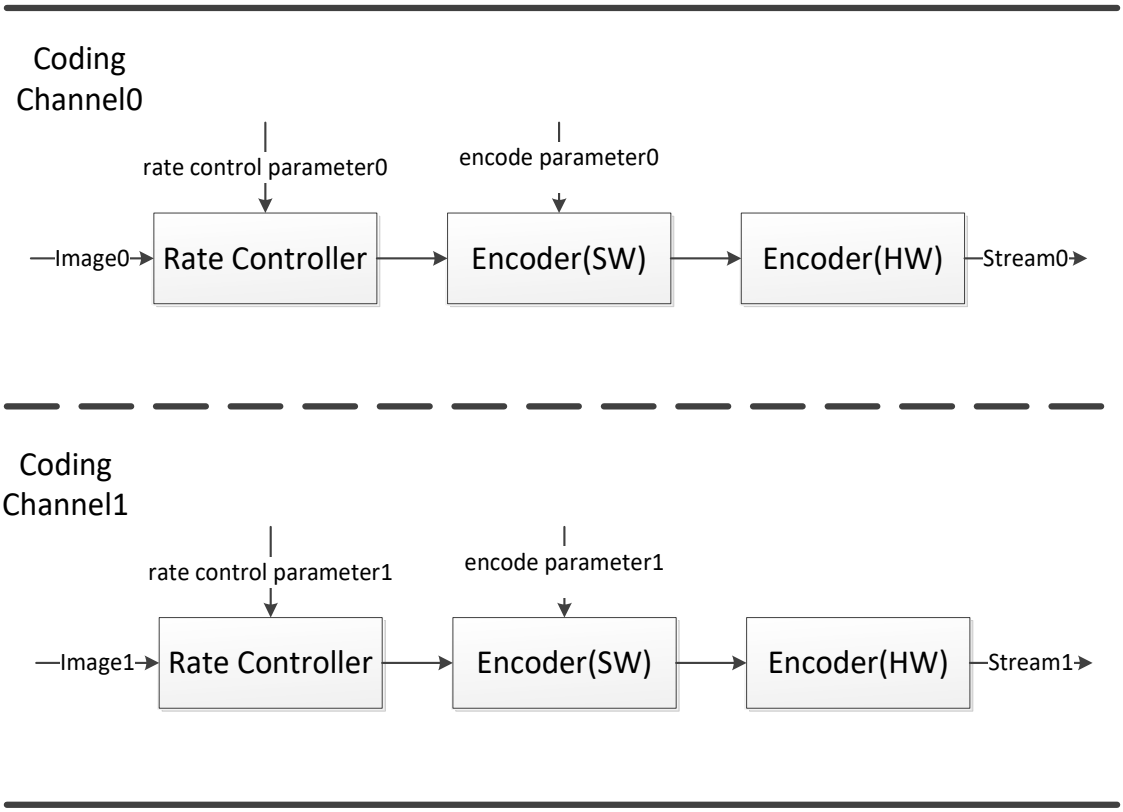
1.2.4 Maruko Encoding Process



The id of the device used to encode H.264/H.265 is MI_VENC_DEV_ID_H264_H265_0;
The id of the device used to encode JPEG is MI_VENC_DEV_ID_JPEG_0;

1.3. Encoding Channel

As a basic control unit, coding channels are completely independent of each other. Encoder (HW) completes the function of converting the original image into a code stream, which is completed by the control of Rate Controller and the Encoder (SW) in cooperation. The Rate Controller and Encoder (SW) of each channel stores all the resources set and managed by all users of the current encoding channel, such as rate control, buffer demand and allocation, etc. The rate control ensures the balance between the output rate and the image quality, while the encoder focuses on the realization of the syntax elements of each codec. Software and hardware control, at the same time, a codec hardware time-sharing multiplexing. Taking two coding channels for example, the basic block diagram of coding channel is as follows:



1.4. Keyword Description

1.4.1 QP

Quantization Parameter. QP value corresponds to the sequence number of quantization step. The smaller the value is, the smaller the quantization step is, and the higher the accuracy of quantization, the better the image quality and the larger the encoded size are.

1.4.2 GOP

Group Of Pictures. The interval between two I frames. The video image sequence consists of one or more GOPs which are independent.

1.4.3 MB

Coding macroblock. Basic unit of H.264 encoding.

1.4.4 CU

Coding Unit. Basic unit of H.265 encoding.

1.4.5 SPS

Sequence Parameter Set. It contains the public information of all images in a GOP.

1.4.6 PPS

Picture Parameter Set. It contains the parameters used to encode an image.

1.4.7 SEI

Supplemental Enhancement information.

1.4.8 ECS

Entropy-coded segment. Compressed image strip after entropy encoding of JPEG.

1.4.9 MCU

Minimum code unit. The basic unit of JPEG encoding.

1.4.10 Rate control

From the perspective of Informatics, the image compression ratio is inversely proportional to the quality: the higher the compression ratio, the lower the quality; the lower the compression ratio, the higher the quality. Taking H.264 as an example, the lower the general image QP, the higher the image quality and the higher the bit rate; the higher the image QP, the lower the image quality and the lower the bit rate. The rate control algorithms supported by different coding protocols are as follows:

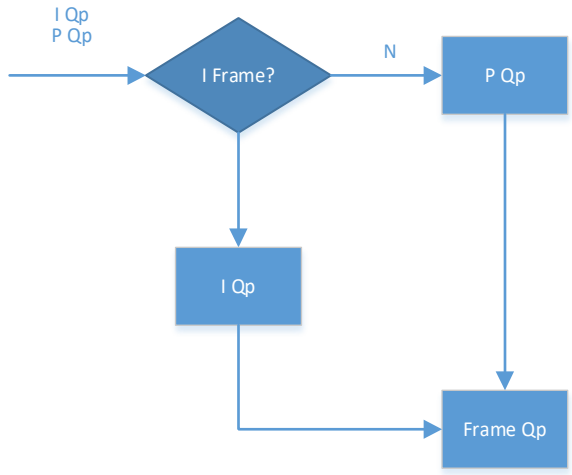
Chip	Rate Control Algorithm			
	FIXQP	CBR	VBR	AVBR
Pretzel	H.264/H.265/JPEG	H.264/H.265/JPEG	H.264/H.265	NONE
Macaron	H.264/H.265/JPEG	H.264/H.265/JPEG	H.264/H.265	NONE
Pudding	H.264/H.265/JPEG	H.264/H.265/JPEG	H.264/H.265	H.264/H.265
Ispahan	H.264/H.265/JPEG	H.264/H.265/JPEG	H.264/H.265	H.264/H.265
Tiramisu	H.264/H.265/JPEG	H.264/H.265/JPEG	H.264/H.265	H.264/H.265
Ikayaki	JPEG	JPEG	None	None
Muffin	H.264/H.265/JPEG	H.264/H.265/JPEG	H.264/H.265	H.264/H.265
Mochi	H.264/H.265/JPEG	H.264/H.265/JPEG	H.264/H.265	H.264/H.265
Maruko	H.264/H.265/JPEG	H.264/H.265/JPEG	H.264/H.265	H.264/H.265

- **FIXQP**
  Fixed Quantization Parameter. At any time point, Qp of all MB/CU of the encoded image can directly be set by the user, and the Qp of I frame and P frame can be set respectively in H.264/H.265. However, some chips cannot set P frame Qp. The behavior of different chips is shown in the following table:

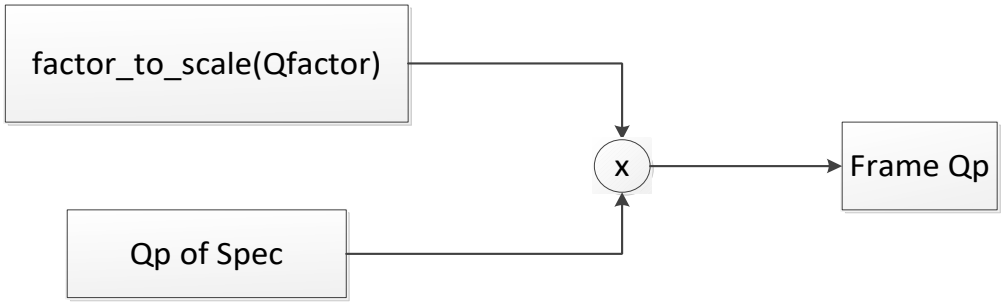
Chip	H.264/H.265	
	Can set I Qp?	Can Set P Qp?
Pretzel	Y	Y
Macaron	Y	Y

Chip	H.264/H.265	
	Can set I Qp?	Can Set P Qp?
Pudding	Y	N
Ispahan	Y	N
Tiramisu	Y	Y
Muffin	Y	Y
Mochi	Y	Y
Maruko	Y	Y

The control flow chart of H.264 and H.265 is as follows:



The control flow chart of JPEG is as follows:



It adopts the Qp table recommended by ITU-t81 K.1. The Qfactor set by the user is converted into a proportional factor according to a certain formula, and the multiplication of the two is the final Qp.

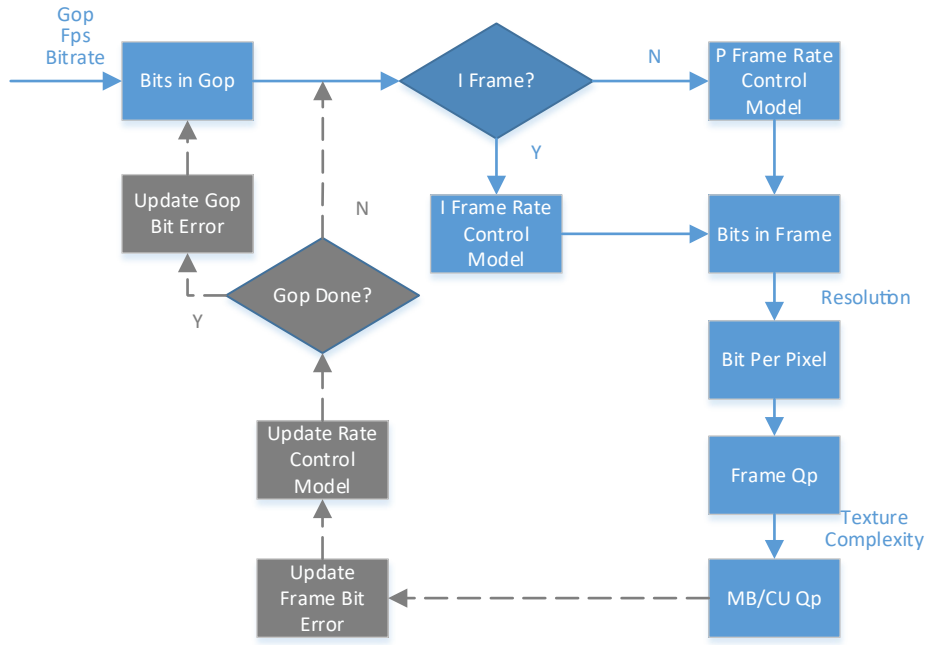
- **CBR**

  Constant Bit Rate. It can guarantee the stability of coding rate during code rate statistics. Because of different prediction methods, the size of I frame and P frame will be significantly different after coding. The bottom statistical time is based on Gop, and the bit accumulation and compensation will be realized between GOPs.

  The main steps are as follows:

1. Convert Fps/Gop/bitrate set by user to bits of each GOP.
2. I/P take different process, and BPP(bitperpixel) is calculated according to GOP bits and resolution.
3. Map BPP to frame QP by rate control model.
4. HW further adjusts MB/Cu QP based on frame QP by image texture complexity and other information.
5. After coding, the rate control model is updated to achieve the continuous stability of the whole sequence, and the bit error is accumulated for the fine-tuning of bit allocation of subsequent frames.
6. The bit error of the whole GOP is accumulated after the completion of the whole GOP, which is used for fine tuning of the next GOP bit allocation.

The control flow chart is as follows:



- **VBR**

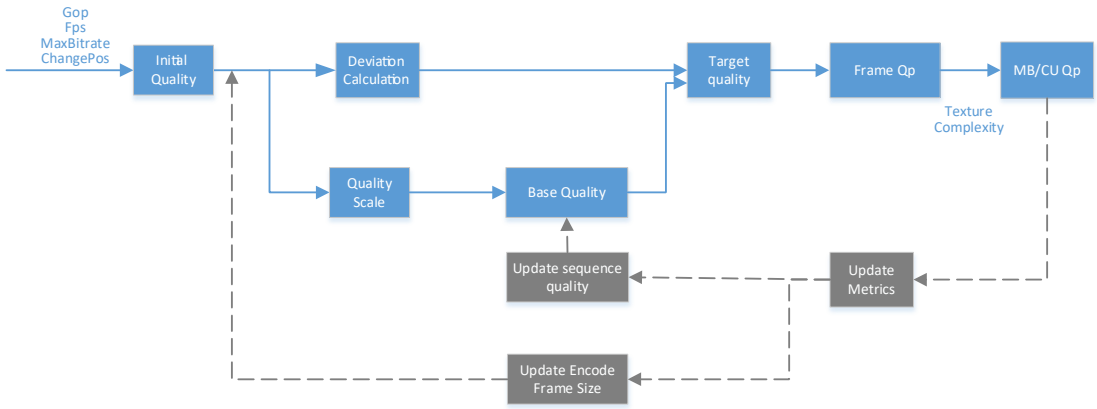
Variable Bit Rate. It allows the code rate to fluctuate within the code rate statistics time, so as to ensure the stable quality of the coding image. Take H.264 for example, users can set MaxQp, MinQp, MaxBitRate and ChangePos, details of which can be found in MI_VENC_ParamH264Vbr_t. MaxQp and MinQp are used to control the quality range of images. MaxBitRate is used for limiting maximum code rate within the statistics time. ChangePos is a percentage of the maximum bit rate, which is used to control the code rate reference line of starting to adjust QP, that is when the code rate is greater than MaxBitRate*ChangePos, the image QP will gradually adjust to MaxQp, if the image QP has reached MaxQp, the image QP will be clamped to the maximum value, the clamping effect of MaxBitRate will be lost, and the code rate may exceed MaxBitRate; if the code rate is less than MaxBitRate*ChangePos, the image QP will gradually adjust to MinQp. If the image QP has reached MinQp, the code rate has reached the maximum value and the image quality is best.

The specific process is as follows:

1. Convert the Fps/Gop/MaxBitRate/ChangePos and resolution set by user into the initial quality benchmark in order to set starting QP of the sequence.
2. HW further adjusts MB/CU QP based on frame QP by image texture complexity and other information.

3. Update rate control model after coding.
4. Update the quality of the overall sequence.
5. Calculate the deviation coefficient according to the difference between the current expected bit and the actual encoded bit, and decide whether the quality can be improved or reduced.
6. Calculate the target quality of the current frame by the deviation coefficient and two quality coefficients.
7. Map the target quality to frame QP, looping from 2.

The control flow chart is as follows:



- **AVBR**
  Adaptive Variable Bit Rate. It allows the code rate to fluctuate within the code rate statistics time, so as to ensure the stable quality of the coding image. The rate control will detect the static state of the current scene, adopt higher rate coding when moving, and actively reduce the rate when still. Take H.264 for example, users can set MaxBitRate, ChangePos and MinStillPercent, details of which can be found in MI_VENC_ParamH264Avbr_t. MaxBitRate represents the maximum code rate in the motion scene, Minstitlpercent is the percentage of the minimum bit rate relative to the adjusted threshold bit rate in the static scene, MaxBitRate*ChangePos*MinStillPercent represents the minimum code rate in the static scene, and the target code rate will be adjusted between the minimum code rate and the maximum code rate according to the different degree of motion. MaxQp and MinQp are used to control the quality range of the image. The highest priority of the rate control is QP clamp. The rate control beyond MinQp and MaxQp will fail.

1.4.11 QPMAP

In qpmap mode, users are allowed to decide the rate control strategy freely, and absolute qpmap and relative qpmap are supported. Please refer to MI_VENC_AllocCustomMap.

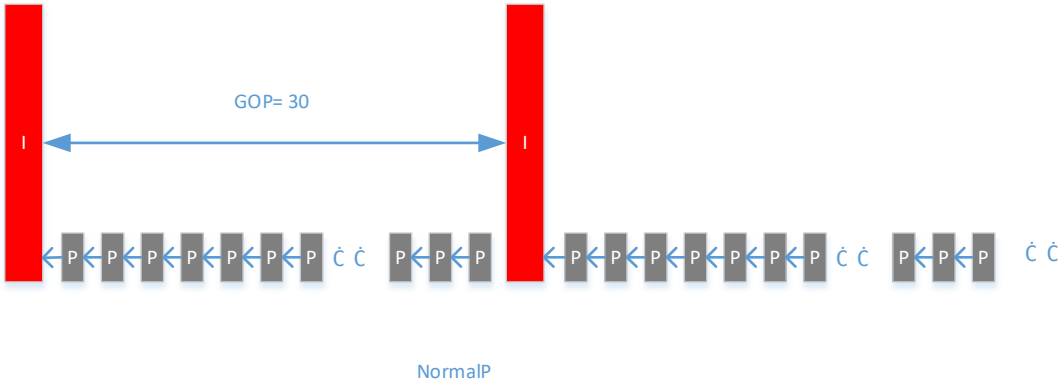
1.4.12 Reference frame structure

H. 264/H.265 only supports one reference frame in a single frame, but the whole stream supports multiple reference frame buffers. For example: in LTR/TSVC3 mode, each P can only refer to one, but at most two reference frames will be saved for different P frames.

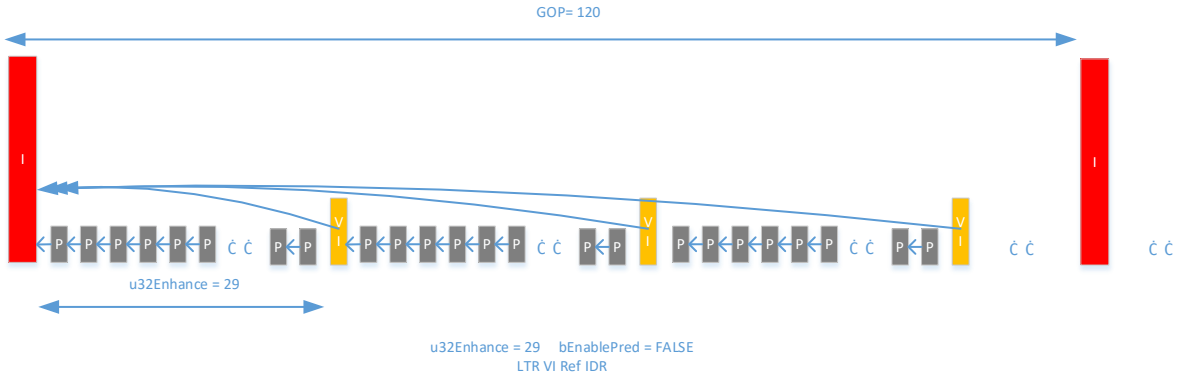
The reference frame supports 5 modes: NormalP, LTR (VI Ref IDR), LTR (VI Ref VI), TSVC-2 and TSVC-3. All

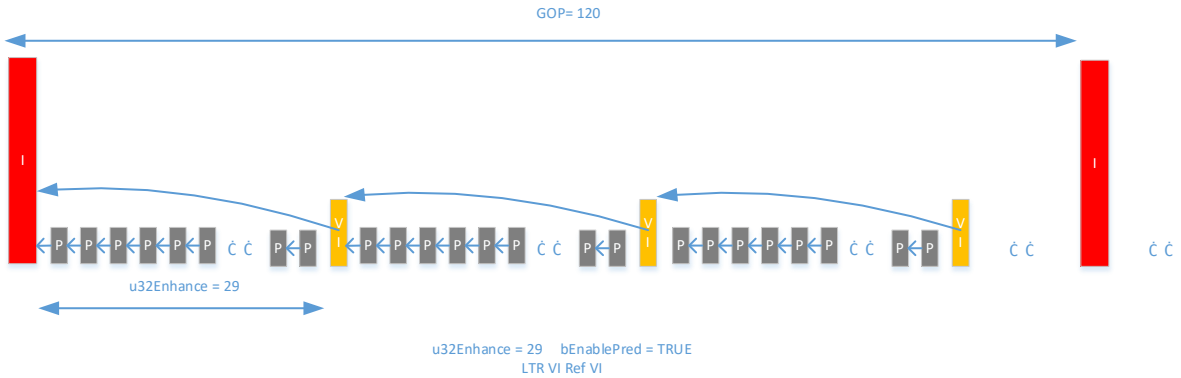
reference frame structures are controlled by three parameters: u32Base, u32Enhance and bEnablePred. Please refer to MI_VENC_ParamRef_t for specific meanings. When u32Enhance is set to 0, it will be converted to NormalP reference frame structure. For other structures, please refer to the corresponding structure chart. The default structure is NormalP reference frame structure. The following details each reference diagram and parameter setting.

- **NormalP**
  NormalP is the most basic and common reference relationship. P frame directly refers to the previous one, and its structure diagram is shown in the following figure:



- **LTR mode**
  LTR (Long Term Reference) uses long-term reference frame and short-term reference frame to realize the same frame can be referred to multiple frames. This mode defines one special P as virtual I frame (VI). Compared with all IDR, using VI can reduce the code rate while maintaining a certain error recovery power. At present, LTR supports two modes: all VI refers to the latest IDR and VI refers to the previous VI or IDR (in the case of the first VI). The structures of the two reference modes are as follows:

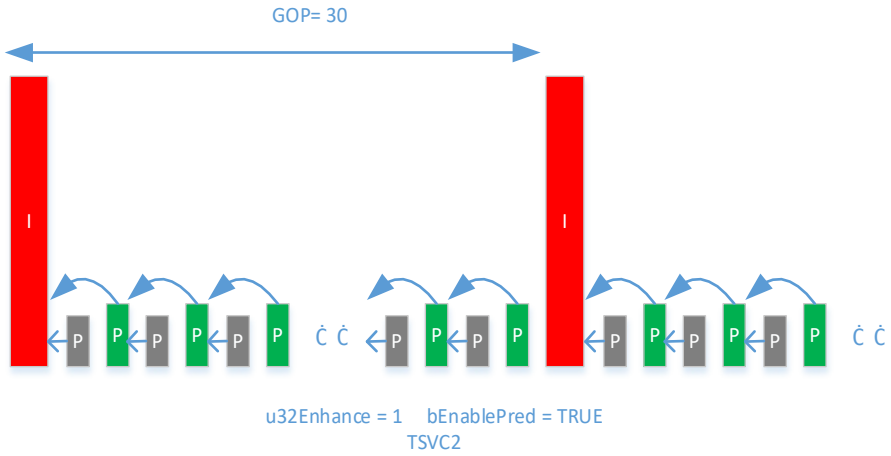




Note: an extra buffer is required to enable LTR mode.

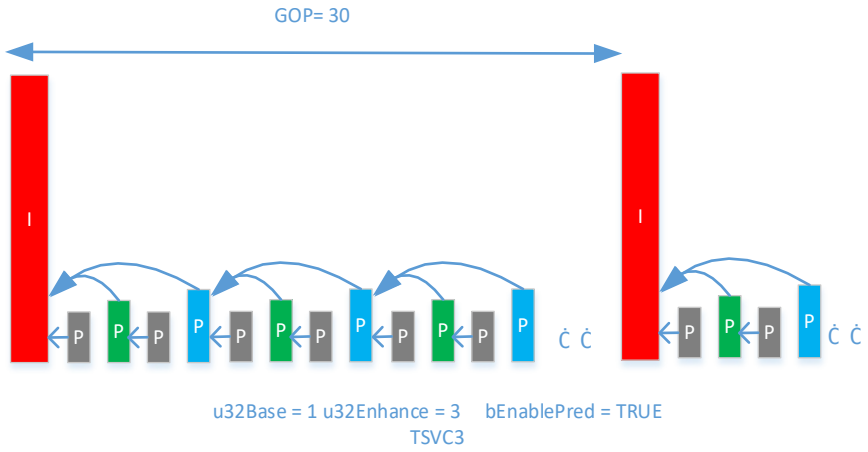
- **TSVC-2**

  TSVC-2 provides two-layers coding, and the frame rate can vary between 1/2 and full. The structure diagram is as follows:



- **TSVC-3**

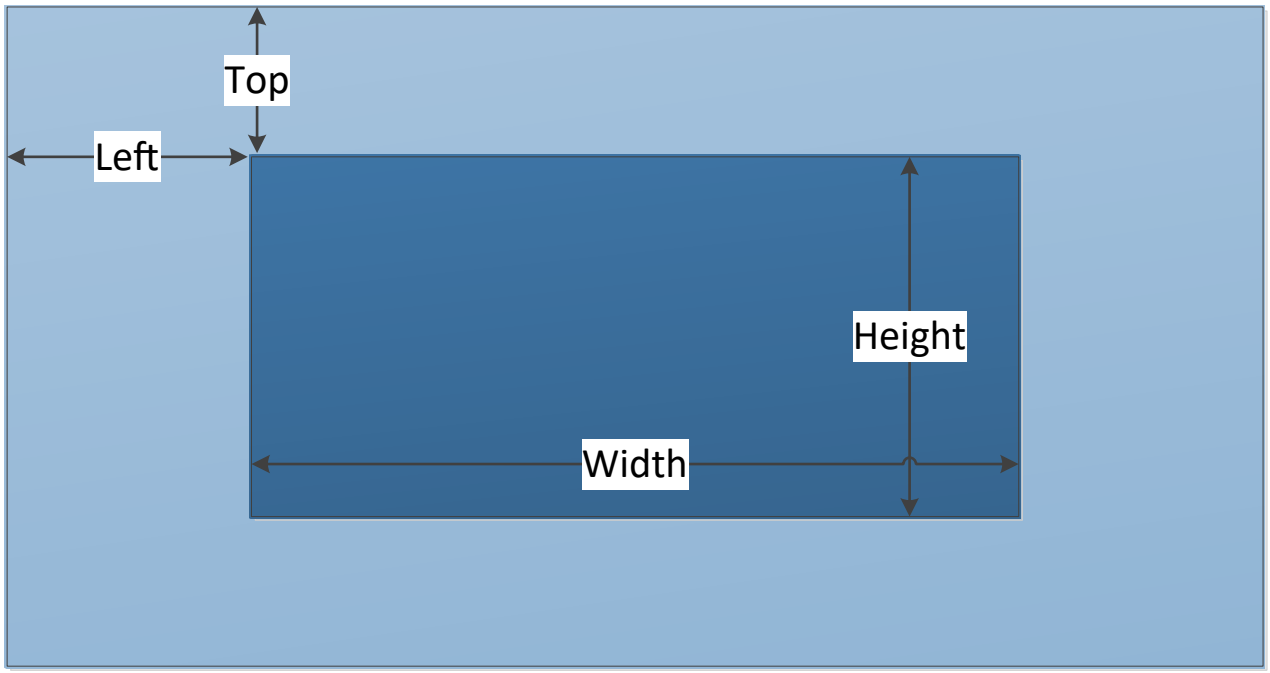
  TSVC-3 provides three-layer coding, and the frame rate can vary between 1/4, 1/2, 3/4 and full. The structure diagram is as follows:



Note: an extra buffer is required to enable TSVC-3 mode.

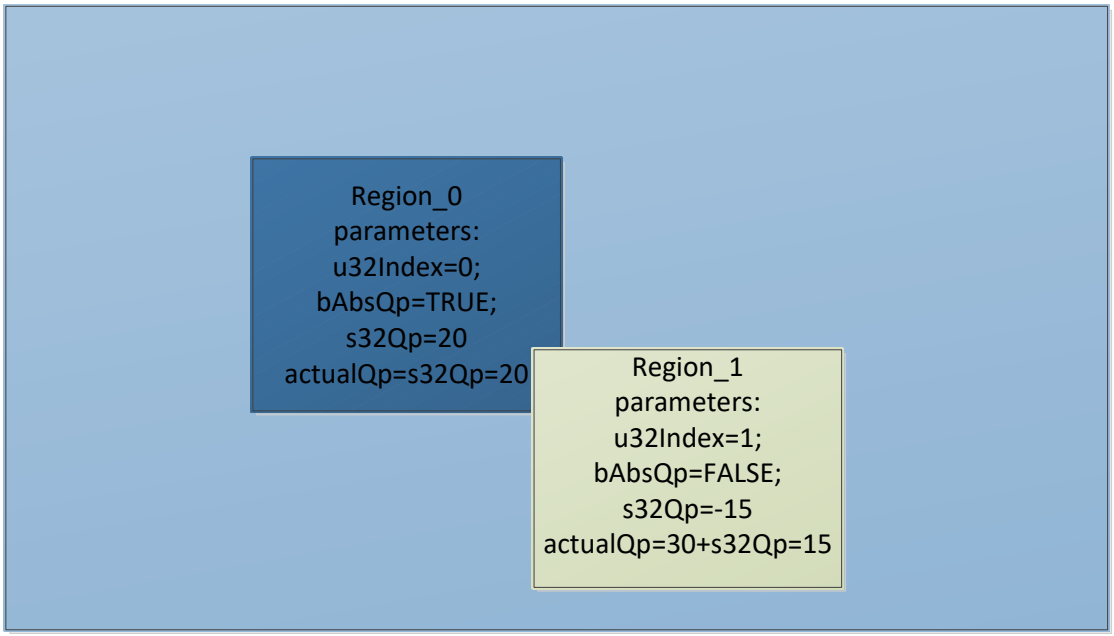
1.4.13 Crop

Crop, that is, cutting out a part of the image for encoding. User can set the starting point of clipping left/top, width and height, please refer to relevant API: MI_VENC_SetCrop. The schematic diagram is as follows:



1.4.14 ROI

Region Of Interest. Region of interest coding, users can configure ROI region to limit the image QP of this region, so as to realize the difference between the QP of this region and other image regions. The system supports H264 and h265 to set ROI, and provides 8 ROI areas for users to use at the same time.
Eight ROI regions can be overlapped with each other, and the priority of the overlapped region is raised in sequence according to the index number of 0-7, that is to say, the QP of the overlapped region is finally determined to be processed only according to the region with the highest priority. The ROI area can be configured with absolute QP and relative QP:
Absolute QP: QP in ROI area is the QP value set by the user.
Relative QP: the QP in ROI area is the sum of the QP generated by rate control and the QP offset value set by the user.
In the following example, the encoded image adopts the fixqp mode, setting the image QP to 30, that is, the QP value of all macroblocks of the image is 30. ROI region 0 is set to absolute QP mode, QP value is 20, index is 0; ROI region 1 is set to relative QP mode, QP is -15, index is 1. Because the index of ROI region 0 is smaller than the index of ROI region 1, the QP of ROI region 1 with high priority is set in the overlapped image region. The QP value of region 1 is 30-15 = 15, and the QP value of region 0 except the overlapped image region is 20.



1.4.15 Low frame rate encode in non ROI area

Low frame rate encode in non ROI area, that is to say, the ROI area is normally encoded after the ROI is turned on, while the non ROI area can reduce the frame rate by setting the proportional relationship between the source and the target. According to the scale relationship, the non ROI area of some frames in a GOP will directly use the data of the corresponding area of the previous frame. The user can set the relative frame rate of non ROI area according to the actual situation, and only when the ROI is turned on can it take effect. Please refer to API:MI_VENC_SetRoiBgFrameRate,s32srcFrmRate and s32dstFrmRate only represent a proportional relationship, independent of the actual frame rate.

1.4.16 Bind Type

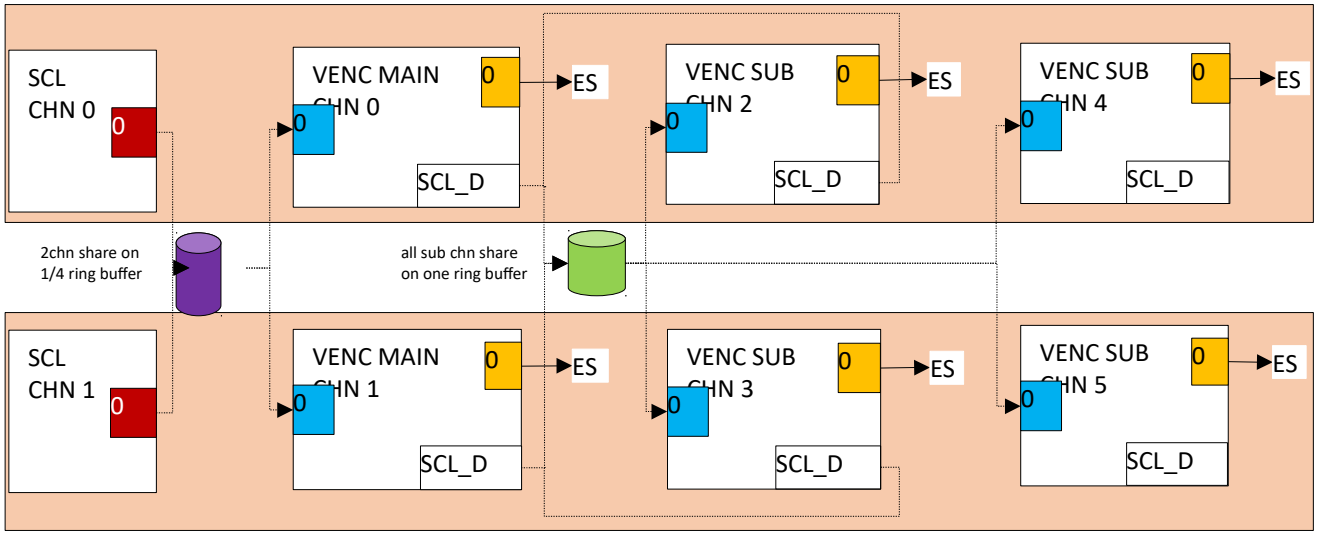
Generally speaking, the bind type between front level and VENC is E_MI_SYS_BIND_TYPE_FRAME_BASE, that is, the front level completes a frame buffer completely, and then outputs it to the Venc. In this mode, at least three frame buffers need to be allocated. If the front level is SCL, there are two bind types which can save memory: E_MI_SYS_BIND_TYPE_HW_RING and E_MI_SYS_BIND_TYPE_REALTIME.
E_MI_SYS_BIND_TYPE_HW_RING means that SCL and VENC can read and write on the same one frame buffer in same time. In addition, Ispahan also supports reading and writing on half frame buffer; please refer to MI_VENC_SetInputSourceConfig for details.
E_MI_SYS_BIND_TYPE_REALTIME means that there is no need to allocate additional DRAM between SCL and VENC, because VPE and VENC are directly hardware connected, when the codec type is JPEG, and the input image YUV format is NV12, it is necessary to ensure that the width of the image is not greater than 1792. It is recommended to use YUYV422 as the input image YUV format.

Different chip and encode protocols support different bind type as follows:

Chip	Codec	Bind Type		
		FRAME_BASE	HW_RING	REALTIME
Pretzel	H.264/H.265	Y	N	N

Chip	Codec	Bind Type		
		FRAME_BASE	HW_RING	REALTIME
	JPEG	Y	N	N
Macaron	H.264/H.265	Y	Y	N
	JPEG	Y	N	Y
Pudding	H.264/H.265	Y	N	N
	JPEG	Y	Y	Y
Ispahan	H.264/H.265	Y	Y	N
	JPEG	Y	N	Y
Tiramisu	H.264/H.265	Y	Y	N
	JPEG	Y	N	Y
Ikayaki	JPEG	Y	Y	Y
Muffin	H.264/H.265	Y	Y	N
	JPEG	Y	N	Y
Mochi	H.264/H.265	Y	Y	N
	JPEG	Y	N	Y
Maruko	H.264/H.265	Y	Y	N
	JPEG	Y	Y	Y

Note:
In order to further save memory, Maruko platform can configure E_MI_SYS_BIND_TYPE_HW_RING binding type between SCL and VENC when encoding H264 or h265, and set buffer mode to E_MI_VENC_INPUT_MODE_RING_UNIFIED_DMA with MI_VENC_SetInputSourceConfig. When SCL enables AFBC function, the minimum frame buffer shared by SCL and VENC can be set to one quarter. If the AFBC function is not turned on, the minimum frame buffer can be set to half sheet. We need to call MI_SYS_ConfigPrivateMMAPool to configure the frame buffer when initializing SCL, please refer to SYS API introduction for specific usage methods. If the YUV output from two SCL channels is inconsistent, the frame buffer should be allocated according to the largest output size. At this case, the subsequent stage of the VENC main channel bound to SCL can cascade VENC sub channels. At present, it supports up to 2-level sub streams. The bind type between VENC channels need to set E_MI_SYS_BIND_TYPE_HW_RING, and there is no need to call MI_VENC_SetInputSourceConfig for additional settings. Each sub channel of VENC shares a frame buffer for YUV data stream transmission, which only supports the reading and writing of the whole sheet. Since the VENC cascade only supports reduction, and the output size of the subsequent channel must be less than or equal to the output of the previous channel, the frame buffer can be configured once by calling the MI_SYS_ConfigPrivateMMAPool interface by VENC main channel and according to the maximum output size in the subchannel. The working model is as follows:



1.4.17 Output buffer configuration of coded bitstream

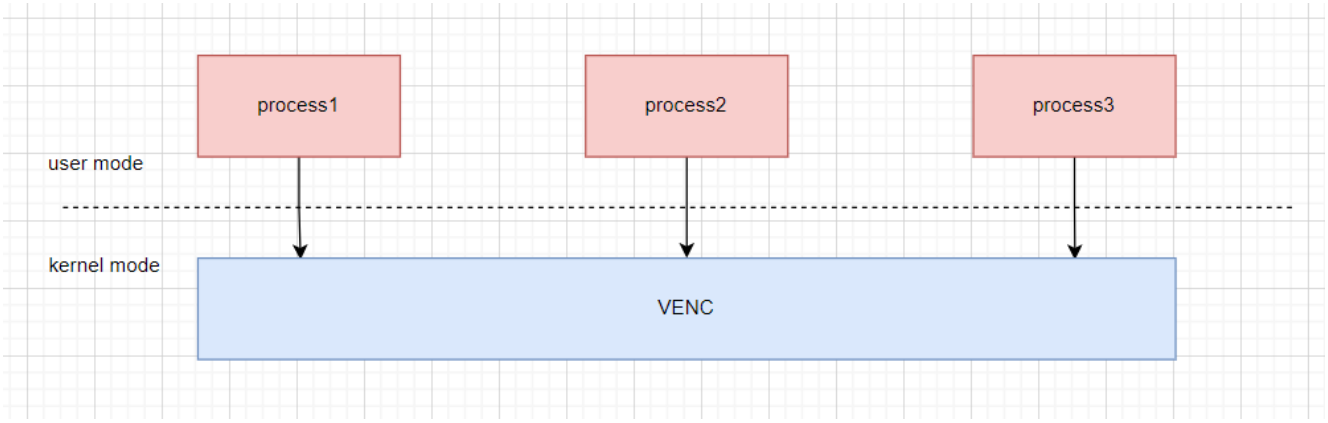
Different chips support different configurations due to different SW architectures. The default configurations are as follows:

Chip	H.264	H.265	JPEG
Pretzel	WidthxHeight/2	WidthxHeight/2	WidthxHeight
Macaron	Ring pool: WidthxHeight/2	Ring pool: WidthxHeight/2	Ring pool: WidthxHeight/2
Pudding	Ring pool: WidthxHeight/2	Ring pool: WidthxHeight/2	Ring pool: WidthxHeight/2
Ispahan	Ring pool: WidthxHeight/2	Ring pool: WidthxHeight/2	Ring pool: WidthxHeight/2
Tiramisu	Ring pool: WidthxHeight/2	Ring pool: WidthxHeight/2	Ring pool: WidthxHeight/2
Ikayaki	Ring pool:WidthxHeight/2	Ring pool:WidthxHeight/2	Ring pool:WidthxHeight/2
Muffin	Ring pool:WidthxHeight/2	Ring pool:WidthxHeight/2	Ring pool:WidthxHeight/2
Mochi	Ring pool:WidthxHeight/2	Ring pool:WidthxHeight/2	Ring pool:WidthxHeight/2
Maruko	Ring pool:WidthxHeight/2	Ring pool:WidthxHeight/2	Ring pool:WidthxHeight/2

The above is the default output buffer configuration of each chip. User can specify the output buffer configuration by MI_VENC_CreateChn. Take H.264 as an example, user can specify the output buffer configuration by setting stVeAttr.stAttrH264e.u32BufSize,if the value is 0, the default configuration is used. In Pretzel, frame mode was used, that is to say, for each stream of coding, a specific size buffer will be allocated, generally the size used in the actual encoding will be smaller than the allocated size, which will cause memory fragmentation; while in the ring pool mode, each channel will separately allocate an output ring pool, and each coding will obtain the maximum free buffer from the pool, and then record the size of the actual coding used, and recycle output ring pool, which effectively improves the utilization of the buffer and reduces memory fragmentation.

Note: Under Pudding/Tiramisu/Ikayaki/Muffin Chip, there will be two output pools inside jpe to support realtime mode full frame output, otherwise only half of the front-end mi scl output frame rate. If jpe only has frame mode, you can add max_jpe_task=1 after insmod mi_venc.ko to force an output pool to save memory.

1.4.18 Description of scenarios supported by multiple processes



Currently only Mochi's platform supports multi-process coding.

Supporting scenarios:
1、 Support the same device, different channel multi-process coding.
For example:
Process1 creates the Device0, channel 0 encoding. Process2 creates the Device0, channel1 encoding. Process3 creates device0, channel2 encoding.

2、 Support multi-process coding for different devices.
For example:
Process1 creates device0, channel0 encoding. Process2 creates the Device8, channel0 encoding. Process3 created the Device9, channel0 encoding.

3、 Support to obtain and set parameters in process 2 other than process 1.
For example:
Process1 creates the Device0, channel0 encoding and device1, channel0 encoding. Process2 creates the Device0, channel1 encoding and device1, channel1 encoding. You can get and set parameters of device0's channel0 and channel1 code channels and Device1's channel0 and channel1 code channels in process3. For example process3 calls the MI_VENC_SetChnAttr function to set the encoding channel properties of Device0, channel0.
Note: Resources created in process 1, such as Device0 and channel0, need to be destroyed in process 1.

Unsupported scenarios:
1、 Streaming and fetching of coded channels created by process 1 in process 2 is not supported.
For example:
Process1 creates Device0, channel0, but does not stream. Process2 sends and receives streams from Device0 and channel0.

2、 The front-level module channels created by process 1 do not support binding to the VENC channels created by process 2. Streams need to ensure that the front-level and front-level channels are created in the same process.
For example:
Process1 creates the Device1, channel0 channel of SCL. Process2 created the Device0, channel0 channel for VENC. Binding Device1, channel0, port0 of SCL to Device0, channel0, port0 of VENC in Process1 or Process2 is

not supported. If you want to stream, you need to ensure that both the front and back level channels of the binding are created in the same process.

2. API REFERENCE

API name	Features
MI_VENC_CreateDev	Create venc device
MI_VENC_DestroyDev	Destroy venc device
MI_VENC_CreateChn	Create a code channel
MI_VENC_DestroyChn	Destroy code channel
MI_VENC_ResetChn	Reset code channel
MI_VENC_StartRecvPic	Turn on the encoding channel to receive the input image
MI_VENC_StartRecvPicEx	Turn on the encoding channel to receive the input image, and automatically stop receiving images after the specified number of frames is exceeded.
MI_VENC_StopRecvPic	Stop encoding channel to receive input image
MI_VENC_Query	Query code channel status
MI_VENC_SetChnAttr	Set the encoding properties of the encoding channel
MI_VENC_GetChnAttr	Get the encoding attribute of the encoding channel
MI_VENC_GetStream	Get the encoded code stream.
MI_VENC_ReleaseStream	Release stream buffer
MI_VENC_InsertUserData	Insert user data
MI_VENC_SetMaxStreamCnt	Set the maximum stream cache frame number
MI_VENC_GetMaxStreamCnt	Get the maximum number of stream cache frames
MI_VENC_RequestIdr	Request IDR frame
MI_VENC_EnableIdr	Enable IDR frame
MI_VENC_SetH264IdrPicId	Set the idr_pic_id of the IDR frame
MI_VENC_GetH264IdrPicId	Get the idr_pic_id of the IDR frame
MI_VENC_GetFd	Obtain the device file handle corresponding to the encoding channel
MI_VENC_CloseFd	Close the device file handle corresponding to the encoding channel
MI_VENC_SetRoiCfg	Set the region of interest encoding configuration for the encoding channel
MI_VENC_GetRoiCfg	Get the region of interest encoding configuration for the encoding channel
MI_VENC_SetRoiBgFrameRate	Set the frame rate configuration of the non-interest area of the encoding channel

API name	Features
MI_VENC_GetRoiBgFrameRate	Get the frame rate configuration of the non-interest area of the encoding channel
MI_VENC_SetH264SliceSplit	Set the slice split configuration of H.264 encoding
MI_VENC_GetH264SliceSplit	Get the slice split configuration of H.264 encoding
MI_VENC_SetH264InterPred	Set the interframe prediction configuration of H.264 encoding
MI_VENC_GetH264InterPred	Get the interframe prediction configuration of H.264 encoding
MI_VENC_SetH264IntraPred	Set the intra prediction configuration of H.264 encoding
MI_VENC_GetH264IntraPred	Get the intra prediction configuration of H.264 encoding
MI_VENC_SetH264Trans	Set the transform and quantization configuration of H.264 encoding
MI_VENC_GetH264Trans	Get the transform and quantization configuration of H.264 encoding
MI_VENC_SetH264Entropy	Set the entropy encoding configuration of H.264 encoding
MI_VENC_GetH264Entropy	Get the entropy encoding configuration of H.264 encoding
MI_VENC_SetH265InterPred	Set the interframe prediction configuration of H.265 encoding
MI_VENC_GetH265InterPred	Get the interframe prediction configuration of H.265 encoding
MI_VENC_SetH265IntraPred	Set H.265 encoded intra prediction configuration
MI_VENC_GetH265IntraPred	Get H.265 encoded intra prediction configuration
MI_VENC_SetH265Trans	Set H.265 coded transform, quantization configuration
MI_VENC_GetH265Trans	Get H.265 encoded transform, quantization configuration
MI_VENC_SetH264Dblk	Set the deblocking configuration of H.264 encoding
MI_VENC_GetH264Dblk	Get the deblocking configuration of H.264 encoding
MI_VENC_SetH265Dblk	Set the deblocking configuration of H.265 encoding
MI_VENC_GetH265Dblk	Get the deblocking configuration of H.265 encoding
MI_VENC_SetH264Vui	Set H.264 encoded VUI configuration
MI_VENC_GetH264Vui	Get H.264 encoded VUI configuration
MI_VENC_SetH265Vui	Set H.265 encoded VUI configuration
MI_VENC_GetH265Vui	Get H.265 encoded VUI configuration
MI_VENC_SetH265SliceSplit	Set the slice split configuration of H.265 encoding
MI_VENC_GetH265SliceSplit	Get the slice split configuration of H.265 encoding
MI_VENC_SetJpegParam	Set the JPEG encoded parameter set
MI_VENC_GetJpegParam	Get the JPEG encoded parameter set
MI_VENC_SetRcParam	Set channel rate control advanced parameters
MI_VENC_GetRcParam	Get channel rate control advanced parameters
MI_VENC_SetRefParam	Set H.264/H.265 encoding channel advanced frame skipping

API name	Features
	reference parameters
MI_VENC_GetRefParam	Get advanced frame skipping reference parameters for H.264/H.265 encoding channel
MI_VENC_SetCrop	Set the cropping properties of VENC
MI_VENC_GetCrop	Get the cropping properties of VENC
MI_VENC_SetFrameLostStrategy	Set the configuration of the drop frame policy when the instantaneous bit rate exceeds the threshold.
MI_VENC_GetFrameLostStrategy	Obtain the configuration of the frame loss policy when the instantaneous bit rate exceeds the threshold.
MI_VENC_SetSuperFrameCfg	Set oversized frame processing configuration
MI_VENC_GetSuperFrameCfg	Get large frame processing configuration
MI_VENC_SetRcPriority	Set the priority type of rate control
MI_VENC_GetRcPriority	Get the priority type of rate control
MI_VENC_AllocCustomMap	Allocate the memory for Custom Map used by smart encoding
MI_VENC_ApplyCustomMap	Apply the configured Custom Map
MI_VENC_GetLastHistoStaticInfo	Get the output information when the latest frame is encoded
MI_VENC_ReleaseHistoStaticInfo	Release the memory used for the output information when the latest frame is encoded
MI_VENC_ SetAdvCustRcAttr	Set the customer defined advanced RC configuration parameters
MI_VENC_SetInputSourceConfig	Set H.264/H.265 input source info
MI_VENC_SetSmartDetInfo	Set smart detector information
MI_VNEC_SetIntraRefresh	Set H.264/H.265 P-frame brush Islice
MI_VNEC_GetIntraRefresh	Get H.264/H.265 P-frame brush Islice
MI_VENC_DupChn	Synchronize status of venc channel

2.1. MI_VENC_CreateDev

- Description

  Create VENC device.

- Syntax

  MI_S32 MI_VENC_CreateDev(MI_VENC_DEV VeDev, MI_VENC_InitParam_t *pstInitParam);

- Parameters

Parameter	Description	Input/Output
VeDev	Encode device number.	Input
pstInitParam	Initialization Parameter pointer.	Input

- Return value
  - MI_SUCCESS: Successful
  - Not MI_SUCCESS: Failed, see ERROR CODE for details.

- Dependency
  - Header file: mi_venc.h, mi_venc_datatype.h
  - Library file: libmi_venc.a/libmi_venc.so

- Note
  - This function does not need to be called, but MI_VENC_CreateChn needs to specify a valid device id, and the corresponding device will be created internally.
  - This interface should be used in pair with MI_VENC_DestroyDev; otherwise a failed message will be returned.

- Example

  None.

- Related topic

  None.

2.2. MI_VENC_DestroyDev

- Description

  Destroy venc device.

- Syntax

  MI_S32 MI_VENC_DestroyDev(MI_VENC_DEV VeDev);

- Parameters

Parameter	Description	Input/Output
VeDev	Encode device number	Input

- Return value
  - MI_SUCCESS: Successful
  - Not MI_SUCCESS: Failed, see ERROR CODE for details.

- Dependency
  - Header file: mi_venc.h, mi_venc_datatype.h
  - Library file: libmi_venc.a/libmi_venc.so

- Note
  - This interface should be called after device has been initialized; otherwise a failed message will be returned.
  - If this interface has not been called after the app exited, venc device will be auto de-initialized.
  - This interface should be used in pair with MI_VENC_CreateDev; otherwise a failed message will be returned.

- This interface parameter setting only takes effect on Pudding, Ispahan and Tiramisu.

➢ Example

None.

➢ Related topic

None.

2.3. MI_VENC_CreateChn

➢ Features

Create a code channel.

➢ Syntax

MI_S32 MI_VENC_CreateChn(MI_VENC_DEV VeDev, MI_VENC_CHN VeChn, MI_VENC_ChnAttr_t*pstAttr);

➢ Parameter

Parameter name	Description	Input/Output
VeDev	Encode device number	Input
VeChn	Encode channel number. Value range: [0, VENC_MAX_CHN_NUM).	Input
pstAttr	Coding channel property pointer.	Input

➢ Return Value

- MI_SUCCESS: Successful
- Not MI_SUCCESS: Failed, see ERROR CODE for details.

➢ Requirement

- Header files: mi_venc.h, mi_venc_datatype.h
- Library file: libmi_venc.a/libmi_venc.so

➢ Chip Difference

The supported coding specifications of different chip are shown in the following table:

Chip	H.264			H.265	JPEG
	Baseline Profile	Main Profile	High Profile	Main Profile	Baseline Profile
Pretzel	Y	Y	N	Y	Y
Macaron	Y	Y	N	Y	Y
Pudding	Y	Y	Y	Y	Y
Ispahan	Y	Y	Y	Y	Y
Tiramisu	Y	Y	Y	Y	Y
Ikayaki	N	N	N	N	Y

Muffin	Y	Y	Y	Y	Y
Mochi	Y	Y	Y	Y	Y
Maruko	Y	Y	Y	Y	Y

The width and height of coding channels supported by different chips are shown in the table below:

Chip	Resolution Spec	H.264	H.265	JPEG
Pretzel	MIN_WIDTH x MIN_HEIGHT	192x128	192x128	16x16
	MAX_WIDTH x MAX_HEIGHT	3840x2176	3840x2176	3840x2176
	MIN_ALIGN	8x2	8x2	8x2
Macaron	MIN_WIDTH x MIN_HEIGHT	192x128	192x128	16x16
	MAX_WIDTH x MAX_HEIGHT	2688x1920	2688x1920	2688x2688
	MIN_ALIGN	8x2	8x2	8x2
Pudding	MIN_WIDTH x MIN_HEIGHT	256x128	256x128	16x16
	MAX_WIDTH x MAX_HEIGHT	3840x2176	3840x2176	3840x2176
	MIN_ALIGN	8x2	8x2	8x2
Ispahan	MIN_WIDTH x MIN_HEIGHT	256x128	256x128	16x16
	MAX_WIDTH x MAX_HEIGHT	2592x1952	2592x1952	2688x2688
	MIN_ALIGN	8x2	8x2	8x2
Tiramisu	MIN_WIDTH x MIN_HEIGHT	256x128	256x128	16x16
	MAX_WIDTH x MAX_HEIGHT	4096x2176	4096x2176	8192x8192
	MIN_ALIGN	8x2	8x2	8x2
Ikayaki	MIN_WIDTHxMIN_HEIGHT	None	None	16x16
	MAX_WIDTHxMAX_HEIGHT	None	None	1920x1088
	MIN_ALIGN	None	None	8x2
Muffin	MIN_WIDTHxMIN_HEIGHT	256x128	256x128	16x16
	MAX_WIDTHxMAX_HEIGHT	8192x4096	8192x4096	8192x8192
	MIN_ALIGN	8x2	8x2	8x2
Mochi	MIN_WIDTHxMIN_HEIGHT	256x128	256x128	16x16
	MAX_WIDTHxMAX_HEIGHT	8192x4096	8192x4096	8192x8192
	MIN_ALIGN	8x2	8x2	8x2
Maruko	MIN_WIDTHxMIN_HEIGHT	256x128	256x128	16x16
	MAX_WIDTHxMAX_HEIGHT	4096x2176	4096x2176	8192x6480
	MIN_ALIGN	8x2	8x2	8x2

The following table describes the alignment parameters of input YUV buffer required by

HW:

Coding protocol	Binding mode	Input YUV format	Width alignment requirements(byte)	Height alignment requirement(byte)
H264/H265	Frame mode	sp420	32	32
	Ring mode		32	2
JPE	Frame mode	sp420	16	16
		yuyv422	16	8
	Ring mode	sp420	16	16
		yuyv422	16	8
	Realtime mode	sp420	16	16
		yuyv422	16	8

The maximum VENC_MAX_CHN_NUM specifications supported by different chips are shown in the table below:

Chip	Max
Macaron	16
Pudding	16
Ispahan	16
Ikayaki	16
Tiramisu	33
Ikayaki	16
Muffin	33
Mochi	33
Maruko	32

➢ Note

- MI_VENC_CreateDev is not called before calling this API, but a valid device id is passed in, the corresponding device will be created internally.
- The encoding channel attribute consists of two parts, the encoder attribute and the rate controller attribute.
- The encoder attribute first needs to select the encoding protocol, and then assign values to the attributes corresponding to the various protocols.
- The maximum width and height of the encoder attributes must be as follows:
  MaxPicWidth∈[MIN_WIDTH,MAX_WIDTH]
  MaxPicHeight∈[MIN_HEIGHT,MAX_HEIGHT]
  PicWidth∈[MIN_WIDTH,MaxPicwidth]
  PicHeight∈[MIN_HEIGHT,MaxPicHeight]
- The maximum width and height, the channel width must be an integer multiple of MIN_ALIGN.
- Where MIN_WIDTH, MAX_WIDTH, MIN_HEIGHT, MAX_HEIGHT, MIN_ALIGN represent the minimum width, maximum width, minimum height, maximum height, and minimum alignment unit (pixels) supported by the encoding channel, respectively.

- The encoding channel can be encoded when the input image size is not greater than the maximum width and height of the channel encoded image.
- The recommended code widths are: 3840x2160 (4k*2k), 1920x1080(1080P)、1280x720 (720P), 960x540, 640x360, 704x576, 704x480, 352x288, 352x240.
- In addition to the channel width and height and profile, the encoder attribute is a static attribute. Once the code channel is created successfully, the static attribute does not support modification unless the channel is destroyed and recreated. Please refer to the MI_VENC_SetChnAttr interface Description for the matters needing attention during setup.
- The rate controller attribute first needs to configure the RC mode. The JPEG capture channel does not need to configure the rate controller attribute. Other protocol type channels (H.264/H.265) must be configured. The rate controller attribute RC mode must match the encoder attribute protocol type.
- u32SrcFrmRateNum is the numerator of the frame rate of the encoding module, and u32SrcFrmRateDen is the denominator. u32SrcFrmRateNum/u32SrcFrmRateDen should be set to the actual input frame rate, and RC needs to count the actual frame rate and perform bit rate control based on it. If the actual input frame rate of the encoded image is 30, set u32SrcFrmRateNum to 30 and u32SrcFrmRateDen to 1.
- In addition to the above attributes, CBR also needs to set the target bit rate. The unit of the target bit rate is bps. The setting of the target bit rate is related to the encoding channel width and the image frame rate. The typical target bit rate settings are shown as follow. Note that the setting of the target bit rate in the table below is the setting when the channel encoding frame rate is the full frame rate (30 fps). When the user sets the code output frame rate not to the full frame rate, the ratio of the user-set frame rate to the full frame rate (30 fps) can be converted to the bit rate in the table below.

Image W/H/bitrate level	D1 (720x576)	720p (1280x720)	1080p (1920x1080)
Low bit rate	<400 Kbps	<800 Kbps	<2000 Kbps
Medium code rate	400~1000Kbps	800~4000Kpbs	2000~8000Kbps
High code rate	>1000Kbps	>4000Kbps	>8000Kbps

- The fluctuation level setting is divided into 5 levels. The larger the fluctuation level, the larger the fluctuation range of the system allowed code rate. If the fluctuation level is set high, the image quality may be smoother for some scenes with complex images and dramatic changes. It is suitable for scenes with rich network bandwidth. If the fluctuation level is set low, the code rate of the code will be relatively stable. For some images, In scenes with dramatic changes, the image quality may not be as high as the volatility level. It is suitable for scenes with less bandwidth, reserved, and temporarily unused.
- In addition to the above attributes, VBR needs to set MaxBitRate, MaxQp, MinQp.
  1) MaxBitRate: The maximum code rate allowed for the encoding channel during the rate statistics period.
  2) MaxQp: The maximum QP allowed for the image.
  3) MinQp: The minimum QP allowed for the image.
- FIXQP addition to the above properties, also need to set IQp, PQp.

1) IQp: QP value used for fixed images in I frames.
2) PQp: The QP value used for fixed images in P frames.

When the I frame QP and the P frame QP are set, the I frame QP and the P frame QP can be adjusted upward or downward simultaneously according to the current bandwidth limitation. In order to reduce the breathing effect, it is recommended that the I frame QP is always 2 to 3 smaller than the QP of the P frame.

- Example

```
MI_S32 StartVenc()
{
   MI_S32 s32Ret;
     MI_VENC_DEV VeDev = MI_VENC_DEV_ID_H264_H265_0;
   MI_VENC_CHN VeChn = 0;
   MI_VENC_ChnAttr_t stAttr;

   /*set h264 channel video encode attribute*/
   stAttr.stVeAttr.eType = E_MI_VENC_MODTYPE_H264E;
   stAttr.stVeAttr.stAttrH264e.u32PicWidth = u32PicWidth;
   stAttr.stVeAttr.stAttrH264e.u32PicHeight = u32PicHeight;
   stAttr.stVeAttr.stAttrH264e.u32MaxPicWidth = u32MaxPicWidth;
   stAttr.stVeAttr.stAttrH264e.u32MaxPicHeight = u32MaxPicHeight;
   stAttr.stVeAttr.stAttrH264e.u32Profile = 2;

   /*set h264 channel rate control attribute*/
   stAttr.stRcAttr.enRcMode = E_MI_VENC_RC_MODE_H264CBR;
   stAttr.stRcAttr.stAttrH264Cbr.u32BitRate = 10*1024*1024;
   stAttr.stRcAttr.stAttrH264Cbr.u32SrcFrmRateNum = 30;
   stAttr.stRcAttr.stAttrH264Cbr.u32SrcFrmRateDen = 1;
   stAttr.stRcAttr.stAttrH264Cbr.u32Gop = 30;
   stAttr.stRcAttr.stAttrH264Cbr.u32FluctuateLevel = 1;
   stAttr.stRcAttr.stAttrH264Cbr.u32StatTime = 1;

   s32Ret = MI_VENC_CreateChn(VeDev, VeChn, &stAttr);
   if (MI_SUCCESS != s32Ret)
   {
      printf("MI_VENC_CreateChn err0x%x\n", s32Ret);
      return E_MI_ERR_FAILED;
   }
   s32Ret = MI_VENC_StartRecvPic(VeDev, VeChn);
   if (s32Ret != MI_SUCCESS)
   {
      printf("MI_VENC_StartRecvPic err0x%x\n",s32Ret);
      return E_MI_ERR_FAILED;
   }

   return MI_SUCCESS;
}
```

- Related APIs

None.

2.4. MI_VENC_DestroyChn

- Features

  Destroy the encoding channel.

- Syntax

  MI_S32 MI_VENC_DestroyChn(MI_VENC_DEV VeDev, MI_VENC_CHN VeChn);

- Parameter

Parameter name	Description	Input/Output
VeDev	Encode device number	Input
VeChn	Encode channel number. Value range: [0, VENC_MAX_CHN_NUM).	Input

- Return Value
  - MI_SUCCESS: Successful
  - Not MI_SUCCESS: Failed, see ERROR CODE for details.

- Requirement
  - Header files: mi_venc.h, mi_venc_datatype.h
  - Library file: libmi_venc.a/libmi_venc.so

- Note
  - Destroy a channel that does not exist and return failure.

- Example

```
MI_S32 StopVenc()
{
    MI_S32 s32Ret;
      MI_VENC_DEV VeDev = MI_VENC_DEV_ID_H264_H265_0;
    MI_VENC_CHN VeChn = 0;

    s32Ret = MI_VENC_StopRecvPic(VeDev, VeChn);
    if (s32Ret != MI_SUCCESS)
    {
        printf("MI_VENC_StopRecvPic err0x%x\n", s32Ret);
        return E_MI_ERR_FAILED;
    }
    s32Ret = MI_VENC_DestroyChn(VeDev, VeChn);
    if (s32Ret != MI_SUCCESS)
    {
        printf("MI_VENC_DestroyChn err0x%x\n", s32Ret);
        return E_MI_ERR_FAILED;
    }

    return MI_SUCCESS;
}
```

- Related APIs

  None.

2.5. MI_VENC_ResetChn

- Features

  Reset the channel. It will clear the cached image and bitstream before calling the interface.

- Syntax

  MI_S32 MI_VENC_ResetChn(MI_VENC_DEV VeDev, MI_VENC_CHN VeChn);

- Parameter

Parameter name	Description	Input/Output
VeDev	Encode device number	Input
VeChn	Channel number. Value range: [0, VENC_MAX_CHN_NUM).	Input

- Return Value
  - MI_SUCCESS: Successful
  - Not MI_SUCCESS: Failed, see ERROR CODE for details.

- Requirement
  - Header files: mi_venc.h, mi_venc_datatype.h
  - Library file: libmi_venc.a/libmi_venc.so

- Note
  - Reset does not exist in the channel, returning failure MI_ERR_VENC_UNEXIST.
  - If a channel does not stop receiving images and resets the channel, it returns a failure.
  - Not supported yet.

- Example

  None.

- Related APIs

  Please refer to MI_VENC_StartRecvPicEx Example.

2.6. MI_VENC_StartRecvPic

- Features

  Turn on the encoding channel to receive the input image.

- Syntax

  MI_S32 MI_VENC_StartRecvPic(MI_VENC_DEV VeDev, MI_VENC_CHN VeChn);

- Parameter

Parameter name	Description	Input/Output

Parameter name	Description	Input/Output
VeChn	Encode channel number. Value range: [0, VENC_MAX_CHN_NUM).	Input

- Return Value
  - MI_SUCCESS: Successful
  - Not MI_SUCCESS: Failed, see ERROR CODE for details.

- Requirement
  - Header files: mi_venc.h, mi_venc_datatype.h
  - Library file: libmi_venc.a/libmi_venc.so

- Note
  - If the channel is not created, the failure MI_ERR_VENC_UNEXIST is returned.
  - This interface does not determine whether the reception is currently enabled, that is, allows repeated activation without returning an error.
  - The encoder starts receiving image coding only after calling this interface.

- Example

  Please refer to MI_VENC_CreateChn Example.

- Related APIs

  None.

2.7. MI_VENC_StartRecvPicEx

- Features

  Turn on the encoding channel to receive the input image, and automatically stop receiving images after the specified number of frames.

- Syntax

  MI_S32 MI_VENC_StartRecvPicEx(MI_VENC_DEV VeDev, MI_VENC_CHN VeChn, MI_VENC_RecvPicParam_t *pstRecvParam);

- Parameter

Parameter name	Description	Input/Output
VeDev	Encode device number	Input
VeChn	Encode channel number. Value range: [0, VENC_MAX_CHN_NUM).	Input
pstRecvParam	Receives an image parameter structure pointer that specifies the number of image frames that need to be received.	Input

- Return Value
  - MI_SUCCESS: Successful

- Not MI_SUCCESS: Failed, see ERROR CODE for details.

➢ Requirement

- Header files: mi_venc.h, mi_venc_datatype.h
- Library file: libmi_venc.a/libmi_venc.so

➢ Chip Difference

The following table shows whether different chips support the function of receiving the specified number of image frames：

Chip	Whether to support
Pretzel	N
Macaron	N
Pudding	N
Ispahan	N
Tiramisu	N
Ikayaki	N
Muffin	Y
Mochi	Y
Maruko	Y

➢ Note

- This interface is equivalent to MI_VENC_StartRecvPic interface only for chips after Muffin that support the function of receiving specified image frames.
- If the channel is not created, the failure MI_ERR_VENC_UNEXIST is returned.
- If the channel has already called MI_VENC_StartRecvPic to start receiving images without stopping receiving images, or has not received enough images since the last call to MI_VENC_StartRecvPicEx, calling this interface again will return MI_SUCCESS and take no effect.
- This interface is used to continuously receive N frames and encode scenes. When N=0, the interface is equivalent to MI_VENC_StartRecvPic.
- If the channel has already called MI_VENC_StartRecvPic to start receiving images, stop receiving images, and call MI_VENC_StartRecvPicEx to start encoding again, it is recommended that the user call MI_VENC_ResetChn to clear the image and code stream buffered by the encoding module before calling the interface.
- If you create a jpeg channel capture, it is recommended that you call MI_VENC_StartRecvPicEx because you need to receive an integer image and stop receiving it automatically.

➢ Example

```
MI_S32 JpegSnapProcess()
{
```

```
MI_S32 s32Ret;
  MI_VENC_DEV VeDev = MI_VENC_DEV_ID_JPEG_0;
MI_VENC_CHN VeChn=0;
MI_VENC_ChnAttr_t stAttr;
MI_VENC_RecvPicParam_t stRecvParam;

/*set jpeg channel video encode attribute*/
stAttr.stVeAttr.eType = E_MI_VENC_MODTYPE_JPEGE;
stAttr.stVeAttr.stAttrJpeg.u32PicWidth = u32PicWidth;
stAttr.stVeAttr.stAttrJpeg.u32PicHeight = u32PicHeight;
stAttr.stVeAttr.stAttrJpeg.u32MaxPicWidth = u32MaxPicWidth;
stAttr.stVeAttr.stAttrJpeg.u32MaxPicHeight = u32MaxPicHeight;

//…omit other video encode assignments here.

//create jpeg channel
s32Ret = MI_VENC_CreateChn(VeDev, VeChn, &stAttr);
if (MI_SUCCESS != s32Ret)
{
   printf("MI_VENC_CreateChn err0x%x\n", s32Ret);
   return E_MI_ERR_FAILED;
}
//start snapping
stRecvParam.s32RecvPicNum = 2;
s32Ret = MI_VENC_StartRecvPicEx(VeDev, VeChn, &stRecvParam);
if (s32Ret != MI_SUCCESS)
{
   printf("MI_VENC_StartRecvPicEx err0x%x\n", s32Ret);
   return E_MI_ERR_FAILED;
}

//…wait until all pictures have been encoded.

s32Ret = MI_VENC_StopRecvPic(VeDev, VeChn);
if (s32Ret != MI_SUCCESS)
{
   printf("MI_VENC_StopRecvPic err0x%x\n", s32Ret);
   return E_MI_ERR_FAILED;
  }



//destroy jpeg channel
s32Ret = MI_VENC_DestroyChn(VeDev, VeChn);
if (s32Ret != MI_SUCCESS)
{
   printf("MI_VENC_DestroyChn err0x%x\n", s32Ret);
   return E_MI_ERR_FAILED;
}

return MI_SUCCESS;
}
```

- Related APIs

None.

2.8. MI_VENC_StopRecvPic

- Features

  Stop the encoding channel to receive the input image.

- Syntax

  MI_S32 MI_VENC_StopRecvPic(MI_VENC_DEV VeDev, MI_VENC_CHN VeChn);

- Parameter

Parameter name	Description	Input/Output
VeDev	Encode device number	Input
VeChn	Encode channel number. Value range: [0, VENC_MAX_CHN_NUM).	Input

- Return Value
  - MI_SUCCESS: Successful
  - Not MI_SUCCESS: Failed, see ERROR CODE for details.

- Requirement
  - Header files: mi_venc.h, mi_venc_datatype.h
  - Library file: libmi_venc.a/libmi_venc.so

- Note
  - If the channel is not created, it returns a failure.
  - This interface does not determine whether to stop receiving currently, that is, to allow repeated stop reception without returning an error.
  - This interface is used to encode the channel to stop receiving images for encoding, and must stop receiving images before the encoding channel is destroyed or reset.
  - Calling this interface only stops receiving the original data encoding, and the stream buffer is not cleared.
  - Call the MI_VENC_StartRecvPic and MI_VENC_StartRecvPicEx interfaces to start receiving images, you can call this interface to stop receiving.

- Example

  Please refer to MI_VENC_DestroyChn Example.

- Related APIs

  None.

2.9. MI_VENC_Query

- Features

  Query the status of the encoded channel.

- Syntax

MI_S32 MI_VENC_Query(MI_VENC_DEV VeDev, MI_VENC_CHN VeChn, MI_VENC_ChnStat_t*pstStat);

- Parameter

Parameter name	Description	Input/Output
VeDev	Encode device number	Input
VeChn	Encode channel number. Value range: [0, VENC_MAX_CHN_NUM).	Input
pstStat	The status pointer of the encoding channel.	Output

- Return Value
  - MI_SUCCESS: Successful
  - Not MI_SUCCESS: Failed, see ERROR CODE for details.

- Requirement
  - Header files: mi_venc.h, mi_venc_datatype.h
  - Library file: libmi_venc.a/libmi_venc.so

- Note
  - If the channel is not created, it returns a failure.
    This interface is used to query the encoder state at the time of this function call. pstStat contains four main pieces of information:
    1) u32LeftPics represents the number of frames to be encoded. Before resetting the channel, you can determine the reset timing by querying whether there are still images to be encoded, and prevent the frames that may need to be cleaned out during reset.
    2) u32LeftStreamBytes represents the number of bytes remaining in the code stream buffer. Before resetting the channel, you can determine the reset timing by querying whether the code stream is still not processed, and prevent the code stream that may be needed from being cleared when resetting.
    3) u32LeftStreamFrames represents the number of frames remaining in the code stream buffer. Before resetting the channel, you can determine the reset timing by querying whether the code stream of the image is not taken, and prevent the code stream that may be needed from being cleared when resetting.
    4) u32CurPacks represents the number of code stream packets of the current frame. Make sure u32CurPacks is greater than 0 before calling MI_VENC_GetStream; otherwise, it will return an error that no buffer is available.
  - The current frame may not be a complete frame (taken a part) when acquired by packet, and the number of packets representing the current complete frame when acquired by frame (or 0 if there is no frame data). When the user needs to obtain the code stream by frame, the number of packets of a complete frame needs to be queried. In this case, the query operation can usually be performed after the select succeeds. At this time, u32CurPacks is the number of packets in the current complete frame.
  - In the code channel state structure, u32LeftRecvPics indicates the number of frames remaining waiting to be received after calling the MI_VENC_StartRecvPicEx interface.
  - In the code channel state structure, u32LeftEncPics indicates the number of frames

waiting to be encoded after calling the MI_VENC_StartRecvPicEx interface.
- If MI_VENC_StartRecvPicEx is not called, the number of u32LeftRecvPics and u32LeftEncPics is always 0.

➢ Example

Please refer to MI_VENC_GetStream Example.

➢ Related APIs

None.

2.10. MI_VENC_SetChnAttr

➢ Features

Set the encoding channel dynamic properties, including coded width and height, profile, and rate control parameters.

➢ Syntax

MI_S32 MI_VENC_SetChnAttr(MI_VENC_DEV VeDev, MI_VENC_CHN VeChn, MI_VENC_ChnAttr_t*pstAttr);

➢ Parameter

Parameter name	Description	Input/Output
VeDev	Encode device number	Input
VeChn	Encode channel number. Value range: [0, VENC_MAX_CHN_NUM).	Input
pstAttr	Coding channel property pointer.	Input

➢ Return Value

- MI_SUCCESS: Successful
- Not MI_SUCCESS: Failed, see ERROR CODE for details.

➢ Requirement

- Header files: mi_venc.h, mi_venc_datatype.h
- Library file: libmi_venc.a/libmi_venc.so

➢ Note

- The maximum width and maximum height of the encoding channel cannot be dynamically set.
- The encoding channel attribute includes two parts: the encoder attribute and the rate controller attribute.
- Sets the properties of an uncreated channel and returns a failure.
- If pstAttr is empty, it returns a failure.
- As the channel limitations and restrictions created when the coded picture size setting
- The coding channel attributes are divided into dynamic attributes and static attributes. The attribute value of the dynamic attribute is configured when the channel is created, and can be modified before the channel is destroyed. The attribute value of the static

attribute is configured when the channel is created, and cannot be modified after the channel is created.

- This interface can only set dynamic properties in the encoding channel properties, or return failure if the static properties are set. The encoding protocol of the encoding channel, the way of acquiring the code stream (acquiring the code stream by frame or by packet), and the maximum width and height of the encoded image attribute are static attributes. In addition, the static attributes of each encoding protocol are specified by each protocol module. For details, see MI_VENC_Chn Attr_t.
- Set the code rate controller attribute in the code channel attribute. The rate control mode is VBR. When the rate control advanced parameter is u32MinIQp is less than u32MinQp, where u32MinQp is the rate controller attribute, this interface changes the value of u32MinIQp to u32MinQp.
- When setting the channel width and height attributes in the encoder properties, the priority of the channel will not be restored to the default. All other parameters of the encoding channel are restored to the default values, the OSD is closed, and the stream buffer and cached image queue are cleared.

➢ Example

None.

➢ Related APIs

None.

2.11. MI_VENC_GetChnAttr

➢ Features

Get the encoding channel properties.

➢ Syntax

MI_S32 MI_VENC_GetChnAttr(MI_VENC_DEV VeDev, MI_VENC_CHN VeChn, MI_VENC_ChnAttr_t*pstAttr);

➢ Parameter

Parameter name	Description	Input/Output
VeDev	Encode device number	Input
VeChn	Encode channel number. Value range: [0, VENC_MAX_CHN_NUM).	Input
pstAttr	Coding channel property pointer.	Output

➢ Return Value

- MI_SUCCESS: Successful
- Not MI_SUCCESS: Failed, see ERROR CODE for details.

➢ Requirement

- Header files: mi_venc.h, mi_venc_datatype.h

- Library file: libmi_venc.a/libmi_venc.so

➢ Note

- Get the properties of the channel that was not created, returning the failure MI_ERR_VENC_UNEXIST.
- If pstAttr is empty, it returns a failure.

➢ Example

None.

➢ Related APIs

None.

2.12. MI_VENC_GetStream

➢ Features

Get the encoded code stream.

➢ Syntax

MI_S32 MI_VENC_GetStream(MI_VENC_DEV VeDev, MI_VENC_CHN VeChn, MI_VENC_Stream_t *pstStream, MI_S32 s32MilliSec);

➢ Parameter

Parameter name	Description	Input/Output
VeDev	Encode device number	Input
VeChn	Encode channel number. Value range: [0, VENC_MAX_CHN_NUM).	Input
pstStream	Code stream structure pointer. The upper layer needs to point the pointer to an allocated block of memory, and also needs to allocate the memory of PackCount*sizeof(MI_VENC_Pack_t) for the Member MI_VENC_Pack_t *pstPack.	Input/Output
s32MilliSec	The waiting time to call the API until the code stream is obtained. Value range: greater than or equal to 1. -1: Blocked. 0: Non-blocking. Greater than 0: Timeout, which is relative time, in ms.	Input

➢ Return Value

- MI_SUCCESS: Successful
- Not MI_SUCCESS: Failed, see ERROR CODE for details.

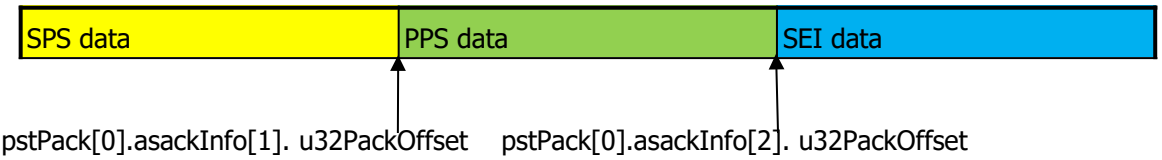
➢ Requirement

- Header files: mi_venc.h, mi_venc_datatype.h

- Library file: libmi_venc.a/libmi_venc.so

➢ Note

- If the channel is not created, the return fails.
- Returns MI_ERR_VENC_NULL_PTR if pstStream is empty.
- If s32MilliSec is less than -1, return MI_ERR_VENC_ILLEGAL_PARAM.
- Support timeout mode acquisition. Support for select/poll system calls.
- When s32MilliSec=0, it is non-blocking acquisition, that is, if there is no data in the buffer, it returns the failure MI_ERR_VENC_BUF_EMPTY. When s32MilliSec=-1, it is blocked, that is, if there is no data in the buffer, it will return to the success if it waits for data. When s32MilliSec>0, it is timeout, that is, if there is no data in the buffer, it will wait for the timeout time set by the user. If there is data within the set time, the return is successful, otherwise the return timeout fails.
- The code stream structure MI_VENC_Stream_t contains four parts:
  1) the code stream packet information pointer pstPack points to a memory space of a group of MI_VENC_Pack_t, which space is allocated by the caller. The size is u32PackCount x sizeof(MI_VENC_Pack_t), which can be obtained by calling MI_VENC_Query after selection.
  2) U32PackCount specifies the number of MI_VENC_Pack_t in the pstPack, which is the number of code streams in a complete frame.
  3) U32Seq represents the sequence of a frame.
  4) The stH264Info/stJpegInfo/stMpeg4Info/stH265Info combinations are the stream feature, which contains the characteristic information of code stream corresponding to different coding protocols. The output of the characteristic information of code stream is used to support the upper application of users.
- This interface should be paired with MI_Venc_ReleaseStream for use. The system will not actively release the bitstream cache after the user obtains the bitstream. The user needs to release the acquired bitstream cache in time; otherwise, it may cause the bitstream buffer to be full, thereby affecting the encoder coding.
- It is recommended that the user use the select method to obtain the code stream, and follow the following process:
  1) call the MI_VENC_Query function to query the encoding channel state.
  2) ensure that u32CurPacks and u32LeftStreamFrames are greater than 0 at the same time.
  3) call malloc to allocate u32CurPacks packet information structures.
  4) call MI_VENC_GetStream to obtain the encoded code stream.
  5) Call MI_VENC_ReleaseStream to release the code stream buffer.
- Taking H.264 as an example to introduce the code stream structure stored in pstPack[0], the corresponding information of pstPack[0] is as follows:
  pstPack[0].u32DataNum=3,
  pstPack[0].stPackInfo[0].u32PackType.enH264EType=E_MI_VENC_H264E_NALU_SPS,
  pstPack[0].stPackInfo[1].u32PackType.enH264EType=E_MI_VENC_H264E_NALU_PPS,
  pstPack[0].stPackInfo[2].u32PackType.enH264EType=E_MI_VENC_H264E_NALU_SEI ;
  The pstPack[0] contains three NALU packages, including SPS/PPS/SEI, as shown in the figure below:



- Tiramisu platform supports decoding, so it can encoding and decoding in a same case, in that case, the packet obtained from MI_VENC_GetStream needs to be ensured if it is a complete frame. If the answer is yes, send the packet to the decoder directly. If not, combine multiple packets into a complete frame and send it to vdec for decoding. Judging whether the packet is a complete frame through the bFrameEnd flag of MI_VENC_Pack_t. If it is 1, it means yes, if it is 0, it is divided into multiple packets.

➢ Example

```
MI_S32 VencGetH264Stream()
{
    MI_S32 i;
    MI_S32 s32Ret;
    MI_S32 s32VencFd;
      MI_U32 u32FrameIdx = 0;
      MI_VENC_DEV VeDev = MI_VENC_DEV_ID_H264_H265_0;
    MI_VENC_CHN VeChn = 0;
    MI_VENC_ChnStat_t stStat;
    MI_VENC_Stream_t stStream;
    fd_set read_fds;
    FILE* pFile=NULL;

    pFile = fopen("stream.h264","wb");
    if (pFile == NULL)
    {
        return E_MI_ERR_FAILED;
    }
    s32VencFd = MI_VENC_GetFd(VeDev, VeChn);
    do{
        FD_ZERO(&read_fds);
        FD_SET(s32VencFd, &read_fds);
        s32Ret = select(s32VencFd+1, &read_fds, NULL, NULL, NULL);
        if (s32Ret < 0)
        {
            printf("select err\n");
            return E_MI_ERR_FAILED;
        }
        else if (0 == s32Ret)
        {
            printf("timeout\n");
            return E_MI_ERR_FAILED;
        }
        else
        {
            if (FD_ISSET(s32VencFd, &read_fds))
            {
```

```
            s32Ret = MI_VENC_Query(VeDev, VeChn, &stStat);
            if (s32Ret != MI_SUCCESS)
            {
                return E_MI_ERR_FAILED;
            }
    /*****************************************************
    suggest to check both u32CurPacks and u32LeftStreamFrames at
    the same time,for example:
            if (0 == stStat.u32CurPacks || 0 == stStat.u32LeftStreamFrames)
            {
                continue;
            }
    *****************************************************/
            if (0 == stStat.u32CurPacks)
            {
                continue;
            }
            stStream.pstPack =
(MI_VENC_Pack_t*)malloc(sizeof(MI_VENC_Pack_t)*stStat.u32CurPacks);
            if (NULL == stStream.pstPack)
            {
                return E_MI_ERR_FAILED;
            }
            stStream.u32PackCount = stStat.u32CurPacks;
            s32Ret = MI_VENC_GetStream(VeDev, VeChn, &stStream, -1);
            if (MI_SUCCESS != s32Ret)
            {
                free(stStream.pstPack);
                stStream.pstPack = NULL;
                return E_MI_ERR_FAILED;
            }
            for (i=0; i<stStream.u32PackCount; i++)
            {
                fwrite(stStream.pstPack[i].pu8Addr+stStream.pstPack[i].u32Offset, 1,
                    stStream.pstPack[i].u32Len-stStream.pstPack[i].u32Offset, pFile);
            }
            s32Ret = MI_VENC_ReleaseStream(VeDev, VeChn, &stStream);
            if (MI_SUCCESS != s32Ret)
            {
                free(stStream.pstPack);
                stStream.pstPack=NULL;
                return E_MI_ERR_FAILED;
            }
            free(stStream.pstPack);
            stStream.pstPack = NULL;
        }
      }
      u32FrameIdx++;
    }while(u32FrameIdx<0xff);
    fclose(pFile);

    return s32Ret;
}
```

- Related APIs

  None.

2.13. MI_VENC_ReleaseStream

- Features

  Release stream buffer.

- Syntax

  MI_S32 MI_VENC_ReleaseStream(MI_VENC_DEV VeDev, MI_VENC_CHN VeChn, MI_VENC_Stream_t *pstStream);

- Parameter

Parameter name	Description	Input/Output
VeDev	Encode device number	Input
VeChn	Encode channel number. Value range: [0, VENC_MAX_CHN_NUM).	Input
pstStream	Code stream structure pointer.	Input

- Return Value
  - MI_SUCCESS: Successful
  - Not MI_SUCCESS: Failed, see ERROR CODE for details.

- Requirement
  - Header files: mi_venc.h, mi_venc_datatype.h
  - Library file: libmi_venc.a/libmi_venc.so

- Note
  - If the channel is not created, an error code is returned MI_ERR_VENC_UNEXIST.
  - in case pstStream Empty, return error code MI_ERR_VENC_NULL_PTR.
  - This interface should be MI_VENC_GetStream Paired and used, the user must release the acquired code stream cache in time after obtaining the code stream, otherwise the code stream may be caused. Buffer Full, affecting the encoder encoding, and the user must release the already acquired stream buffer in the order in which they were first released first.
  - After the code channel is reset, all unreleased stream packets are invalid and cannot be used or released.
  - Releasing an invalid stream will return a failure MI_ERR_VENC_ILLEGAL_PARAM.

- Example

  Please refer to MI_VENC_GetStream Example.

- Related APIs

  None.

2.14.MI_VENC_InsertUserData

- Features

  Insert user data. The user applies for a space, fills in the custom information, and passes in the corresponding pointer and data length. The NAL of the next SEI (before the latest code stream packet) will have the corresponding information.

- Syntax

  MI_S32 MI_VENC_InsertUserData(MI_VENC_DEV VeDev, MI_VENC_CHN VeChn, MI_U8 *pu8Data, MI_U32 u32Len);

- Parameter

Parameter name	Description	Input/Output
VeDev	Encode device number	Input
VeChn	Encode channel number. Value range: [0, VENC_MAX_CHN_NUM).	Input
pu8Data	User data pointer.	Input
u32Len	User data length. Value range: (0,1024), in bytes.	Input

- Return Value
  - MI_SUCCESS: Successful
  - Not MI_SUCCESS: Failed, see ERROR CODE for details.

- Requirement
  - Header files: mi_venc.h, mi_venc_datatype.h
  - Library file: libmi_venc.a/libmi_venc.so

- Chip Difference

  The maximum number of memory space blocks that can be allocated to cache user data at the same time supported by different Chips is shown in the following table:

芯片	最大值
Pretzel	4
Macaron	4
Pudding	1
Ispahan	1
Tiramisu	1
Muffin	1
Mochi	1
Maruko	1

- Note
  - If the channel is not created, it returns a failure.
  - In case of pu8Data, if it is empty, it will return failure.

- Insertion of user data only supports H.264/H.265.
- H.264/H.265 protocol channel supports the maximum number of memory space blocks allocated at the same time for buffering user data as shown in the above table, and the size of each segment of user data does not exceed 1kbyte. If the user inserts more data than the maximum number of blocks, or the inserted segment of user data is greater than 1kbyte, this interface will return an error. Each piece of user data is inserted as an SEI package before the latest code stream package. After a certain segment of user data packet is encoded and sent, the memory space for buffering this segment of user data in the H.264/H.265 channel is cleared to store new user data.

➢ Example

```
MI_S32 VencInsertUserData()
{
    MI_S32 s32Ret;
      MI_VENC_DEV VeDev = MI_VENC_DEV_ID_H264_H265_0;
    MI_VENC_CHN VeChn = 0;
    MI_U8 au8UserData[]="sigmastar2020";
    s32Ret = MI_VENC_InsertUserData(VeDev, VeChn, au8UserData,
      sizeof(au8UserData));
    if (MI_SUCCESS != s32Ret)
    {
        printf("MI_VENC_InsertUserData err0x%x\n", s32Ret);
        return E_MI_ERR_FAILED;
    }

    return s32Ret;
}
```

➢ Related APIs

None.

2.15. MI_VENC_SetMaxStreamCnt

➢ Features

Set the maximum number of cache frames for the stream.

➢ Syntax

MI_S32 MI_VENC_SetMaxStreamCnt(MI_VENC_DEV VeDev, MI_VENC_CHN VeChn, MI_U32 u32MaxStrmCnt);

➢ Parameter

Parameter name	Description	Input/Output
VeDev	Encode device number	Input
VeChn	Encode channel number. Value range: [0, VENC_MAX_CHN_NUM).	Input
u32MaxStrmCnt	Maximum number of stream buffer frames.	Input

- Return Value
  - MI_SUCCESS: Successful
  - Not MI_SUCCESS: Failed, see ERROR CODE for details.

- Requirement
  - Header files: mi_venc.h, mi_venc_datatype.h
  - Library file: libmi_venc.a/libmi_venc.so

- Note
  - This interface is used to set the code stream of the encoding channel. Buffer The maximum number of streams that can be cached.
  - If the number of cached stream frames has reached the maximum number of stream frames, the current image to be encoded is due to the code stream. Buffer Full without being encoded.
  - The maximum number of streams is specified by the system when the channel is created. The default value is 3.
  - This interface can be called after the channel is created and before the code channel is destroyed. Take effect before the next frame starts encoding. This interface allows multiple calls. It is recommended to set the encoding before starting the encoding. It is not recommended to dynamically adjust during the encoding process.

- Example

  None.

- Related APIs

  None.

2.16.MI_VENC_GetMaxStreamCnt

- Features

  Get the maximum number of cache frames in the stream.

- Syntax

  MI_S32 MI_VENC_GetMaxStreamCnt(MI_VENC_DEV VeDev, MI_VENC_CHN VeChn, MI_U32 *pu32MaxStrmCnt);

- Parameter

Parameter name	Description	Input/Output
VeDev	Encode device number	Input
VeChn	Encode channel number. Value range: [0, VENC_MAX_CHN_NUM).	Input
pu32MaxStrmCnt	A pointer to the maximum number of stream buffer frames.	Output

- Return Value
  - MI_SUCCESS: Successful

- Not MI_SUCCESS: Failed, see ERROR CODE for details.

➢ Requirement
- Header files: mi_venc.h, mi_venc_datatype.h
- Library file: libmi_venc.a/libmi_venc.so

➢ Note

None.

➢ Example

None.

➢ Related APIs

None.

2.17.MI_VENC_RequestIdr

➢ Features

Request an IDR frame.

➢ Syntax

MI_S32 MI_VENC_RequestIdr(MI_VENC_DEV VeDev, MI_VENC_CHN VeChn, MI_BOOL bInstant);

➢ Parameter

Parameter name	Description	Input/Output
VeDev	Encode device number	Input
VeChn	Encode channel number. Value range: [0, VENC_MAX_CHN_NUM).	Input
bInstant	Whether to enable encoding of IDR frames immediately. 0: IDR frames are coded in the shortest possible time. 1: The next frame is an IDR frame.	Input

➢ Return Value
- MI_SUCCESS: Successful
- Not MI_SUCCESS: Failed, see ERROR CODE for details.

➢ Requirement
- Header files: mi_venc.h, mi_venc_datatype.h
- Library file: libmi_venc.a/libmi_venc.so

➢ Note
- If the channel is not created, it returns a failure.
- I Frame request, only support H.264/H.265Coding protocol.
- Since this interface is not affected by frame rate control, an IDR will be encoded when call

this interface. Frequent calls will affect the stability of frame rate and code rate.

- Example

```
MI_S32 VencRequestIdr()
{
    MI_S32 s32Ret;
    MI_VENC_DEV VeDev = MI_VENC_DEV_ID_H264_H265_0;
  MI_VENC_CHN VeChn = 0;
  MI_BOOL bInstant;

  bInstant = TRUE;
  s32Ret = MI_VENC_RequestIdr(VeDev, VeChn, bInstant);
  if (MI_SUCCESS != s32Ret)
  {
    printf("MI_VENC_RequestIDR err0x%x\n", s32Ret);
    return E_MI_ERR_FAILED;
  }

  return s32Ret;
}
```

- Related APIs

None.

2.18. MI_VENC_EnableIdr

- Features

Whether to enable IDR frames.

- Syntax

MI_S32 MI_VENC_EnableIdr(MI_VENC_DEV VeDev, MI_VENC_CHN VeChn, MI_BOOL bEnableIdr);

- Parameter

Parameter name	Description	Input/Output
VeDev	Encode device number	Input
VeChn	Encode channel number. Value range: [0, VENC_MAX_CHN_NUM).	Input
bEnableIDR	Whether the flag is enabled.	Input

- Return Value
  - MI_SUCCESS: Successful
  - Not MI_SUCCESS: Failed, see ERROR CODE for details.

- Requirement
  - Header files: mi_venc.h, mi_venc_datatype.h
  - Library file: libmi_venc.a/libmi_venc.so

- Note
  - This interface is not supported currently.
  - If the channel is not created, it returns a failure.
  - If not enabled IDR Frame, it will not be edited after the next frame IDR Frame or I Frame until it is enabled again.
  - This interface only supports H.264/H.265 Coding protocol.

- Example

```
MI_S32 VencEnableIdr()
{
    MI_S32 s32Ret;
    MI_VENC_DEV VeDev = MI_VENC_DEV_ID_H264_H265_0;
    MI_VENC_CHN VeChn = 0;
    MI_BOOL bEnableIdr = TRUE;
  s32Ret = MI_VENC_EnableIdr(VeDev, VeChn, bEnableIdr);
  if (MI_SUCCESS != s32Ret)
  {
     printf("MI_VENC_EnableIdr err0x%x\n",s32Ret);
     return E_MI_ERR_FAILED;
  }

  return s32Ret;
}
```

- Related APIs

  None.

2.19. MI_VENC_SetH264IdrPicId

- Features

  Set the idr_pic_id attribute of the IDR frame.

- Syntax

  MI_S32 MI_VENC_SetH264IdrPicId(MI_VENC_DEV VeDev, MI_VENC_CHN VeChn, MI_VENC_H264IdrPicIdCfg_t*pstH264eIdrPicIdCfg);

- Parameter

Parameter name	Description	Input/Output
VeDev	Encode device number	Input
VeChn	Encode channel number. Value range: [0, VENC_MAX_CHN_NUM).	Input
pstH264IdrPicIdCfg	The parameter pointer to idr_pic_id.	Input

- Return Value
  - MI_SUCCESS: Successful
  - Not MI_SUCCESS: Failed, see ERROR CODE for details.

- Requirement
  - Header files: mi_venc.h, mi_venc_datatype.h
  - Library file: libmi_venc.a/libmi_venc.so

- Note
  - This interface is not supported currently.
  - If the channel is not created, it returns a failure.
  - Idr_pic_id The parameters are mainly determined by two parameters:
    1) enH264eIdrPicIdMode: Settings IDR frame Idr_pic_id Mode, enH264eIdrPicIdMode = E_MI_VENC_H264E_IDR_PIC_ID_MODE_AUTO Representation generated automatically by the internal process of the code Idr_pic_id, enH264eIdrPicIdMode=E_MI_VENC_H264E_IDR_PIC_ID_MODE_USR Indicates that the user is set Idr_pic_id Value.
    2) u32H264eIdrPicId: Settings IDR frame Idr_pic_id value. This value is valid only when enH264eIdrPicIdMode = E_MI_VENC_H264E_IDR_PIC_ID_MODE_USR.
  - Setting IDR Framed Idr_pic_id, only supported H.264 Coding protocol.
  - This interface can be called after the channel is created and before the code channel is destroyed. Take effect before the next frame starts encoding. This interface allows multiple calls. It is recommended to set the encoding before starting the encoding. It is not recommended to dynamically adjust during the encoding process.

- Example

  None.

- Related APIs

  None.

2.20. MI_VENC_GetH264IdrPicId

- Features

  Get the configuration attribute of the idr_pic_id of the IDR frame.

- Syntax

  MI_S32 MI_VENC_GetH264IdrPicId(MI_VENC_DEV VeDev, MI_VENC_CHN VeChn, MI_VENC_H264IdrPicIdCfg_t*pstH264eIdrPicIdCfg);

- Parameter

Parameter name	Description	Input/Output
VeDev	Encode device number	Input
VeChn	Encode channel number. Value range: [0, VENC_MAX_CHN_NUM).	Input
pstH264eIdrPicIdCfg	The parameter pointer to idr_pic_id.	Output

- Return Value

- MI_SUCCESS: Successful
- Not MI_SUCCESS: Failed, see ERROR CODE for details.

➢ Requirement

- Header files: mi_venc.h, mi_venc_datatype.h
- Library file: libmi_venc.a/libmi_venc.so

➢ Note

- This interface is not supported currently.
- If the channel is not created, it returns a failure.
- This interface can be called after the channel is created and before the code channel is destroyed.

➢ Example

None.

➢ Related APIs

None.

2.21.MI_VENC_GetFd

➢ Features

Get the device file handle corresponding to the encoding channel. It will be called before calling select/poll to listen for encoded data, thereby reducing CPU usage compared to polling.

➢ Syntax

MI_S32 MI_VENC_GetFd(MI_VENC_DEV VeDev, MI_VENC_CHN VeChn);

➢ Parameter

Parameter name	Description	Input/Output
VeDev	Encode device number	Input
VeChn	Encode channel number. Value range: [0, VENC_MAX_CHN_NUM).	Input

➢ Return Value

- Device file handle: Successful
- Others: Failed, see ERROR CODE for details.

➢ Requirement

- Header files: mi_venc.h, mi_venc_datatype.h
- Library file: libmi_venc.a/libmi_venc.so

➢ Note

None.

- Example

  See the example in MI_VENC_GetStream.

- Related APIs

  None.

2.22.MI_VENC_CloseFd

- Features

  Close the device file handle corresponding to the encoding channel.

- Syntax

  MI_S32 MI_VENC_CloseFd(MI_VENC_DEV VeDev, MI_VENC_CHN VeChn);

- Parameter

Parameter name	Description	Input/Output
VeDev	Encode device number	Input
VeChn	Video encoding channel number. Value range: [0, VENC_MAX_CHN_NUM).	Input

- Return Value
  - MI_SUCCESS: Successful
  - Not MI_SUCCESS: Failed, see ERROR CODE for details.

- Requirement
  - Header files: mi_venc.h, mi_venc_datatype.h
  - Library file: libmi_venc.a/libmi_venc.so

- Note

  None.

- Example

  None.

- Related APIs

  None.

2.23.MI_VENC_SetRoiCfg

- Features

  Set the ROI attribute of the H.264/H.265 channel.

- Syntax

  MI_S32 MI_VENC_SetRoiCfg(MI_VENC_DEV VeDev, MI_VENC_CHN VeChn,
  MI_VENC_RoiCfg_t *pstVencRoiCfg);

- Parameter

Parameter name	Description	Input/Output
VeDev	Encode device number	Input
VeChn	Encode channel number. Value range: [0, VENC_MAX_CHN_NUM).	Input
pstVencRoiCfg	ROI area parameter pointer.	Input

- Return Value
  - MI_SUCCESS: Successful
  - Not MI_SUCCESS: Failed, see ERROR CODE for details.

- Requirement
  - Header files: mi_venc.h, mi_venc_datatype.h
  - Library file: libmi_venc.a/libmi_venc.so

- Chip Difference

Different chip and coding protocols have different restrictions on ROI parameters, as shown in the following table:

Chip	Codec	Alignment				bAbsQp	
		u32Left	u32Top	u32Width	u32Height	AbsQp	RelQP
Pretzel	H.264	16	16	16	16	[12,48]	[-7,7]
	H.265	32	32	32	32	[12,48]	[-7,7]
Macaron	H.264	16	16	16	16	[12,48]	[-7,7]
	H.265	32	32	32	32	[12,48]	[-7,7]
Pudding	H.264	16	16	16	16	No support	[-32,31]
	H.265	32	32	32	32	No support	[-32,31]
Ispahan	H.264	16	16	16	16	No support	[-32,31]
	H.265	32	32	32	32	No support	[-32,31]
Tiramisu	H.264	16	16	16	16	No support	[-32,31]]
	H.265	32	32	32	32		
Muffin	H.264	16	16	16	16	No support	[-32,31]
	H.265	32	32	32	32		
Mochi	H.264	16	16	16	16	No support	[-32,31]
	H.265	32	32	32	32		
Maruko	H.264	16	16	16	16	No support	[-32,31]
	H.265	32	32	32	32		

- Note
  - This interface is used to set H.264/H.265 Protocol coding channel ROI Regional parameters, ROI Mainly by parameters 5 Parameter decision.

1) u32Index: System supports each channel to be set 8 One ROI Area, inside the system 0 ~ 7 Index number pair ROI Regional management, u32Index User settings indicated ROI Index number. ROI Areas can be superimposed on each other, and when overlays occur, ROI Priority between regions according to index number 0 ~ 7 Increase in turn.
2) bEnable: specify the current ROI Whether the area is enabled.
3) bAbsQp: specify the current ROI absolute use of the area QP Way or relative QP.
4) s32Qp: when bAbsQp for True Time, s32Qp Express ROI All macroblocks within the region are used QP Value, when bAbsQp for False Time, s32Qp Express ROI Relative to all macroblocks within the region QP value.
5) tRect: specify the current ROI The position coordinates of the area and size of the area. ROI starting point coordinates of the area must be within the image range.

- This interface belongs to the advanced interface. The system does not have a default. ROI The zone is enabled and the user must call this interface to start ROI.
- This interface can be set before the code channel is destroyed after the code channel is created. This interface takes effect when the next frame is called when it is called during encoding.
- It is recommended that the user call this interface before starting the encoding after creating the channel, reducing the number of calls during the encoding process.
- Before calling this interface, it is recommended that call MI_VENC_GetRoiCfg first to get the ROI config of current channel.

➢ Example

```
MI_S32 VencSetRoi()
{
   MI_S32 s32Ret;
   MI_VENC_RoiCfg_t stRoiCfg;
     MI_S32 index=0;
     MI_VENC_DEV VeDev = MI_VENC_DEV_ID_H264_H265_0;
   MI_VENC_CHN VeChnId=0;
   //...omit other thing
   s32Ret = MI_VENC_GetRoiCfg(VeDev, VeChnId, index, &stRoiCfg);
   if (MI_SUCCESS != s32Ret)
   {
      printf("MI_VENC_GetRoiCfg err0x%x\n", s32Ret);
      return E_MI_ERR_FAILED;
   }
   stRoiCfg.bEnable = TRUE;
   stRoiCfg.bAbsQp= FALSE;
   stRoiCfg.s32Qp = 10;
   stRoiCfg.stRect.u32Left = 16;
   stRoiCfg.stRect.u32Top = 16;
   stRoiCfg.stRect.u32Width = 16;
   stRoiCfg.stRect.u32Height = 16;
   stRoiCfg.u32Index = 0;
   s32Ret = MI_VENC_SetRoiCfg(VeDev, VeChnId, &stRoiCfg);
   if (MI_SUCCESS != s32Ret)
   {
```

```
        printf("MI_VENC_SetRoiCfg err0x%x\n", s32Ret);
        return E_MI_ERR_FAILED;
       }

    return s32Ret;
}
```

- Related APIs

  None.

2.24. MI_VENC_GetRoiCfg

- Features

  Get the ROI configuration properties of the H.264/H.265 channel.

- Syntax

  MI_S32 MI_VENC_GetRoiCfg(MI_VENC_DEV VeDev, MI_VENC_CHN VeChn, MI_U32 u32Index, MI_VENC_RoiCfg_t*pstVencRoiCfg);

- Parameter

Parameter name	Description	Input/Output
VeDev	Encode device number	Input
VeChn	Encode channel number. Value range: [0, VENC_MAX_CHN_NUM).	Input
u32Index	H.264/H.265 protocol encoding channel ROI region index.	Input
pstVencRoiCfg	Corresponds to the configuration pointer of the ROI area.	Output

- Return Value
  - MI_SUCCESS: Successful
  - Not MI_SUCCESS: Failed, see ERROR CODE for details.

- Requirement
  - Header files: mi_venc.h, mi_venc_datatype.h
  - Library file: libmi_venc.a/libmi_venc.so

- Note
  - This interface is used to obtain H.264/H.265 Protocol coding channel Index of ROI configuration.
  - This interface can be called after the encoding channel is destroyed after the encoding channel is created.
  - It is recommended that the user call this interface before starting the encoding after creating the channel, reducing the number of calls during the encoding process.

- Example

Please refer to MI_VENC_SetRoiCfg Example.

- Related APIs

  None.

2.25. MI_VENC_SetRoiBgFrameRate

- Features

  Set the non-ROI area frame rate attribute of the H.264/H.265 channel.

- Syntax

  MI_S32 MI_VENC_SetRoiBgFrameRate(MI_VENC_DEV VeDev, MI_VENC_CHN VeChn, MI_VENC_RoiBgFrameRate_t*pstRoiBgFrmRate);

- Parameter

Parameter name	Description	Input/Output
VeDev	Encode device number	Input
VeChn	Encode channel number. Value range: [0, VENC_MAX_CHN_NUM).	Input
pstRoiBgFrmRate	Non-ROI area frame rate control parameter pointer.	Input

- Return Value
  - MI_SUCCESS: Successful
  - Not MI_SUCCESS: Failed, see ERROR CODE for details.

- Requirement
  - Header files: mi_venc.h, mi_venc_datatype.h
  - Library file: libmi_venc.a/libmi_venc.so

- Note
  - This interface is used to set the frame rate control parameters of the non-ROI area of the H.264/H.265 protocol encoding channel.
  - The ROI area must be enabled before calling this interface.
  - This interface belongs to the advanced interface. The system is not enabled by default. ROI The area frame rate attribute, the user must call this interface to enable.
  - This interface can be set before the code channel is destroyed after the code channel is created. This interface takes effect when the next frame is called when it is called during encoding.
  - Input frame rate when setting channel frame rate control attribute SrcFrmRate Greater than the output frame rate DstFrmRate And DstFrmRate greater or equal to 0 Or at the same time -1.
  - It is recommended that the user call this interface before starting the encoding after creating the channel, reducing the number of calls during the encoding process.
  - It is recommended that the user call MI_VENC_GetRoiBgFrameRate interface to get

the non ROI Regional frame rate configuration of current channel before calling this interface.

- When setting the frame rate of the non-ROI area to be lower than the ROI area, it is not recommended to use frame skip reference or svc-t encoding. In the frame skipping reference or svc-t coding scenario, if the frame rate of the non-ROI area is set to be lower than the ROI area, the target frame rate of the actual encoded non-ROI area may be greater than the target frame rate configured by the user, because the base layer is non-ROI Regions cannot be encoded as pskip blocks. For example, before calling this interface to set the frame rate ratio of non ROI area to 2:1, MI_VENC_SetRefParam is also called to set the long reference frame, which will result in the actual frame rate ratio of non ROI area being less than 2:1.

➢ Example

```
MI_S32 VencSetRoiFrameRate()
{
   MI_S32 s32Ret;
   MI_VENC_RoiBgFrameRate_t stRoiBgFrameRate;
     MI_S32 index = 0;
     MI_VENC_DEV VeDev = MI_VENC_DEV_ID_H264_H265_0;
   MI_VENC_CHN VeChnId=0;

      //...omit other thing
    s32Ret = MI_VENC_GetRoiBgFrameRate (VeDev, VeChnId, &stRoiBgFrameRate);
    if (MI_SUCCESS != s32Ret)
    {
        printf("MI_VENC_GetRoiBgFrameRate err0x%x\n", s32Ret);
        return E_MI_ERR_FAILED;
    }

   //...omit other thing
   stRoiBgFrameRate.s32SrcFrmRate=30;
   stRoiBgFrameRate.s32DstFrmRate=15;
   s32Ret = MI_VENC_SetRoiBgFrameRate(VeDev, VeChnId, &stRoiBgFrameRate);
   if (MI_SUCCESS != s32Ret)
   {
      printf("MI_VENC_SetRoiBgFrameRate err0x%x\n", s32Ret);
      return E_MI_ERR_FAILED;
   }

   return s32Ret;
}
```

➢ Related APIs

None.

2.26.MI_VENC_GetRoiBgFrameRate

- Features

  Get the non-ROI area frame rate configuration attribute of the H.264/H.265 channel.

- Syntax

  MI_S32 MI_VENC_GetRoiBgFrameRate(MI_VENC_DEV VeDev, MI_VENC_CHN VeChn, MI_VENC_RoiBgFrameRate_t*pstRoiBgFrmRate);

- Parameter

Parameter name	Description	Input/Output
VeDev	Encode device number	Input
VeChn	Encode channel number. Value range: [0, VENC_MAX_CHN_NUM).	Input
pstRoiBgFrmRate	Configuration pointer of non-ROI area frame rate.	Output

- Return Value
  - MI_SUCCESS: Successful
  - Not MI_SUCCESS: Failed, see ERROR CODE for details.

- Requirement
  - Header files: mi_venc.h, mi_venc_datatype.h
  - Library file: libmi_venc.a/libmi_venc.so

- Note
  - This interface can be called after the encoding channel is destroyed after the encoding channel is created.
  - It is recommended that the user call this interface before starting the encoding after creating the channel, reducing the number of calls during the encoding process.

- Example

  Please refer to MI_VENC_SetRoiBgFrameRate Example.

- Related APIs

  None.

2.27.MI_VENC_SetH264SliceSplit

- Features

  Set the slice split attribute of the H.264 channel.

- Syntax

  MI_S32 MI_VENC_SetH264SliceSplit(MI_VENC_DEV VeDev, MI_VENC_CHN VeChn, MI_VENC_ParamH264SliceSplit_t*pstSliceSplit);

- Parameter

Parameter name	Description	Input/Output
VeDev	Encode device number	Input

Parameter name	Description	Input/Output
VeChn	Encode channel number. Value range: [0, VENC_MAX_CHN_NUM).	Input
pstSliceSplit	H.264 code stream slice segmentation parameter pointer.	Input

➢ Return Value

- MI_SUCCESS: Successful
- Not MI_SUCCESS: Failed, see ERROR CODE for details.

➢ Requirement

- Header files: mi_venc.h, mi_venc_datatype.h
- Library file: libmi_venc.a/libmi_venc.so

➢ Note

- This interface is used to set H.264 Protocol encoding channel code stream segmentation.
- Slice Split attribute is mainly Two Parameter decision:
  1) bSplitEnable: Whether the current frame is in progress Slice segmentation.
  2) u32SliceRowCount: Slice Split according to the macro block line. u32SliceRowCount represents the number of lines of each splice in the image macro block. And when encoding to the last few lines of the image, insufficient u32SliceRowCount will occur when the remaining macroblock lines are divided into one Slice.
- The default value of bSplitEnable is false. When bSplitEnable is set to true, the u32SliceRowCount macroblock row is used as a slice to encode separately. After each slice is encoded, the user can get the stream data of the slice. It is better not to set u32SliceRowCount too small. The smaller the u32SliceRowCount is, the more frequent the user receives the notification, resulting in greater system overhead.
- It is recommended to call this interface after creating the channel and before starting the channel, to reduce the number of calls during the encoding process. It takes effect at the next frame.
- Before calling this interface, it is recommended that call MI_VENC_GetH264SliceSplit first to get slicesplit config of current channel.

➢ Example

```
MI_S32 VencSetSliceSplit()
{
    MI_S32 s32Ret;
      MI_VENC_ParamH264SliceSplit_t stSlice;
      MI_VENC_DEV VeDev = MI_VENC_DEV_ID_H264_H265_0;
    MI_VENC_CHN VeChnId = 0;

    //...omit other thing
    s32Ret = MI_VENC_GetH264SliceSplit(VeDev, VeChnId, &stSlice);
    if (MI_SUCCESS != s32Ret)
    {
        printf("MI_VENC_GetH264SliceSplit err0x%x\n", s32Ret);
```

```
        return E_MI_ERR_FAILED;
    }
    stSlice.bSplitEnable = TRUE;
    stSlice.u32SliceRowCount = 8;
    s32Ret = MI_VENC_SetH264SliceSplit(VeDev, VeChnId, &stSlice);
    if(MI_SUCCESS != s32Ret)
    {
        printf("MI_VENC_SetH264SliceSplit err0x%x\n", s32Ret);
        return E_MI_ERR_FAILED;
    }

    return s32Ret;
}
```

➢ Related APIs

None.

2.28. MI_VENC_GetH264SliceSplit

➢ Description

Get the slice split attribute of the H.264 channel

➢ Syntax

MI_S32 MI_VENC_GetH264SliceSplit(MI_VENC_DEV VeDev, MI_VENC_CHN VeChn, MI_VENC_ParamH264SliceSplit_t*pstSliceSplit);

➢ Parameters

parameter name	Description	Input/Output
VeDev	Encode device number	Input
VeChn	EnoEncode channel number.. Value range: [0, VENC_MAX_CHN_NUM).	input
pstSliceSplit	H.264 stream slice splite parameter pointer.	output

➢ Return value

- MI_SUCCESS: Successful
- Not MI_SUCCESS: Failed, see ERROR CODE for details.

➢ Dependence

- Header files: mi_venc.h, mi_venc_datatype.h
- Library file: libmi_venc.a/libmi_venc.so

➢ Note

- This interface is used to obtain the slice segmentation attribute of the H.264 protocol encoding channel.
- This interface can be called before the encoding channel is destroyed and after the encoding channel is created.
- It is recommended that the user call this interface before starting the encoding and

after creating the channel, reducing the number of calls during the encoding process

- Example

  See the example of MI_VENC_SetH264SliceSplit

- related topic

  None.

2.29. MI_VENC_SetH264InterPred

- Description

  Set the inter prediction property of the H.264 protocol encoding channel.

- Syntax

  MI_S32 MI_VENC_SetH264InterPred(MI_VENC_DEV VeDev, MI_VENC_CHN VeChn, MI_VENC_ParamH264InterPred_t *pstH264InterPred);

- Parameters

Parameter Name	Description	Input/Output
VeDev	Encode device number	Input
VeChn	Encode channel number. Value range: [0, VENC_MAX_CHN_NUM).	Input
pstH264InterPred	Inter-prediction configuration pointer of the H.264 protocol encoding channel.	Input

- Return value
  - MI_SUCCESS: Successful
  - Not MI_SUCCESS: Failed, see ERROR CODE for details.

- Dependence
  - Header files: mi_venc.h, mi_venc_datatype.h
  - Library file: libmi_venc.a/libmi_venc.so

- note
  - This interface is not supported currently.
  - This interface is used to set the interframe prediction configuration of the H.264 protocol encoding channel.
  - The properties of inter prediction are mainly determined by seven parameters:
    1) u32HWSize: the size of the horizontal search window when inter prediction.
    2) u32VWSize: The size of the vertical search window for inter prediction.
    3) bInter16x16PredEn: 16x16 block inter prediction enable flag.
    4) bInter16x8PredEn: 16x8 block inter prediction enable flag.
    5) bInter8x16PredEn: 8x16 block inter prediction enable flag.
    6) bInter8x8PredEn: 8x8 block inter prediction enable flag.
    7) bExtedgeEn: Interframe prediction expansion search enable. When inter-prediction is

performed on a macroblock on an image boundary, the range of the search window may exceed the boundary of the image. At this time, the expansion search enables the image to be expanded and inter-frame predicted. If the extended search enable is turned off, no prediction will be made for search windows that are outside the range of the image.

- This interface belongs to the advanced interface, which can be called by the user. It is recommended not to call, and the system will have default values. The default search window size will vary depending on the resolution of the image, and the other five predictive enable switches are all enabled by default.
- The predictive switch of bInter16x16PredEn can only be turned on.
- This interface can be set before the code channel is destroyed after the code channel is created. When this interface is called during the encoding process, it takes effect until the next frame.
- It is recommended that the user call this interface before starting the encoding after creating the channel, reducing the number of calls during the encoding process
- Users are advised before calling this interface, first call MI_VENC_GetH264InterPred interface configured to obtain InterPred current channel coding, and then set it.

➢ Example

```
MI_S32 VencSetInterPred()
{
    MI_S32 s32Ret;
     MI_VENC_ParamH264InterPred_t stInterPred;
       MI_VENC_DEV VeDev = MI_VENC_DEV_ID_H264_H265_0;
    MI_VENC_CHN VeChnId = 0;

    //...omit other thing
    s32Ret = MI_VENC_GetH264InterPred(VeDev, VeChnId, &stInterPred);
    if (MI_SUCCESS != s32Ret)
    {
        printf("MI_VENC_GetH264InterPred err0x%x\n", s32Ret);
        return E_MI_ERR_FAILED;
    }
    stInterPred.bExtedgeEn = TRUE;
    stInterPred.bInter16x16PredEn = TRUE;
    stInterPred.bInter16x8PredEn = FALSE;
    stInterPred.bInter8x16PredEn = FALSE;
    stInterPred.bInter8x8PredEn = FALSE;
    stInterPred.u32HWSize = 4;
    stInterPred.u32VWSize = 2;
    s32Ret = MI_VENC_SetH264InterPred(VeDev, VeChnId, &stInterPred);
    if (MI_SUCCESS != s32Ret)
    {
        printf("MI_VENC_SetH264InterPred err0x%x\n", s32Ret);
        return E_MI_ERR_FAILED;
    }

    return s32Ret;
}
```

- related topic

  None.

2.30. MI_VENC_GetH264InterPred

- Description

  Obtain the interframe prediction property of the H.264 protocol coding channel.

- Syntax

  MI_S32 MI_VENC_GetH264InterPred(MI_VENC_DEV VeDev, MI_VENC_CHN VeChn, MI_VENC_ParamH264InterPred_t*pstH264InterPred);

- parameters

parameter name	Description	Input/Output
VeDev	Encode device number	Input
VeChn	Encode channel number. Value range: [0, VENC_MAX_CHN_NUM).	input
pstH264InterPred	Interframe prediction parameter pointer of the H.264 protocol coding channel.	output

- Return value
  - MI_SUCCESS: Successful
  - Not MI_SUCCESS: Failed, see ERROR CODE for details.

- Dependence
  - Header files: mi_venc.h, mi_venc_datatype.h
  - Library file: libmi_venc.a/libmi_venc.so

- note
  - This interface is not supported currently.
  - This interface is used to obtain the interframe prediction property of the H.264 protocol coding channel.
  - This interface can be called after the encoding channel is destroyed after the encoding channel is created.
  - It is recommended that the user call this interface before starting the encoding after creating the channel, reducing the number of calls during the encoding process.

- Example

  See the example of MI_VENC_SetH264InterPred.

- related topic

  None.

2.31. MI_VENC_SetH264IntraPred

- Description

Set the intraframe prediction property of the H.264 protocol encoding channel.

- Syntax

MI_S32 MI_VENC_SetH264IntraPred(MI_VENC_DEV VeDev, MI_VENC_CHN VeChn, MI_VENC_ParamH264IntraPred_t*pstH264IntraPred);

- Parameters

parameter name	Description	Input/Output
VeDev	Encode device number	Input
VeChn	Encode channel number. Value range: [0, VENC_MAX_CHN_NUM).	Input
pstH264IntraPred	Intraframe prediction configuration pointer of the H.264 protocol coding channel.	Input

- Return value
  - MI_SUCCESS: Successful
  - Not MI_SUCCESS: Failed, see ERROR CODE for details.

- Dependence
  - Header files: mi_venc.h, mi_venc_datatype.h
  - Library file: libmi_venc.a/libmi_venc.so

- note
  - This interface is not supported currently.
  - This interface is used to set the intraframe prediction configuration of the H.264 protocol encoding channel.
  - The intraframe prediction property is mainly determined by four parameters:
    1) bIntra16x16PredEn: 16x16 block intra prediction enable flag.
    2) bIntraNxNPredEn: NxN block intra prediction enable flag. Among them, NxN means 4x4 and 8x8.
    3) bIpcmEn: IPCM block intra prediction enable flag.
    4) bConstrainedIntraPredFlag: For the specific meaning, please refer to the H.264 protocol.
  - This interface is a high-level interface that can be called by the user. It is not recommended to call. The system has default values. bIntra16x16PredEn and bIntraNxNPredEn are enabled by default. bIpcmEn has different default values for different chips. bConstrainedIntraPredFlag is 0 by default.
  - There must be one of the two intraframe prediction enable switches, bIntra16x16PredEn and bIntraNxNPredEn, and the system does not support that all two switches are turned off.
  - This interface can be set before the code channel is destroyed after the code channel is created. When this interface is called during the encoding process, it takes effect until the next I frame.
  - It is recommended that the user call this interface before starting the encoding after creating the channel, reducing the number of calls during the encoding process.
  - It is recommended that the user call the MI_VENC_GetH264IntraPred interface to obtain the IntraPred configuration of the current encoding channel before calling this

interface, and then set it.

- Example

```
MI_S32 VencSetIntraPred()
{
    MI_S32 s32Ret;
      MI_VENC_ParamH264IntraPred_t stIntraPred;
      MI_VENC_DEV VeDev = MI_VENC_DEV_ID_H264_H265_0;
    MI_VENC_CHN VeChnId = 0;

    //...omit other thing
    s32Ret = MI_VENC_GetH264IntraPred(VeDev, VeChnId, &stIntraPred);
    if (MI_SUCCESS != s32Ret)
    {
        printf("MI_VENC_GetH264IntraPred err0x%x\n", s32Ret);
        return E_MI_ERR_FAILED;
    }
    stIntraPred.bIntra16x16PredEn = TRUE;
    stIntraPred.bIntraNxNPredEn   = FALSE;
    stIntraPred.bIpcmEn = FALSE;
    stIntraPred.bConstrainedIntraPredFlag = 0;
    s32Ret = MI_VENC_SetH264IntraPred(VeDev, VeChnId, &stIntraPred);
    if (MI_SUCCESS != s32Ret)
    {
        printf("MI_VENC_SetH264IntraPred err0x%x\n", s32Ret);
        return E_MI_ERR_FAILED;
    }

    return s32Ret;
}
```

- related topic

None.

2.32. MI_VENC_GetH264IntraPred

- Description

Get the intraframe prediction attribute of the H.264 protocol encoding channel.

- Syntax

MI_S32 MI_VENC_GetH264IntraPred(MI_VENC_DEV VeDev, MI_VENC_CHN VeChn, MI_VENC_ParamH264IntraPred_t*pstH264IntraPred);

- Parameters

parameter name	Description	Input/Output
VeDev	Encode device number	Input
VeChn	Encode channel number. Value range: [0, VENC_MAX_CHN_NUM).	input

pstH264IntraPred	Intraframe prediction parameter pointer of the H.264 protocol coding channel.	output

- Return value
  - MI_SUCCESS: Successful
  - Not MI_SUCCESS: Failed, see ERROR CODE for details.

- Dependence
  - Header files: mi_venc.h, mi_venc_datatype.h
  - Library file: libmi_venc.a/libmi_venc.so

- Note
  - This interface is not supported currently.
  - This interface is used to obtain the intraframe prediction mode of the H.264 protocol coding channel.
  - This interface can be called after the encoding channel is destroyed after the encoding channel is created.
  - It is recommended that the user call this interface before starting the encoding after creating the channel, reducing the number of calls during the encoding process.

- Example

  See the example of MI_VENC_SetH264IntraPred.

- related topic

  None.

2.33. MI_VENC_SetH264Trans

- Description

  Set the transform and quantized attributes of the H.264 protocol encoding channel.

- Syntax

  MI_S32 MI_VENC_SetH264Trans(MI_VENC_DEV VeDev, MI_VENC_CHN VeChn, MI_VENC_ParamH264Trans_t*pstH264Trans);

- Parameters

parameter name	Description	Input/Output
VeDev	Encode device number	Input
VeChn	Encode channel number. Value range: [0, VENC_MAX_CHN_NUM).	input
pstH264Trans	The transform and quantization attribute pointer of the H.264 protocol coding channel.	input

- Return value
  - MI_SUCCESS: Successful
  - Not MI_SUCCESS: Failed, see ERROR CODE for details.

- Dependence
  - Header files: mi_venc.h, mi_venc_datatype.h
  - Library file: libmi_venc.a/libmi_venc.so

- Note
  - This interface is used to set the transformation and quantization configuration of the H.264 protocol encoding channel.
  - Transform, quantization property mainly by three parameters.
    1) u32IntraTransMode: The transform attribute of the intra prediction macroblock. This is a reserved interface that is currently not supported.
    2) u32InterTransMode: The transform attribute of the inter prediction macroblock. This is a reserved interface that is currently not supported.
    3) s32ChromaQpIndexOffset: For details, see the chroma_qp_index_offset of H.264 protocol.
  - This interface is a high-level interface that can be called by the user. It is not recommended to call this interface. The system has default values.
  - This interface can be set before the code channel is destroyed after the code channel is created. When this interface is called during the encoding process, it takes effect until the next I frame.
  - It is recommended that the user call this interface before starting the encoding after creating the channel, reducing the number of calls during the encoding process.
  - Users are advised before calling this interface, first call MI_VENC_GetH264Trans interfaces, access to pre-coded channel when the trans configuration, and then set it.

- Example

```
MI_S32 VencSetTrans()
{
   MI_S32 s32Ret;
     MI_VENC_ParamH264Trans_t stTrans;
     MI_VENC_DEV VeDev = MI_VENC_DEV_ID_H264_H265_0;
   MI_VENC_CHN VeChnId = 0;

   //...omit other thing
   s32Ret = MI_VENC_GetH264Trans(VeDev, VeChnId, &stTrans);
   if (MI_SUCCESS != s32Ret)
   {
      printf("MI_VENC_GetH264Trans err0x%x\n", s32Ret);
      return E_MI_ERR_FAILED;
   }
   stTrans.u32IntraTransMode = 2;
   stTrans.u32InterTransMode = 2;
   stTrans.s32ChromaQpIndexOffset = 2;
   s32Ret = MI_VENC_SetH264Trans(VeDev, VeChnId, &stTrans);
   if (MI_SUCCESS != s32Ret)
   {
      printf("MI_VENC_SetH264Trans err0x%x\n", s32Ret);
      return E_MI_ERR_FAILED;
   }
```

```
    return s32Ret;
}
```

- related topic

None.

2.34. MI_VENC_GetH264Trans

- Description

Obtain the transform and quantization attributes of the H.264 protocol coding channel.

- Syntax

MI_S32 MI_VENC_GetH264Trans(MI_VENC_DEV VeDev, MI_VENC_CHN VeChn, MI_VENC_ParamH264Trans_t *pstH264Trans);

- Parameters

parameter name	Description	Input/Output
VeDev	Encode device number	Input
VeChn	Encode channel number. Value range: [0, VENC_MAX_CHN_NUM).	input
pstH264Trans	The transform and quantization parameter ponter of the H.264 protocol coding channel.	output

- Return value
  - MI_SUCCESS: Successful
  - Not MI_SUCCESS: Failed, see ERROR CODE for details.

- Dependence
  - Header files: mi_venc.h, mi_venc_datatype.h
  - Library file: libmi_venc.a/libmi_venc.so

- Note
  - This interface is used to obtain the transform and quantization configuration of the H.264 protocol coding channel.
  - This interface can be called after the encoding channel is destroyed after the encoding channel is created.
  - It is recommended that the user call this interface before starting the encoding after creating the channel, reducing the number of calls during the encoding process.

- Example

See the example of MI_VENC_SetH264Trans.

- related topic

None.

2.35. MI_VENC_SetH264Entropy

- Description

  Set the entropy coding mode of the H.264 protocol coding channel.

- Syntax

  MI_S32 MI_VENC_SetH264Entropy(MI_VENC_CHN VeChn,
  MI_VENC_ParamH264Entropy_t *pstH264EntropyEnc);

- Parameters

parameter name	Description	Input/Output
VeChn	Encode channel number. Value range: [0, VENC_MAX_CHN_NUM).	input
pstH264EntropyEnc	Entropy coding mode pointer of the H.264 protocol coding channel.	input

- Return value
  - MI_SUCCESS: Successful
  - Not MI_SUCCESS: Failed, see ERROR CODE for details.

- Dependence
  - Header files: mi_venc.h, mi_venc_datatype.h
  - Library file: libmi_venc.a/libmi_venc.so

- Note
  - This interface is used to set the entropy coding configuration of the H.264 protocol coding channel.
  - Transform quantization property mainly by two parameters:
    1) u32EntropyEncModeI: entropy coding mode I frame, u32EntropyEncModeI = 0 indicates that the I-frame coding using cavlc, u32EntropyEncModeI = 1 indicates an I frame encoding cabac used.
    2) u32EntropyEncModeP: entropy encoding of P frames, u32EntropyEncModeP = 0 indicates coding P frames use cavlc, u32EntropyEncModeP = 1 indicates P-frame coding using cabac.
  - The entropy coding mode of the I frame and the entropy coding mode of the P frame can be set separately.
  - Baselineprofile does not support cabac encoding, supports cavlc encoding, main profile and high profile support cabac encoding and cavlc encoding.
  - The Cabac encoding method requires a larger amount of computation than the cavlc encoding method, but generates less code streams. If the system performance is not rich enough, it is recommended that users can take I frame cavlcencoding, P frame cabac encoding.
  - This interface is a high-level interface that can be called by the user. It is not recommended to call. The system has default values. The default parameters are set according to different profiles. The default entropy encoding parameters of the system under different profiles are shown in the table below.

Profile	u32EntropyEncModeI	u32EntropyEncModeP
Baseline	0	0
Mainprofile/Highprofile	1	1

- This interface can be set before the code channel is destroyed after the code channel is created. When this interface is called during the encoding process, it takes effect until the next I frame.
- It is recommended that the user call this interface before starting the encoding after creating the channel, reducing the number of calls during the encoding process.
- It is recommended that the user call the MI_VENC_GetH264Entropy interface to obtain the Entropy configuration of the current encoding channel before calling this interface, and then set it.

➢ Example

```
MI_S32 VencSetEntropy()
{
    MI_S32 s32Ret;
    MI_VENC_ParamH264Entropy_t stEntropy;
      MI_VENC_DEV VeDev = MI_VENC_DEV_ID_H264_H265_0;
    MI_VENC_CHN VeChnId = 0;

    //...omit other thing
    s32Ret = MI_VENC_GetH264Entropy(VeDev, VeChnId, &stEntropy);
    if (MI_SUCCESS != s32Ret)
    {
        printf("MI_VENC_GetH264Entropy err0x%x\n", s32Ret);
        return E_MI_ERR_FAILED;
    }
    stEntropy.u32EntropyEncModeI = 0;
    stEntropy.u32EntropyEncModeP = 0;
    s32Ret = MI_VENC_SetH264Entropy(VeDev, VeChnId, &stEntropy);
    if (MI_SUCCESS != s32Ret)
    {
        printf("MI_VENC_SetH264Entropy err0x%x\n", s32Ret);
        return E_MI_ERR_FAILED;
    }

    return s32Ret;
}
```

➢ related topic

None.

2.36. MI_VENC_GetH264Entropy

➢ Description

Obtain the entropy encoding attribute of the H.264 protocol encoding channel.

➢ Syntax

MI_S32 MI_VENC_GetH264Entropy(MI_VENC_DEV VeDev, MI_VENC_CHN VeChn,

MI_VENC_ParamH264Entropy_t*pstH264EntropyEnc);

- Parameters

parameter name	Description	Input/Output
VeDev	Encode device number	Input
VeChn	Encode channel number. Value range: [0, VENC_MAX_CHN_NUM).	input
pstH264EntropyEnc	The entropy coding attribute pointer of the H.264 protocol coding channel.	output

- Return value
  - MI_SUCCESS: Successful
  - Not MI_SUCCESS: Failed, see ERROR CODE for details.

- Dependence
  - Header files: mi_venc.h, mi_venc_datatype.h
  - Library file: libmi_venc.a/libmi_venc.so

- Note
  - This interface is used to obtain the entropy coding configuration of the H.264 protocol coding channel.
  - This interface can be called after the encoding channel is destroyed after the encoding channel is created.
  - It is recommended that the user call this interface before starting the encoding after creating the channel, reducing the number of calls during the encoding process.

- Example

  See the example of MI_VENC_SetH264Entropy

- related topic

  None.

2.37.MI_VENC_SetH265InterPred

- Description

  Set the inter prediction property of the H.265 protocol encoding channel.

- Syntax

  MI_S32 MI_VENC_SetH265InterPred(MI_VENC_DEV VeDev, MI_VENC_CHN VeChn, MI_VENC_ParamH265InterPred_t*pstH265InterPred);

- Parameters

parameter name	Description	Input/Output
VeDev	Encode device number	Input
VeChn	Encode channel number. Value range: [0, VENC_MAX_CHN_NUM).	input

pstH265InterPred	Inter-prediction configuration pointer of the H.265 protocol coding channel	input

- Return value
  - MI_SUCCESS: Successful
  - Not MI_SUCCESS: Failed, see ERROR CODE for details.

- Dependence
  - Header files: mi_venc.h, mi_venc_datatype.h
  - Library file: libmi_venc.a/libmi_venc.so

- Note
  - This interface is not supported currently.
  - This interface is used to set the interframe prediction configuration of the H.265 protocol encoding channel.
  - The properties of inter prediction are mainly determined by seven parameters:
    1) u32HWSize: the size of the horizontal search window when inter prediction.
    2) u32VWSize: The size of the vertical search window for inter prediction.
    3) bInter16x16PredEn: 16x16 block inter prediction enable flag.
    4) bInter16x8PredEn: 16x8 block inter prediction enable flag.
    5) bInter8x16PredEn: 8x16 block inter prediction enable flag.
    6) bInter8x8PredEn: 8x8 block inter prediction enable flag.
    7) bExtedgeEn: Interframe prediction expansion search enable. When inter-prediction is performed on a macroblock on an image boundary, the range of the search window may exceed the boundary of the image. At this time, the expansion search enables the image to be expanded and inter-frame predicted. If the extended search enable is turned off, no prediction will be made for search windows that are outside the range of the image.
  - This interface belongs to the advanced interface, which can be called by the user. It is recommended not to call, and the system will have default values. The default search window size will vary depending on the resolution of the image, and the other five predictive enable switches are all enabled by default.
  - The predictive switch of bInter16x16PredEn can only be turned on.
  - This interface can be set before the code channel is destroyed after the code channel is created. When this interface is called during the encoding process, it takes effect until the next frame.
  - It is recommended that the user call this interface before starting the encoding after creating the channel, reducing the number of calls during the encoding process. Before calling this interface, users are advised to call MI_VENC_GetH265InterPred interface first to obtain InterPred current channel coding, and then set it.

- Example

  Please refer to the example of MI_VENC_SetH264InterPred

- related topic

  None.

2.38. MI_VENC_GetH265InterPred

- Description

  Obtain the inter prediction property of the H.265 protocol coding channel.

- Syntax

  MI_S32 MI_VENC_GetH265InterPred(MI_VENC_DEV VeDev, MI_VENC_CHN VeChn, MI_VENC_ParamH265InterPred_t*pstH265InterPred);

- Parameters

parameter name	Description	Input/Output
VeDev	Encode device number	Input
VeChn	Encode channel number. Value range: [0, VENC_MAX_CHN_NUM).	input
pstH265InterPred	Inter-prediction parameter pointer of the H.265 protocol coding channel.	output

- Return value
  - MI_SUCCESS: Successful
  - Not MI_SUCCESS: Failed, see ERROR CODE for details.

- Dependence
  - Header files: mi_venc.h, mi_venc_datatype.h
  - Library file: libmi_venc.a/libmi_venc.so

- Note
  - This interface is not supported currently.
  - This interface is used to obtain the inter-frame prediction property of the H.265 protocol coding channel.
  - This interface can be called after the encoding channel is destroyed after the encoding channel is created.
  - It is recommended that the user call this interface before starting the encoding after creating the channel, reducing the number of calls during the encoding process.

- Example

  See the example of MI_VENC_SetH264InterPred

- related topic

  None.

2.39. MI_VENC_SetH265IntraPred

- Description

  Set the intra prediction property of the H.265 protocol encoding channel.

- Syntax

MI_S32 MI_VENC_SetH265IntraPred(MI_VENC_DEV VeDev, MI_VENC_CHN VeChn, MI_VENC_ParamH265IntraPred_t*pstH265IntraPred);

➢ Parameters

parameter name	Description	Input/Output
VeDev	Encode device number	Input
VeChn	Encode channel number. Value range: [0, VENC_MAX_CHN_NUM).	input
pstH265IntraPred	Intra prediction configuration ponter of the H.265 protocol coding channel.	input

➢ Return value

- MI_SUCCESS: Successful
- Not MI_SUCCESS: Failed, see ERROR CODE for details.

➢ Dependence

- Header files: mi_venc.h, mi_venc_datatype.h
- Library file: libmi_venc.a/libmi_venc.so

※ Note

- This interface is not supported currently.
- This interface is used to set the intra prediction configuration of the H.265 protocol encoding channel.
- The intra-frame prediction is mainly determined by the properties of three parameters.
  1) u32Intra32x32Penalty: The probability that 32 x 32 block intra prediction is selected. The larger the value, the lower the probability.
  2) u32Intra16x16Penalty: The probability that 16 x 16 block intra prediction is selected. The larger the value, the lower the probability.
  3) u32Intra8x8Penalty: 8 x 8 block intra prediction is selected probability, the larger the value, the lower the probability.
- This interface is a high-level interface that can be called by the user. It is not recommended to call. The system has default values.
- This interface can be set before the code channel is destroyed after the code channel is created. When this interface is called during the encoding process, it takes effect until the next I frame.
- It is recommended that the user call this interface before starting the encoding after creating the channel, reducing the number of calls during the encoding process.
- It is recommended that the user call the MI_VENC_GetH265IntraPred interface to obtain the IntraPred configuration of the current encoding channel before calling this interface, and then set it.

➢ Example

```
MI_S32 H265SetIntraPred()
{
   MI_S32 s32Ret;
     MI_VENC_ParamH265IntraPred_t stIntraPred;
```

```
    MI_VENC_DEV VeDev = MI_VENC_DEV_ID_H264_H265_0;
  MI_VENC_CHN VeChnId = 0;

  //...omit other thing
  s32Ret = MI_VENC_GetH265IntraPred(VeDev, VeChnId, &stIntraPred);
  if (MI_SUCCESS != s32Ret)
  {
     printf("MI_VENC_GetH265IntraPred err0x%x\n", s32Ret);
     return E_MI_ERR_FAILED;
  }
  stIntraPred.u32Intra32x32Penalty = 0;
  stIntraPred.u32Intra16x16Penalty = 0;
  stIntraPred.u32Intra8x8Penalty = 10;
  s32Ret = MI_VENC_SetH265IntraPred(VeDev, VeChnId, &stIntraPred);
  if (MI_SUCCESS != s32Ret)
  {
     printf("MI_VENC_SetH265IntraPred err0x%x\n", s32Ret);
     return E_MI_ERR_FAILED;
  }

  return s32Ret;
}
```

- related topic

  None.

2.40. MI_VENC_GetH265IntraPred

- Description

  Obtain the intra prediction attribute of the H.265 protocol coding channel.

- Syntax

  MI_S32 MI_VENC_GetH265IntraPred(MI_VENC_DEV VeDev, MI_VENC_CHN VeChn, MI_VENC_ParamH265IntraPred_t*pstH265IntraPred);

- Parameters

parameter name	Description	Input/Output
VeDev	Encode device number	Input
VeChn	Encode channel number. Value range: [0, VENC_MAX_CHN_NUM).	input
pstH265IntraPred	The intra prediction parameter pointer of the H.265 protocol coding channel.	output

- Return value
  - MI_SUCCESS: Successful
  - Not MI_SUCCESS: Failed, see ERROR CODE for details.

- Dependence
  - Header files: mi_venc.h, mi_venc_datatype.h
  - Library file: libmi_venc.a/libmi_venc.so

- Note

- This interface is not supported currently.
- This interface is used to obtain the intra prediction property of the H.265 protocol coding channel.
- This interface can be called after the encoding channel is destroyed after the encoding channel is created.
- It is recommended that the user call this interface before starting the encoding after creating the channel, reducing the number of calls during the encoding process.

➢ Example

See the example of MI_VENC_SetH265IntraPred

➢ related topic

None.

2.41. MI_VENC_SetH265Trans

➢ Description

Set the transform and quantized attributes of the H.265 protocol encoding channel.

➢ Syntax

MI_S32 MI_VENC_SetH265Trans(MI_VENC_DEV VeDev, MI_VENC_CHN VeChn, MI_VENC_ParamH265Trans_t*pstH265Trans);

➢ Parameters

parameter name	Description	Input/Output
VeDev	Encode device number	Input
VeChn	Encode channel number. Value range: [0, VENC_MAX_CHN_NUM).	input
pstH265Trans	Transform and quantization attribute pointer of the H.265 protocol coding channel.	input

➢ Return value

- MI_SUCCESS: Successful
- Not MI_SUCCESS: Failed, see ERROR CODE for details.

➢ Dependence

- Header files: mi_venc.h, mi_venc_datatype.h
- Library file: libmi_venc.a/libmi_venc.so

➢ Note

- This interface is used to set the conversion and quantization configuration of the H.265 protocol encoding channel.
- Transform, quantization property mainly by three parameters.
  1) u32IntraTransMode: The transform attribute of the intra prediction macroblock. This is a reserved interface that is currently not supported.
  2) u32InterTransMode: The transform attribute of the inter prediction macroblock. This is a reserved interface that is currently not supported.

3) s32ChromaQpIndexOffset: For details, see the slice_cb_qp_offset and slice_cr_qp_offset of H.265 protocol.

- This interface is a high-level interface that can be called by the user. It is not recommended to call this interface. The system has default values.
- This interface can be set before the code channel is destroyed after the code channel is created. When this interface is called during the encoding process, it takes effect until the next I frame.
- It is recommended that the user call this interface before starting the encoding after creating the channel, reducing the number of calls during the encoding process.
- It is recommended that the user call the MI_VENC_GetH265Trans interface before calling this interface.

➢ Example

```
MI_S32 H265SetTrans()
{
    MI_S32 s32Ret;
    MI_VENC_ParamH265Trans_t stTrans;
      MI_VENC_DEV VeDev = MI_VENC_DEV_ID_H264_H265_0;
    MI_VENC_CHN VeChnId = 0;

    //...omit other thing
    s32Ret = MI_VENC_GetH265Trans(VeDev, VeChnId, &stTrans);
    if (MI_SUCCESS != s32Ret)
    {
        printf("MI_VENC_GetH265Trans err0x%x\n", s32Ret);
        return E_MI_ERR_FAILED;
    }
    stTrans.u32IntraTransMode = 2;
    stTrans.u32InterTransMode = 2;
    stTrans.s32ChromaQpIndexOffset = 2;
    s32Ret = MI_VENC_SetH265Trans(VeDev, VeChnId, &stTrans);
    if (MI_SUCCESS != s32Ret)
    {
        printf("MI_VENC_SetH265Trans err0x%x\n", s32Ret);
        return E_MI_ERR_FAILED;
    }

    return s32Ret;
}
```

➢ related topic

None.

2.42. MI_VENC_GetH265Trans

➢ Description

Obtain the transform and quantization attributes of the H.265 protocol coding channel.

➢ Syntax

MI_S32 MI_VENC_GetH265Trans(MI_VENC_DEV VeDev, MI_VENC_CHN VeChn, MI_VENC_ParamH265Trans_t *pstH265Trans);

- Parameters

parameter name	Description	Input/Output
VeDev	Encode device number	Input
VeChn	Encode channel number. Value range: [0, VENC_MAX_CHN_NUM).	input
pstH265Trans	The transform and quantization parameter pointer of the H.265 protocol coding channel.	output

- Return value
  - MI_SUCCESS: Successful
  - Not MI_SUCCESS: Failed, see ERROR CODE for details.

- Dependence
  - Header files: mi_venc.h, mi_venc_datatype.h
  - Library file: libmi_venc.a/libmi_venc.so

- Note
  - This interface is used to obtain the transform and quantization configuration of the H.265 protocol coding channel.
  - This interface can be called after the encoding channel is destroyed after the encoding channel is created.
  - It is recommended that the user call this interface before starting the encoding after creating the channel, reducing the number of calls during the encoding process.

- Example

  See the example of MI_VENC_SetH265Trans.

- related topic

  None.

2.43. MI_VENC_SetH264Dblk

- Description

  Set the Deblocking type of the H.264 protocol encoding channel.

- Syntax

  MI_S32 MI_VENC_SetH264Dblk(MI_VENC_DEV VeDev, MI_VENC_CHN VeChn, MI_VENC_ParamH264Dblk_t *pstH264Dblk);

- Parameters

parameter name	Description	Input/Output
VeDev	Encode device number	Input
VeChn	Encode channel number.	input

	Value range: [0, VENC_MAX_CHN_NUM).	
pstH264Dblk	Deblocking parameter pointer of the H.264 protocol encoding channel.	input

- Return value
  - MI_SUCCESS: Successful
  - Not MI_SUCCESS: Failed, see ERROR CODE for details.

- Dependence
  - Header files: mi_venc.h, mi_venc_datatype.h
  - Library file: libmi_venc.a/libmi_venc.so

- Note
  - This interface is used to set the Deblocking configuration of the H.264 protocol encoding channel.
  - The Deblocking attributes are mainly composed of three parameters:
    1) disable_deblocking_filter_idc: For details, see the H.264 protocol.
    2) Slice_alpha_c0_offset_div2: For details, see the H.264 protocol.
    3) Slice_beta_offset_div2: See the H.264 protocol for the specific meaning.
  - The system disables the deblocking function by default. The default disable_deblocking_filter_idc=0, slice_alpha_c0_offset_div2=0, slice_beta_offset_div2=0.
  - If the user wants to turn off the deblocking function, set disable_deblocking_filter_idc to 1.
  - This interface can be set before the code channel is destroyed after the code channel is created. When this interface is called during the encoding process, it takes effect until the next frame.
  - It is recommended that the user call this interface before starting the encoding after creating the channel, reducing the number of calls during the encoding process.
  - It is recommended that the user call the MI_VENC_GetH264Dblk interface before calling this interface to get the Dblk configuration and then set.

- Example

```
MI_S32 VencSetDblk()
{
   MI_S32 s32Ret;
     MI_VENC_ParamH264Dblk_t stDblk;
     MI_VENC_DEV VeDev = MI_VENC_DEV_ID_H264_H265_0;
   MI_VENC_CHN VeChnId = 0;

   //...omit other thing
   s32Ret = MI_VENC_GetH264Dblk(VeDev, VeChnId, &stDblk);
   if (MI_SUCCESS != s32Ret)
   {
      printf("MI_VENC_GetH264Dblk err0x%x\n", s32Ret);
      return E_MI_ERR_FAILED;
   }
   stDblk.disable_deblocking_filter_idc = 0;
```

```
    stDblk.slice_alpha_c0_offset_div2 = 6;
    stDblk.slice_beta_offset_div2 = 5;
    s32Ret = MI_VENC_SetH264Dblk(VeDev, VeChnId, &stDblk);
    if (MI_SUCCESS != s32Ret)
    {
        printf("MI_VENC_SetH264Dblk err0x%x\n", s32Ret);
        return E_MI_ERR_FAILED;
    }

    return s32Ret;
}
```

- related topic

  None.

2.44. MI_VENC_GetH264Dblk

- Description

  Get the Deblocking type of the H.264 protocol encoding channel.

- Syntax

  MI_S32 MI_VENC_GetH264Dblk(MI_VENC_DEV VeDev, MI_VENC_CHN VeChn, MI_VENC_ParamH264Dblk_t *pstH264Dblk);

- Parameters

parameter name	Description	Input/Output
VeDev	Encode device number	Input
VeChn	Encode channel number. Value range: [0, VENC_MAX_CHN_NUM).	input
pstH264Dblk	The Deblocking attribute pointer of the H.264 protocol encoding channel.	output

- Return value
  - MI_SUCCESS: Successful
  - Not MI_SUCCESS: Failed, see ERROR CODE for details.

- Dependence
  - Header files: mi_venc.h, mi_venc_datatype.h
  - Library file: libmi_venc.a/libmi_venc.so

- Note
  - This interface is used to obtain the Deblocking configuration of the H.264 protocol encoding channel.
  - This interface can be called after the encoding channel is destroyed after the encoding channel is created.
  - It is recommended that the user call this interface before starting the encoding after creating the channel, reducing the number of calls during the encoding process.

- Example

  See the example of MI_VENC_SetH264Dblk

- related topic

  None.

2.45. MI_VENC_SetH265Dblk

- Description

  Set the Deblocking type of the H.265 protocol encoding channel.

- Syntax

  MI_S32 MI_VENC_SetH265Dblk(MI_VENC_DEV VeDev, MI_VENC_CHN VeChn, MI_VENC_ParamH265Dblk_t*pstH265Dblk);

- Parameters

parameter name	Description	Input/Output
VeDev	Encode device number	Input
VeChn	Encode channel number. Value range: [0, VENC_MAX_CHN_NUM).	input
pstH265Dblk	H.265 Deblocking parameter pointer of the protocol coding channel.	input

- Return value
  - MI_SUCCESS: Successful
  - Not MI_SUCCESS: Failed, see ERROR CODE for details.

- Dependence
  - Header files: mi_venc.h, mi_venc_datatype.h
  - Library file: libmi_venc.a/libmi_venc.so

- Note
  - This interface is used to set the channel coding H.265 protocol configuration Deblocking
  - The Deblocking attributes are mainly composed of three parameters:
    1) disable_deblocking_filter_idc: For details, see the slice_deblocking_filter_disabled_flag of H.265 Protocol.
    2) slice_tc_offset_div2: For details, see H.265 Protocol.
    3) slice_beta_offset_div2: For details, see H.265 protocol.
  - The system disables the deblocking function by default. The default disable_deblocking_filter_idc=0, slice_tc_offset_div2=0, slice_beta_offset_div2=0.
  - If the user wants to turn off the deblocking function, set disable_deblocking_filter_idc to 1.
  - This interface can be set before the code channel is destroyed after the code channel is created. When this interface is called during the encoding process, it takes effect until the next frame.
  - It is recommended that the user call this interface before starting the encoding after

creating the channel, reducing the number of calls during the encoding process.
- It is recommended that the user call the MI_VENC_GetH265_Dblk interface to get the Deblocking configuration before calling this interface.

➢ Example

See MI_VENC_SetH264Dblk Example.

➢ related topic

None.

2.46. MI_VENC_GetH265Dblk

➢ Description

Get the Deblocking type of the H.265 protocol encoding channel.

➢ Syntax

MI_S32 MI_VENC_GetH265Dblk(MI_VENC_DEV VeDev, MI_VENC_CHN VeChn, MI_VENC_ParamH265Dblk_t *pstH265Dblk);

➢ Parameters

parameter name	Description	Input/Output
VeDev	Encode device number	Input
VeChn	Encode channel number. Value range: [0, VENC_MAX_CHN_NUM).	input
pstH265Dblk	H.265 Deblocking channel coding protocol attribute pointer.	output

➢ Return value

- MI_SUCCESS: Successful
- Not MI_SUCCESS: Failed, see ERROR CODE for details.

➢ Dependence

- Header files: mi_venc.h, mi_venc_datatype.h
- Library file: libmi.a

➢ Note

- This interface is used to obtain the Deblocking configuration of the H.265 protocol encoding channel.
- This interface can be called after the encoding channel is destroyed after the encoding channel is created.
- It is recommended that the user call this interface before starting the encoding after creating the channel, reducing the number of calls during the encoding process.

➢ Example

See the example of MI_VENC_SetH264Dblk

➢ related topic

None.

2.47.MI_VENC_SetH264Vui

- Description

  Set the VUI parameter of the H.264 protocol encoding channel.

- Syntax

  MI_S32 MI_VENC_SetH264Vui(MI_VENC_DEV VeDev, MI_VENC_CHN VeChn, MI_VENC_ParamH264Vui_t*pstH264Vui);

- Parameters

parameter name	Description	Input/Output
VeDev	Encode device number	Input
VeChn	Encode channel number. Value range: [0, VENC_MAX_CHN_NUM).	input
pstH264Vui	The VUI parameter pointer of the H.264 protocol encoding channel.	input

- Return value
  - MI_SUCCESS: Successful
  - Not MI_SUCCESS: Failed, see ERROR CODE for details.

- Dependence
  - Header files: mi_common.h, mi_venc.h
  - Library file: libmi_venc.a/libmi_venc.so

- Note
  - This interface is used to set the VUI configuration of the H.264 protocol encoding channel.
  - This interface can be set before the code channel is destroyed after the code channel is created. When this interface is called during the encoding process, it takes effect until the next I frame.
  - It is recommended that the user call this interface before starting the encoding after creating the channel, reducing the number of calls during the encoding process.
  - Users are advised before calling this interface, first call MI_VENC_GetH264Vui interface to get the current channel coding VUI configuration, and then set it.

- Example

```
MI_S32 SetVui()
{
   MI_S32 s32Ret;
    MI_VENC_ParamH264Vui_t stVui;
    MI_VENC_DEV VeDev = MI_VENC_DEV_ID_H264_H265_0;
   MI_VENC_CHN VeChnId = 0;

   //...omit other thing
```

```
    s32Ret = MI_VENC_GetH264Vui(VeDev, VeChnId, &stVui);
    if (MI_SUCCESS != s32Ret)
    {
        printf("MI_VENC_GetH264Vui err0x%x\n", s32Ret);
        return E_MI_ERR_FAILED;
    }
    stVui.stVuiTimeInfo.u8TimingInfoPresentFlag = 1;
    s32Ret = MI_VENC_SetH264Vui(VeDev, VeChnId,&stVui);
    if (MI_SUCCESS != s32Ret)
    {
        printf("MI_VENC_SetH264Vui err0x%x\n", s32Ret);
        return E_MI_ERR_FAILED;
    }

    return s32Ret;
}
```

- related topic

  None.

2.48.MI_VENC_GetH264Vui

- Description

  Obtain the VUI configuration of the H.264 protocol encoding channel.

- Syntax

  MI_S32 MI_VENC_GetH264Vui(MI_VENC_DEV VeDev, MI_VENC_CHN VeChn, MI_VENC_ParamH264Vui_t *pstH264Vui);

- Parameters

parameter name	Description	Input/Output
VeDev	Encode device number	Input
VeChn	Encode channel number. Value range: [0, VENC_MAX_CHN_NUM).	input
pstH264Vui	The VUI attribute pointer of the H.264 protocol encoding channel.	output

- Return value
  - MI_SUCCESS: Successful
  - Not MI_SUCCESS: Failed, see ERROR CODE for details.

- Dependence
  - Header files: mi_venc.h, mi_venc_datatype.h
  - Library file: libmi_venc.a/libmi_venc.so

- Note
  - This interface is used to obtain the VUI configuration of the H.264 protocol encoding channel.
  - This interface can be called after the encoding channel is destroyed after the encoding

channel is created.
- It is recommended that the user call this interface before starting the encoding after creating the channel, reducing the number of calls during the encoding process.

➢ Example

See the example of MI_VENC_SetH264Vui

➢ related topic

None.

2.49.MI_VENC_SetH265Vui

➢ Description

Set the VUI parameter of the H.265 protocol encoding channel.

➢ Syntax

MI_S32 MI_VENC_SetH265Vui(MI_VENC_DEV VeDev, MI_VENC_CHN VeChn, MI_VENC_ParamH265Vui_t *pstH265Vui);

➢ Parameters

parameter name	Description	Input/Output
VeDev	Encode device number	Input
VeChn	Encode channel number. Value range: [0, VENC_MAX_CHN_NUM).	input
pstH265Vui	H.265 VUI parameter pointer of the protocol coding channel.	input

➢ Return value

- MI_SUCCESS: Successful
- Not MI_SUCCESS: Failed, see ERROR CODE for details.

➢ Dependence

- Header files: mi_venc.h, mi_venc_datatype.h
- Library file: libmi_venc.a/libmi_venc.so

➢ Note

- This interface is used to set the VUI configuration of the H.265 protocol encoding channel.
- This interface can be set before the code channel is destroyed after the code channel is created. When this interface is called during the encoding process, it takes effect until the next I frame.
- It is recommended that the user call this interface before starting the encoding after creating the channel, reducing the number of calls during the encoding process.
- It is recommended that the user call the MI_VENC_GetH265 Vui interface before calling this interface to obtain the VUI configuration of the current encoding channel, and then set it.

- Example

  See the example of MI_VENC_SetH264Vui Example

- related topic

  None.

2.50. MI_VENC_GetH265Vui

- Description

  Obtain the VUI configuration of the H.265 protocol encoding channel.

- Syntax

  MI_S32 MI_VENC_GetH265Vui(MI_VENC_DEV VeDev, MI_VENC_CHN VeChn, MI_VENC_ParamH265Vui_t *pstH265Vui);

- Parameters

parameter name	Description	Input/Output
VeDev	Encode device number	Input
VeChn	Encode channel number. Value range: [0, VENC_MAX_CHN_NUM).	input
pstH265Vui	H.265 VUI attribute pointer of the protocol encoding channel.	output

- Return value
  - MI_SUCCESS: Successful
  - Not MI_SUCCESS: Failed, see ERROR CODE for details.

- Dependence
  - Header files: mi_venc.h, mi_venc_datatype.h
  - Library file: libmi_venc.a/libmi_venc.so

- Note
  - This interface is used to obtain the VUI configuration of the H.264 5 protocol encoding channel.
  - This interface can be called after the encoding channel is destroyed after the encoding channel is created.
  - It is recommended that the user call this interface before starting the encoding after creating the channel, reducing the number of calls during the encoding process.

- Example

  See the example of MI_VENC_SetH264Vui

- related topic

  None.

2.51. MI_VENC_SetH265SliceSplit

- Description

  Set the slice split attribute of H.265 channels.

- Syntax

  MI_S32 MI_VENC_SetH265SliceSplit(MI_VENC_DEV VeDev, MI_VENC_CHN VeChn, MI_VENC_ParamH265SliceSplit_t*pstSliceSplit);

- Parameters

parameter name	Description	Input/Output
VeDev	Encode device number	Input
VeChn	Encode channel number. Value range: [0, VENC_MAX_CHN_NUM).	input
pstSliceSplit	H.265 code stream slice segmentation parameter pointer.	input

- Return value
  - MI_SUCCESS: Successful
  - Not MI_SUCCESS: Failed, see ERROR CODE for details.

- Dependence
  - Header files: mi_venc.h, mi_venc_datatype.h
  - Library file: libmi_venc.a/libmi_venc.so

- Note
  - This interface is used to set the split mode of the H.265 protocol coded channel code stream.
  - Slice dividing property consists of two decision parameters.
    1) bSplitEnable: whether the current frame slice division.
    2) u32SliceRowCount: The slice is split according to the macro block line. u32SliceRowCount indicates that each slice occupies the number of image macroblock lines. And when encoding to the last few lines of the image, less than u32SliceRowCount, the remaining macroblock lines are divided into a slice.
  - This interface belongs to the advanced interface. The user can call it selectively. It is recommended not to call. The default bSplitEnable is false. This interface can be set before the code channel is destroyed after the code channel is created. This interface takes effect when the next frame is called when it is called during encoding.
  - It is recommended that the user call this interface before starting the encoding after creating the channel, reducing the number of calls during the encoding process.
  - It is recommended that the user call the MI_VENC_GetH265 SliceSplit interface to obtain the sliceplit configuration of the current channel before calling this interface, and then set it.

- Example

  See the example of MI_VENC_SetH264SliceSplit

- related topic

None.

2.52.MI_VENC_GetH265SliceSplit

- Description

  Get the slice split attribute of H.265 channel.

- Syntax

  MI_S32 MI_VENC_GetH265SliceSplit(MI_VENC_DEV VeDev, MI_VENC_CHN VeChn, MI_VENC_ParamH265SliceSplit_t *pstSliceSplit);

- Parameters

parameter name	Description	Input/Output
VeDev	Encode device number	Input
VeChn	Encode channel number. Value range: [0, VENC_MAX_CHN_NUM).	input
pstSliceSplit	H.265 code stream slice segmentation parameter pointer.	output

- Return value
  - MI_SUCCESS: Successful
  - Not MI_SUCCESS: Failed, see ERROR CODE for details.

- Dependence
  - Header files: mi_venc.h, mi_venc_datatype.h
  - Library file: libmi_venc.a/libmi_venc.so

- Note
  - This interface is used to obtain the slice split attribute of the H.265 protocol code channel.
  - This interface can be called after the encoding channel is destroyed after the encoding channel is created.
  - It is recommended that the user call this interface before starting the encoding after creating the channel, reducing the number of calls during the encoding process.

- Example

  See the example of MI_VENC_SetH264SliceSplit.

- related topic

  None.

2.53.MI_VENC_SetJpegParam

- Description

  Set advanced parameters for the JPEG protocol encoding channel.

- Syntax

  MI_S32 MI_VENC_SetJpegParam(MI_VENC_DEV VeDev, MI_VENC_CHN VeChn, MI_VENC_ParamJpeg_t *pstJpegParam);

- Parameters

parameter name	Description	Input/Output
VeDev	Encode device number	Input
VeChn	Encode channel number. Value range: [0, VENC_MAX_CHN_NUM).	input
pstJpegParam	The advanced parameter pointer of JPEG protocol encoding channel.	input

- Return value
  - MI_SUCCESS: Successful
  - Not MI_SUCCESS: Failed, see ERROR CODE for details.

- Dependence
  - Header files: mi_venc.h, mi_venc_datatype.h
  - Library file: libmi_venc.a/libmi_venc.so

- Note
  - This interface is used to set advanced parameters of the JPEG protocol encoding channel.
  - The advanced parameters are mainly composed of 4 parameters:
    1) u32Qfactor: The quantization table factor range is [1,90]. The larger the u32Qfactor is, the smaller the quantization coefficient in the quantization table is, the better the image quality will be, and the lower the compression ratio. Similarly the smaller the u32Qfactor is, the greater the quantization coefficients in the quantization table, the image quality will be obtained even worse, while a higher code compression rate. See the RFC2435 standard for the relationship between the specific u32Qfactor and the quantization table.
    2) au8YQt [64], au8CbCrQt [64]: corresponding to two quantization table space, by which the user can two sets a quantization table of user parameters.
    3) u32McuPerEcs: Each Ecs contains much Mcu. The system mode u32MCUPerECS=0 indicates that all MCUs of the current frame are encoded as one ECS. The minimum value of u32MCUPerECS is 0, and the maximum value does not exceed (picwidth+15)>>4x(picheight+15)>>4x2. This parameter is not supported yet.
  - If you want to use your own quantization table, set the Qfactor to 50 while setting the quantization table.
  - This interface can be set before the code channel is destroyed after the code channel is created. When this interface is called during the encoding process, it will take effect when the next frame is encoded.
  - It is recommended that the user call this interface before starting the encoding after creating the channel, reducing the number of calls during the encoding process.
  - Users are advised before calling this interface, first call MI_VENC_GetJpegParam interface to obtain when the front channel coding JpegParam configuration, and then set it.

- Example

```
MI_S32 SetJpegParam()
{
    MI_S32 s32Ret;
     MI_VENC_ParamJpeg_t stParamJpeg;
     MI_VENC_DEV VeDev = MI_VENC_DEV_ID_JPEG_0;
    MI_VENC_CHN VeChnId = 0;
    int i;

    //...omit other thing
    s32Ret = MI_VENC_GetJpegParam(VeDev, VeChnId, &stParamJpeg);
    if (MI_SUCCESS != s32Ret)
    {
        printf("MI_VENC_GetJpegParam err0x%x\n", s32Ret);
        return E_MI_ERR_FAILED;
    }
    stParamJpeg.u32MCUPerEcs = 100;
    for (i = 0; i < 64; i++)
    {
        stParamJpeg.au8YQt[i] = 16;
        stParamJpeg.au8CbCrQt[i] = 17;
    }
    s32Ret=MI_VENC_SetJpegParam(VeDev, VeChnId, &stParamJpeg);
    if (MI_SUCCESS != s32Ret)
    {
        printf("MI_VENC_SetJpegParam err0x%x\n",s32Ret);
        return E_MI_ERR_FAILED;
    }

    return s32Ret;
}
```

- related topic

  None.

2.54. MI_VENC_GetJpegParam

- Description

  Get the advanced parameter configuration of the JPEG protocol encoding channel.

- Syntax

  MI_S32 MI_VENC_GetJpegParam(MI_VENC_DEV VeDev, MI_VENC_CHN VeChn, MI_VENC_ParamJpeg_t *pstJpegParam);

- Parameters

parameter name	Description	Input/Output
VeDev	Encode device number	Input
VeChn	Encode channel number. Value range: [0, VENC_MAX_CHN_NUM).	input

pstJpegParam	Advanced parameter configuration pointer of Jpeg protocol encoding channel.	output

- Return value
  - MI_SUCCESS: Successful
  - Not MI_SUCCESS: Failed, see ERROR CODE for details.

- Dependence
  - Header files: mi_venc.h, mi_venc_datatype.h
  - Library file: libmi_venc.a/libmi_venc.so

- Note
  - This interface is used to obtain the advanced parameter configuration of the JPEG protocol encoding channel.
  - This interface can be called after the encoding channel is destroyed after the encoding channel is created.
  - It is recommended that the user call this interface before starting the encoding after creating the channel, reducing the number of calls during the encoding process.

- Example

  See the example of MI_VENC_SetJpegParam

- related topic

  None.

2.55.MI_VENC_SetRcParam

- Description

  Set advanced parameters for the code channel rate controller.

- Syntax

  MI_S32 MI_VENC_SetRcParam(MI_VENC_DEV VeDev, MI_VENC_CHN VeChn, MI_VENC_RcParam_t *pstRcParam);

- Parameters

parameter name	Description	Input/Output
VeDev	Encode device number	Input
VeChn	Encode channel number. Value range: [0, VENC_MAX_CHN_NUM).	input
pstRcParam	Advanced parameter pointer for encoding channel rate controllers.	input

- Return value
  - MI_SUCCESS: Successful
  - Not MI_SUCCESS: Failed, see ERROR CODE for details.

- Dependence

- Header files: mi_venc.h, mi_venc_datatype.h
- Library file: libmi_venc.a/libmi_venc.so

➢ Chip Difference

The rate control algorithms supported by different chips and different coding protocols are different, as shown in the following table:

Chip	H.264	H.265	JPEG
Pretzel	FIXQP/CBR/VBR	FIXQP/CBR/VBR	FIXQP/CBR
Macaron	FIXQP/CBR/VBR	FIXQP/CBR/VBR	FIXQP/CBR
Pudding	FIXQP/CBR/VBR/AVBR	FIXQP/CBR/VBR/AVBR	FIXQP/CBR
Ispahan	FIXQP/CBR/VBR/AVBR	FIXQP/CBR/VBR/AVBR	FIXQP/CBR
Tiramisu	FIXQP/CBR/VBR/AVBR	FIXQP/CBR/VBR/AVBR	FIXQP/CBR
Ikayaki	None	None	FIXQP/CBR
Muffin	FIXQP/CBR/VBR/AVBR	FIXQP/CBR/VBR/AVBR	FIXQP/CBR
Mochi	FIXQP/CBR/VBR/AVBR	FIXQP/CBR/VBR/AVBR	FIXQP/CBR
Maruko	FIXQP/CBR/VBR/AVBR	FIXQP/CBR/VBR/AVBR	FIXQP/CBR

➢ Note

- The advanced parameters of the coded channel rate controller have default values, rather than having to call this interface to start the code channel.
- It is recommended that the user first call the MI_VENC_GetRcParam interface to obtain the RC advanced parameters, then modify the corresponding parameters, and then call this interface to set the advanced parameters.
- The RC advanced parameters now only support H.264/H.265/Mjpeg rate control modes.
- The advanced parameters of the rate controller consist of the following parameters (Note: The parameters are not supported yet):
  u32ThrdI[RC_TEXTURE_THR_SIZE], u32ThrdP[RC_TEXTURE_THR_SIZE]: A set of thresholds for measuring the macroblock complexity of I frames and P frames, respectively. The thresholds are arranged in order from small to large, and each threshold has a value range of [0, 255]. This set of thresholds is used to appropriately adjust the Qp of each macroblock according to image complexity when performing macroblock level rate control. For H.264, the macroblock level rate control only adds direction (maximum plus 12), that is, if the image complexity of the current macroblock is between some two thresholds, the Qp value of the current macroblock starts at the macroblock line. Qp based on adding value X.
  The value of X is shown in the following table (C represents image complexity):

C Range	x
C<=u32Thrd[0]	0
u32Thrd[0]<C<=u32Thrd[1]	1
u32Thrd[1]<C<=u32Thrd[2]	2
u32Thrd[2]<C<=u32Thrd[3]	3
u32Thrd[3]<C<=u32Thrd[4]	4

C Range	x
u32Thrd[4]<C<=u32Thrd[5]	5
u32Thrd[5]<C<=u32Thrd[6]	6
u32Thrd[6]<C<=u32Thrd[7]	7
u32Thrd[7]<C<=u32Thrd[8]	8
u32Thrd[8]<C<=u32Thrd[9]	9
u32Thrd[9]<C<=u32Thrd[10]	10
u32Thrd[10]<C<=u32Thrd[11]	11
u32Thrd[11]<C	12

For H.265, macroblock The code rate control has both the plus direction (maximum plus 8) and the minus direction (maximum minus 4), that is, if the image complexity of the current macroblock is less than or equal to the u32Thr[3] threshold, the Qp value of the current macroblock is in the macro. Block rows starting Qp subtracting the value on the basis of X ; If the current macroblock image complexity is greater than u32Thr [3] the threshold value, the current macroblock Qp value is a macroblock row starting Qp based on adding value y. The values of X and Y are shown in the following table (C represents image complexity):

C Range	x or y
C<u32Thrd[0]	x=4
u32Thrd[0]<=C<u32Thrd[1]	x=3
u32Thrd[1]<=C<u32Thrd[2]	x=2
u32Thrd[2]<=C<u32Thrd[3]	x=1
u32Thrd[3]<=C<=u32Thrd[4]	x=y=0
u32Thrd[4]<C<=u32Thrd[5]	y=1
u32Thrd[5]<C<=u32Thrd[6]	y=2
u32Thrd[6]<C<=u32Thrd[7]	y=3
u32Thrd[7]<C<=u32Thrd[8]	y=4
u32Thrd[8]<C<=u32Thrd[9]	y=5
u32Thrd[9]<C<=u32Thrd[10]	y=6
u32Thrd[10]<C<=u32Thrd[11]	y=7
u32Thrd[11]<C	y=8

- u32RowQpDelta: The amplitude of the fluctuation of the starting QP of each row of macroblocks relative to the starting QP of the frame at the time of macroblock level rate control. For the next rate fluctuations more stringent scenario, this parameter can try to transfer large to achieve a more accurate rate control, but may cause image quality to the interior of the frame images have some difference different. At high bit rates, this value is recommended to be 0; the medium code rate is recommended to be 0 or 1; for low bit rates it is recommended to be 2 to 5. (Note: The parameters are not supported yet.)

- The CBR parameters are as follows:
  1) u32MinIPProp&u32MaxIPProp: CBR advanced parameters, which represent the minimum IP ratio and the maximum IP ratio. The IP ratio represents the ratio of the number of Bits of the I frame and the P frame. These two values are used to clamp the range of IP ratios. When u32MinIPProp is adjusted to be large, the I frame is clear and the P frame is blurred. When u32MaxIPProp is adjusted to be small, the I frame is blurred and the P frame is clear. Under normal circumstances, it is not recommended to constrain the IP size ratio to avoid breathing effects and code rate fluctuations. The default u32MinIPProp is set to 1 and u32MaxIPProp is set to 20. In a scenario where the I frame size is constrained, u32MinIPProp and u32MaxIPProp can be set according to the dependence on the I frame size fluctuation. u32MinIPProp is not supported yet.
  2) u32MaxQp&u32MinQp&u32MaxStartQp: CBR advanced parameters, u32MaxQp, u32MinQp represent the maximum QP and minimum QP of the current frame. This clamp has the strongest effect, and all other adjustments to the imageQP, such as macroblock rate control, are ultimately constrained to this maximum QP and minimum QP. u32MaxStartQp represents the maximum QP of the frame-level rate control output, which is in the range [u32MinQp, u32MaxQp]. Macroblock level rate control may cause QPs of some macroblocks to be larger or smaller than u32MaxStartQp, but will be clamped to [u32MinQp, u32MaxQp]. The default value u32MinQp is 12, u32MaxQp is 48, andu32MaxStartQp is 48. It is recommended not to change this group of parameters without special requirements for quality. u32MaxStartQp is not supported yet.
  3) u32MaxPPDeltaQp&u32MaxIPDeltaQp: The CBR advanced parameter indicates the maximum difference between the maximum difference between the starting Qp of two consecutive P frames and the starting QP of consecutive IP frames. When there is a large motion, scene switching, etc., the guaranteed code rate is stable, which may cause the QP difference between consecutive P frames to be too large, resulting in image mosaic, etc., and the QP changes can be clamped by adjusting these two parameters. Avoid image quality fluctuations. u32MaxPPDeltaQp and u32MaxIPDeltaQp are not supported yet.
  4) s32IPQPDelta: CBR advanced parameter, which indicates the difference between the average QP value and the current I frame QP. The CBR advanced parameter represents the difference between the average QP value and the QP of the current I frame. This parameter can be a positive or negative value to adjust the excessive I frame and breathing effect. The default value of the system is 2, increasing it will clear the I frame and reduce the breathing effect.
- The VBR parameters are as follows:
  1) s32IPQPDelta: The VBR advanced parameter indicates the maximum QP change between frames and frames during VBR quality fluctuations. This parameter can be used to prevent mosaics from appearing. The system defaults to 2.
  2) s32ChangePos: VBR advanced parameter, which indicates the ratio of the code rate to the maximum code rate when VBR starts to adjust QP. The system defaults to 90. If the content changes drastically and the maximum bit rate is not exceeded, it is recommended to reduce this value and increase s32IPQPDelta, but the code rate is small and the quality deviation is stable when the rate control is stable.
- The AVBR parameters are as follows.
  1) s32ChangePos: AVBR advanced parameter, which indicates the ratio of code rate to maximum code rate when AVBR starts to adjust QP. The default value of the system

is 80. If the content changes drastically and the maximum bit rate is not exceeded, it is recommended to reduce this value and increase s32IPQPDelta, but the code rate is small and the quality deviation is stable when the rate control is stable.

2) u32MaxQp&u32MinQp: AVBR advanced parameters, u32MaxQp and u32MinQp represent the maximum and minimum QP of the current frame. The clamping effect is the most powerful, and all other adjustments to image QP such as macroblock level rate control, will eventually be constrained to this maximum and minimum QP. The default value of u32MinQp is 12, and u32MaxQp is 48. If there is no special requirement for quality, it is not recommended to change this group of parameters.
3) u32MaxIQp&u32MinIQp: AVBR advanced parameters, u32MaxIQp and u32MinIQp represent the maximum and minimum QP of IDR frame in the current sequence. The clamping effect is the most powerful, and all other adjustment of image QP such as macroblock level rate control, will eventually be constrained to the maximum and minimum QP. If there is no special requirement for quality, it is not recommended to change this group of parameters.
4) u32MotionSensivity: AVBR advanced parameters for motion sensitivity. If this value is higher, it means the rate control will have faster response for motion info, but then it will be more sensitive to noise.

- pRcParam: RC advanced parameters formulated by the user, reserved, not used at the moment.
- It is recommended that the user call this interface before starting the encoding after creating the channel, reducing the number of calls during the encoding process.

➢ Example

```
MI_S32 VencSetRcParam()
{
   MI_S32 s32Ret;
     MI_VENC_RcParam_t stVencRcPara;
     MI_VENC_DEV VeDev = MI_VENC_DEV_ID_H264_H265_0;
   MI_VENC_CHN VeChnId = 0;

   //...omit other thing
   s32Ret = MI_VENC_GetRcParam(VeDev, VeChnId, &stVencRcPara);
   if (MI_SUCCESS != s32Ret)
   {
      printf("MI_VENC_GetRcParam err0x%x\n", s32Ret);
      return E_MI_ERR_FAILED;
   }

   stVencRcPara.stParamH264Cbr.s32IPQPDelta = 2;
   s32Ret = MI_VENC_SetRcParam(VeDev, VeChnId, &stVencRcPara);
   if (MI_SUCCESS != s32Ret)
   {
      printf("MI_VENC_SetRcParam err0x%x\n", s32Ret);
      return E_MI_ERR_FAILED;
```

```
    }
    //...omit other thing

    return s32Ret;
}
```

- related topic

  None.

2.56. MI_VENC_GetRcParam

- Description

  Get the channel rate control advanced parameters.

- Syntax

  MI_S32 MI_VENC_GetRcParam(MI_VENC_DEV VeDev, MI_VENC_CHN VeChn, MI_VENC_RcParam_t *pstRcParam);

- Parameters

parameter name	Description	Input/Output
VeDev	Encode device number	Input
VeChn	Encode channel number. Value range: [0, VENC_MAX_CHN_NUM).	input
pstRcParam	Channel rate control parameter pointer.	output

- Return value
  - MI_SUCCESS: Successful
  - Not MI_SUCCESS: Failed, see ERROR CODE for details.

- Dependence
  - Header files: mi_venc.h, mi_venc_datatype.h
  - Library file: libmi_venc.a/libmi_venc.so

- Note
  - This interface is used to obtain advanced parameters of the encoding channel rate control. The specific meaning of each parameter, please refer to MI_VENC_RcParam_t.
  - If pstRcParam is empty, it returns a failure.

- Example

  None.

- related topic

  None.

2.57. MI_VENC_SetRefParam

- Description

  Set the H.264/H.265 encoding channel advanced frame skipping reference parameters.

- Syntax

  MI_S32 MI_VENC_SetRefParam(MI_VENC_DEV VeDev, MI_VENC_CHN VeChn, MI_VENC_ParamRef_t*pstRefParam)

- Parameters

parameter name	Description	Input/Output
VeDev	Encode device number	Input
VeChn	Encode channel number. Value range: [0, VENC_MAX_CHN_NUM).	input
pstRefParam	H.264/H.265 coding channel advanced frame skipping reference parameter pointer.	input

- Return value
  - MI_SUCCESS: Successful
  - Not MI_SUCCESS: Failed, see ERROR CODE for details.

- Dependence
  - Header files: mi_venc.h, mi_venc_datatype.h
  - Library file: libmi_venc.a/libmi_venc.so

- Note
  - If pstRefParam is empty, it returns a failure.
  - Create H.264 / H.265 when channel coding protocol, the default mode is the reference frame skipping 1 times the reference frame skip mode. If the user needs to modify the frame skipping reference of the encoding channel, it is recommended to call this interface before starting the encoding after creating the channel, reducing the number of calls during the encoding process
  - When this interface is called during the encoding process, it takes effect until the next I frame.
  - If the user sets a time reference frame skip mode, the corresponding configuration is: bEnablePred = TRUE, u32Enhance = 0, u32Base = 1;
    If set 2 times the reference frame skip mode, the corresponding configuration is:bEnablePred = TRUE, u32Enhance = 1, u32Base = 1;
    If set 4 times the reference frame skip mode, the corresponding configuration is: bEnablePred = TRUE, u32Enhance = 3, u32Base = 1.
    For other than the above frame skip reference mode, the value of u32Enhance can refer to the following formula: u32Enhance = gop / (number of virtual I frames + 1)-1, u32Base can take any value.

- Example

```
MI_S32 VencSetRefParam()
{
   MI_S32 s32Ret;
```

```
    MI_VENC_ParamRef_t stRefParam;
    MI_VENC_DEV VeDev = MI_VENC_DEV_ID_H264_H265_0;
  MI_VENC_CHN VeChnId = 0;

  //...omit other thing
  s32Ret = MI_VENC_GetRefParam(VeDev, VeChn, &stRefParam);
  if (MI_SUCCESS != s32Ret)
  {
      printf("MI_VENC_GetRefParam err0x%x\n", s32Ret);
      return E_MI_ERR_FAILED;
  }

   //...omit other thing
  stRefParam.u32Base = 1;
  stRefParam.u32Enhance = 3;
  stRefParam.bEnablePred = TRUE;
  s32Ret = MI_VENC_SetRefParam(VeDev, VeChn, &stRefParam);
  if (MI_SUCCESS != s32Ret)
  {
      printf("MI_VENC_SetRefParam err0x%x\n", s32Ret);
      return E_MI_ERR_FAILED;
  }

  return s32Ret;
}
```

- related topic

  None.

2.58. MI_VENC_GetRefParam

- Description

  Obtain the advanced frame skipping reference parameters of the H.264/H.265 coding channel.

- Syntax

  MI_S32 MI_VENC_GetRefParam(MI_VENC_DEV VeDev, MI_VENC_CHN VeChn, MI_VENC_ParamRef_t*pstRefParam)

- Parameters

parameter name	Description	Input/Output
VeDev	Encode device number	Input
VeChn	Encode channel number. Value range: [0, VENC_MAX_CHN_NUM).	input
pstRefParam	H.264/H.265 coding channel advanced frame skipping reference parameter pointer.	output

- Return value
  - MI_SUCCESS: Successful
  - Not MI_SUCCESS: Failed, see ERROR CODE for details.

- Dependence
  - Header files: mi_venc.h, mi_venc_datatype.h
  - Library file: libmi_venc.a/libmi_venc.so

- Note

  If pstRefParam is empty, it returns a failure.

- Example

  Please refer to the example of MI_VENC_SetRefParam.

- related topic

  None.

2.59. MI_VENC_SetCrop

- Description

  Set the crop properties of the channel.

- Syntax

  MI_S32 MI_VENC_SetCrop (MI_VENC_DEV VeDev, MI_VENC_CHN VeChn, MI_VENC_CropCfg_t *pstCropCfg)

- Parameters

parameter name	Description	Input/Output
VeDev	EnEncode device number.	Input
VeChn	EnEncode channel number. Value range: [0, VENC_MAX_CHN_NUM).	input
pstCropCfg	Channel cropping property pointer.	input

- Return value
  - MI_SUCCESS: Successful
  - Not MI_SUCCESS: Failed, see ERROR CODE for details.

- Dependence
  - Header files: mi_venc.h, mi_venc_datatype.h
  - Library file: libmi_venc.a/libmi_venc.so

- Chip Difference

  The crop alignment parameters supported by different Chips are shown in the following table.

Chip	Alignment of (stRect.u32Left, stRect.u32Top, stRect.u32Width, stRect.u32Height)		
	H.264	H.265	JPEG
Pretzel	(16,16,8,2)	(16,16,8,2)	(256,16,8,2)

Chip	Alignment of (stRect.u32Left, stRect.u32Top, stRect.u32Width, stRect.u32Height)		
	H.264	H.265	JPEG
Macaron	(16,16,8,2)	(16,16,8,2)	(256,16,8,2)
Pudding	(16,16,8,2)	(16,16,8,2)	(256,16,8,2)
Ispahan	(16,16,8,2)	(16,16,8,2)	(256,16,8,2)
Tiramisu	(16,16,8,2)	(16,16,8,2)	(256,16,8,2)
Ikayaki	None	None	(256,16,8,2)
Muffin	(16,16,8,2)	(16,16,8,2)	(16,2,8,2)
Mochi	(16,16,8,2)	(16,16,8,2)	(16,2,8,2)
Maruko	(16,16,8,2)	(16,16,8,2)	(16,2,8,2)

- Note
  - This interface is used to set the cropping properties of the channel.
  - This interface must be called after the channel is created and before the channel is destroyed.
  - The crop property consists of two parts:
    1) bEnable: Whether the channel cropping function is enabled.
    2) stRect: Crop area properties, including the coordinates of the starting point of the crop area, and the size of the crop area.

- Example

  None.

- related topic

  None.

2.60. MI_VENC_GetCrop

- Description

  Get the crop properties of the channel.

- Syntax

  MI_S32 MI_VENC_GetCrop (MI_VENC_DEV VeDev, MI_VENC_CHN VeChn, MI_VENC_CropCfg_t*pstCropCfg)

- Parameters

parameter name	Description	Input/Output
VeDev	EnEncode device number.	Input
VeChn	EnEncode channel number. Value range: [0, VENC_MAX_CHN_NUM).	input
pstCropCfg	Channel cropping property pointer.	output

- Return value

- MI_SUCCESS: Successful
- Not MI_SUCCESS: Failed, see ERROR CODE for details.

➢ Dependence
- Header files: mi_venc.h, mi_venc_datatype.h
- Library file: libmi_venc.a/libmi_venc.so

➢ Note
- This interface is used to obtain the cropping properties of the channel.
- This interface must be called after the channel is created and before the channel is destroyed.

➢ Example

None.

➢ related topic

None.

2.61.MI_VENC_SetFrameLostStrategy

➢ Description

Set the frame loss policy when the code channel instantaneous code rate exceeds the threshold.

➢ Syntax

MI_S32 MI_VENC_SetFrameLostStrategy (MI_VENC_DEV VeDev, MI_VENC_CHN VeChn, MI_VENC_ParamFrameLost_t*pstFrmLostParam)

➢ Parameters

parameter name	Description	Input/Output
VeDev	Encode device number	Input
VeChn	Encode channel number. Value range: [0, VENC_MAX_CHN_NUM).	input
pstFrmLostParam	The parameter pointer of the encode channel drop frame strategy.	input

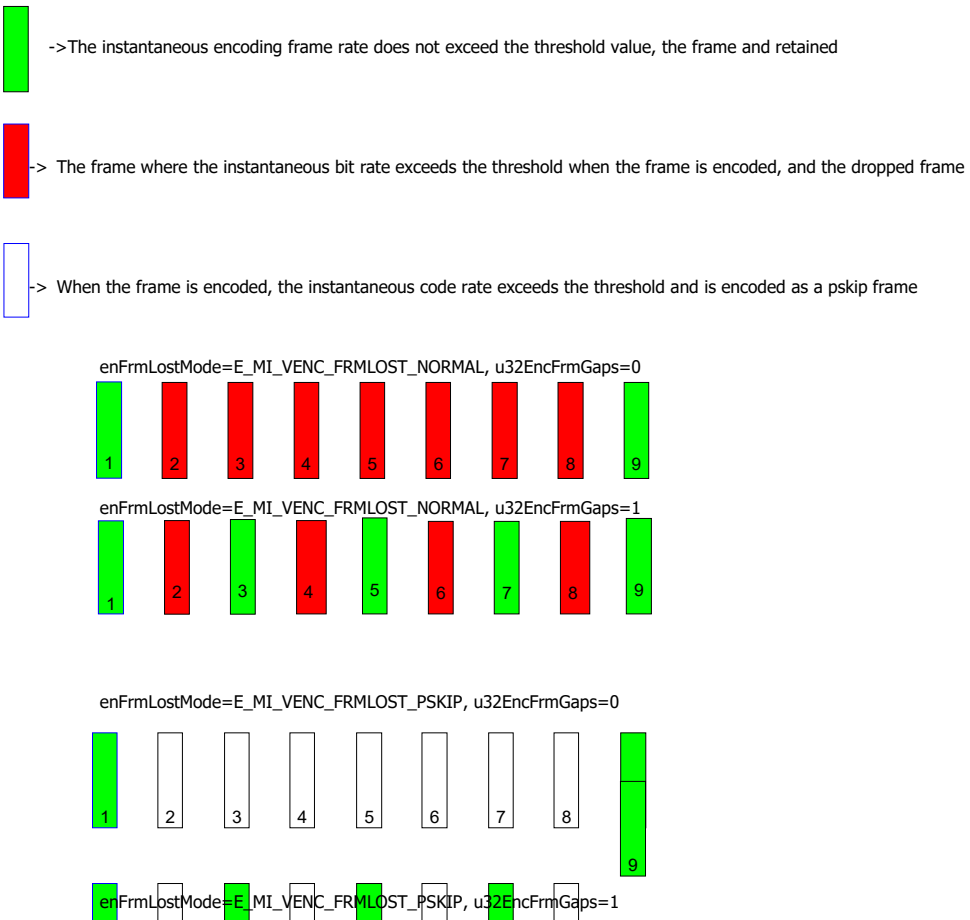
➢ Return value
- MI_SUCCESS: Successful
- Not MI_SUCCESS: Failed, see ERROR CODE for details.

➢ Dependence
- Header files: mi_venc.h, mi_venc_datatype.h
- Library file: libmi_venc.a/libmi_venc.so

➢ Note
- If pstFrmLostParam is empty, it returns a failure.
- pstFrmLostParam is mainly determined by four parameters.

1) eFrmLostMode: frame loss strategy mode.
2) u32EncFrmGaps: Drop frame interval.
3) bFrmLostOpen: Drop frame switch.
4) u32FrmLostBpsThr: Drop frame threshold.

- This interface belongs to the advanced interface. The user can select it selectively. The system has default values. By default, the frame is dropped when the instantaneous code rate exceeds the threshold.
- This interface provides two processing modes when the instantaneous bit rate exceeds the threshold: frame loss and encoding pskip frames. u32EncFrmGaps controls whether frames are evenly dropped or uniformly encoded pskip frames. (Note: It is not supported encoded pskip frames mode yet.) As the figure below:



- This interface can be set before the code channel is destroyed after the code channel is created. This interface takes effect when the next frame is called when it is called during encoding.

➢ Example

None.

➢ related topic

None.

2.62.MI_VENC_GetFrameLostStrategy

- Description

  Obtain the frame loss policy when the instantaneous code rate of the coding channel exceeds the threshold.

- Syntax

  MI_S32 MI_VENC_GetFrameLostStrategy(MI_VENC_DEV VeDev, MI_VENC_CHN VeChn, MI_VENC_ParamFrameLost_t*pstFrmLostParam);

- Parameters

parameter name	Description	Input/Output
VeDev	Encode device number.	Input
VeChn	Encode channel number. Value range: [0, VENC_MAX_CHN_NUM).	input
pstFrmLostParam	The parameter pointer of the encode channel drop frame strategy.	output

- Return value
  - MI_SUCCESS: Successful
  - Not MI_SUCCESS: Failed, see ERROR CODE for details.

- Dependence
  - Header files: mi_venc.h, mi_venc_datatype.h
  - Library file: libmi_venc.a/libmi_venc.so

- Note

  If pstFrmLostParam is empty, it returns a failure.

- Example

  None.

- related topic

  None.

2.63.MI_VENC_SetSuperFrameCfg

- Description

  Set the encoding jumbo frame configuration.

- Syntax

  MI_S32 MI_VENC_SetSuperFrameCfg(MI_VENC_DEV VeDev, MI_VENC_CHN VeChn, MI_VENC_SuperFrameCfg_t*pstSuperFrmParam);

- Parameters

parameter name	Description	Input/Output

VeDev	Encode device number	Input
VeChn	Encode channel number. Value range: [0, VENC_MAX_CHN_NUM).	input
pstSuperFrmParam	Encode jumbo frame configuration parameter pointer.	input

- Return value
  - MI_SUCCESS: Successful
  - Not MI_SUCCESS: Failed, see ERROR CODE for details.

- Dependence
  - Header files: mi_venc.h, mi_venc_datatype.h
  - Library file: libmi_venc.a/libmi_venc.so

- Chip Diffenrence

Whether different chips support re-encoding is as follows:

Chip	Whether to support
Pretzel	Y
Macaron	Y
Pudding	Y
Ispahan	N
Tiramisu	Y
Muffin	Y
Mochi	Y
Maruko	Y

- Note
  - If the channel is not created, it returns a failure.
  - This interface is a high-level interface that users can optionally call. The system default value is eSuperFrmMode = E_MI_VENC_SUPERFRM_NONE.
  - This interface can be set before the code channel is destroyed after the code channel is created.
  - When super frame mode is E_MI_VENC_SUPERFRM_NONE, MI do nothing.
  - When super frame mode is E_MI_VENC_SUPERFRM_DISCARD, MI will discard the super frame and continue encoding the next frame. If the chip version is after Macaron, MI will force to request I frame after discarding the super frame because of insufficient memory.
  - When super frame mode is E_MI_VENC_SUPERFRM_REENCODE, MI will increase the QP of the frame by 4 and reencode the super frame.If it still exceeds the threshold after repeating up to 4 times, MI will discard the frame.
  - Super frame processing is mainly to prevent the picture from getting stuck due to a certain frame being too large.The unit of threshold is bit.It is recommended to set it according to the actual scene.
  - If the previous binding relationship of VENC is E_MI_SYS_BIND_TYPE_HW_RING, the super frame mode is not supported.

- Example

  None.

- related topic

  None.

2.64. MI_VENC_GetSuperFrameCfg

- Description

  Get the encoded jumbo frame configuration.

- Syntax

  MI_S32 MI_VENC_GetSuperFrameCfg(MI_VENC_DEV VeDev, MI_VENC_CHN VeChn, MI_VENC_SuperFrameCfg_t*pstSuperFrmParam);

- Parameters

parameter name	Description	Input/Output
VeDev	Encode device number	Input
VeChn	Encode channel number. Value range: [0, VENC_MAX_CHN_NUM).	input
pstSuperFrmParam	Pointer to Encode jumbo frame configuration parameter.	output

- Return value
  - MI_SUCCESS: Successful
  - Not MI_SUCCESS: Failed, see ERROR CODE for details.

- Dependence
  - Header files: mi_venc.h, mi_venc_datatype.h
  - Library file: libmi_venc.a/libmi_venc.so

- Note

  If pstSuperFrmParam is empty, it returns a failure.

- Example

  None.

- related topic

  None.

2.65. MI_VENC_SetRcPriority

- Description

  Set the priority type for rate control.

- Syntax

  MI_S32 MI_VENC_SetRcPriority(MI_VENC_DEV VeDev, MI_VENC_CHN VeChn, MI_VENC_RcPriority_e *peRcPriority);

- Parameters

Parameter	Description	Input/Output
VeDev	Encode device number.	Input
VeChn	Encode channel number. Value range: [0, VENC_MAX_CHN_NUM).	Input
peRcPriority	Pointer to the priority type.	Input

- Return value
  - MI_SUCCESS: Successful
  - Not MI_SUCCESS: Failed, see ERROR CODE for details.

- Dependency
  - Header file: mi_venc.h, mi_venc_datatype.h
  - Library file: libmi_venc.a/libmi_venc.so

- Note
  - This interface is used to set the rate control to the target bit rate or to the high frame threshold with high priority.
  - When the rate control is high priority with the target code rate, when the code rate is insufficient, a large frame may be encoded to compensate the code rate, and the oversized frame will not be reprogrammed. When the rate control is based on the jumbo frame threshold being high priority, the jumbo frame will reprogram the number of bits, which may result in insufficient code rate.
  - This interface must be called after the code channel is created and before the code channel is destroyed.
  - This interface only supports the rate control mode of both H.264 and H.265 protocols.

- Example

  None.

- Related topic

  None.

2.66. MI_VENC_GetRcPriority

- Description

  Get the priority type of rate control.

- Syntax

  MI_S32 MI_VENC_GetRcPriority(MI_VENC_DEV VeDev, MI_VENC_CHN VeChn, MI_VENC_RcPriority_e *peRcPriority);

- Parameters

Parameter	Description	Input/Output
VeDev	Encode device number.	Input
VeChn	Encode channel number. Value range: [0, VENC_MAX_CHN_NUM).	Input
peRcPriority	Pointer to the priority type.	Output

- Return value
  - MI_SUCCESS: Successful
  - Not MI_SUCCESS: Failed, see ERROR CODE for details.

- Dependency
  - Header file: mi_venc.h, mi_venc_datatype.h
  - Library file: libmi_venc.a/libmi_venc.so

- Note

  This interface must be called after the code channel is created and before the code channel is destroyed.

- Example

  None.

- Related topic

  None.

2.67.MI_VENC_AllocCustomMap

- Description

  Allocate the memory for Custom Map used by smart encoding. The Custom Map includes QP Map and Mode Map.

- Syntax

  MI_S32 MI_VENC_ AllocCustomMap (MI_VENC_DEV VeDev, MI_VENC_CHN VeChn, MI_PHY *pstPhyAddr, void **ppCpuAddr);

- Parameters

Parameter	Description	Input/Output
VeDev	Encode device number.	Input
VeChn	Encode channel number. Value range: [0, VENC_MAX_CHN_NUM].	Input
pstPhyAddr	Pointer to the allocated virtual physical address.	Input/Output
ppCpuAddr	Pointer to the allocated CPU address.	Input/Output

- Return value
  - MI_SUCCESS: Successful
  - Not MI_SUCCESS: Failed, see ERROR CODE for details.

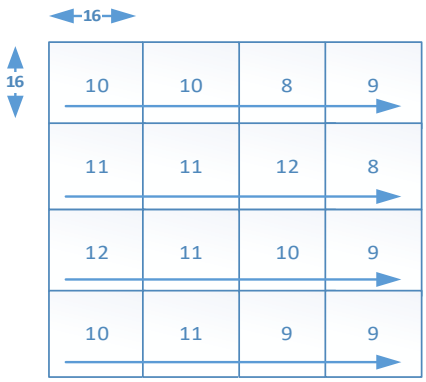
➢ Dependency

- Header file: mi_venc.h, mi_venc_datatype.h
- Library file: libmi_venc.a/libmi_venc.so

➢ Note

- This interface is used for allocating the memory needed by Custom Map. It returns the address of the allocated memory including the physical address and CPU address.
- If the memory allocation fails, the pstPhyAddr or ppCpuAddr is null and a failure will be returned.
- This interface must be called after the code channel is created and before the code channel is destroyed. The allocated memory will be free when the code channel is destroyed.
- The Custom Map includes QP Map and Mode Map. The two types of Custom Map could be applied simultaneously.
- In H.264, the QP Map contains QPs for every 16x16 block. The QP value for each 16x16 block could be set with the customer defined value. The QP Map is as illustrated below. The QP values should be successively set to the memory which is assigned for QP Map. The QP value could be set with relative or absolute value (refer to MI_VENC_SetAdvCustRcAttr). The value range is [12, 48] if absolute QP is configured, while the value range should be [0, 63] if relative QP is configured. (The efficient QP value used is the relative QP value minus 32 and the corresponding efficient value range is [-32, 31].)
- The Mode Map contains Mode value for every 16x16 block. The Mode value for each 16x16 block could be set with the customer defined value. The Mode Map is as illustrated below. The Mode values should be successively set to the memory which is assigned for Mode Map.
- One byte with Mode/QP value could be set every 16x16 blocks. The configuration data struct is as shown below.

```
struct {
            uint8  qp     : 6;
            uint8  mode  : 2;
} H264CustomMapConf;
```

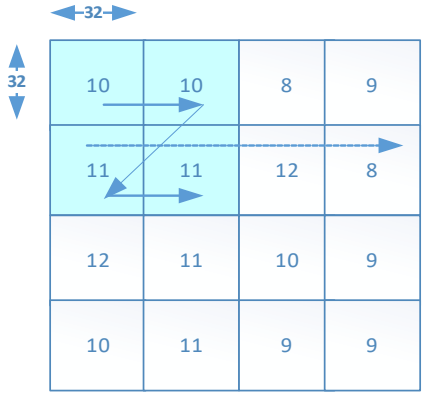


AVC

- In H.265, the QP Map contains QPs for every 4x32x32 block. The QP value for each 4x32x32 block could be set with the customer defined value. The QP Map is as illustrated below. The QP values should be successively set to the memory which is assigned for QP

Map. The QP value could be set with relative or absolute value (refer to MI_VENC_SetAdvCustRcAttr).
The Mode Map contains Mode value for every 4x32x32 block. The Mode value for each 4x32x32 block could be set with the customer defined value. The Mode Map is just the same as the QP Map. The Mode values should be successively set to the memory which is assigned for Mode Map.
Four bytes with QP/Mode value could be set every 4x32x32 blocks. The configuration data structure is as shown below. The Mode value uses 2 bits. The QP value for every 32x32 block uses 6 bits.

```
struct {
        uint32  sub_ctu_qp_0    : 6;
        uint32  mode            : 2;
        uint32  sub_ctu_qp_1    : 6;
        uint32  reserved_0      : 2;
        uint32  sub_ctu_qp_2    : 6;
        uint32  reserved_1      : 2;
        uint32  sub_ctu_qp_3    : 6;
        uint32  reserved_2      : 2;
} H265CustomMapConf;
```



HEVC

- Example

  None.

- Related topic

  None.

2.68. MI_VENC_ApplyCustomMap

- Description

  Apply the configured Custom Map.

- Syntax

  MI_S32 MI_VENC_ApplyCustomMap (MI_VENC_DEV VeDev, MI_VENC_CHN VeChn, MI_PHY PhyAddr);

- Parameters

Parameter	Description	Input/Output
VeDev	Encode device number	Input
VeChn	Encode channel number. Value range: [0, VENC_MAX_CHN_NUM].	Input
PhyAddr	The allocated virtual physical address.	Input

- Return value
  - MI_SUCCESS: Successful
  - Not MI_SUCCESS: Failed, see ERROR CODE for details.

- Dependency
  - Header file: mi_venc.h, mi_venc_datatype.h
  - Library file: libmi_venc.a/libmi_venc.so

- Note
  - This interface is used for applying the configuration after the customer defined Custom Map is configured into the corresponding address.
  - This interface must be called after the Custom Map is initialized or modified.

- Example

  None.

- Related topic

  None.

2.69.MI_VENC_GetLastHistoStaticInfo

- Description

  Get the output information when the latest frame is encoded.

- Syntax

  MI_S32 MI_VENC_GetLastHistoStaticInfo (MI_VENC_DEV VeDev, MI_VENC_CHN VeChn, MI_VENC_FrameHistoStaticInfo_t *ppFrmHistoStaticInfo);

- Parameters

Parameter	Description	Input/Output
VeDev	Encode device number	Input
VeChn	Encode channel number. Value range: [0, VENC_MAX_CHN_NUM].	Input
ppFrmHistoStaticInfo	Pointer to the address of the saved output information for the latest encoded frame.	Output

- Return value
  - MI_SUCCESS: Successful
  - Not MI_SUCCESS: Failed, see ERROR CODE for details.

- Dependency
  - Header file: mi_venc.h, mi_venc_datatype.h
  - Library file: libmi_venc.a/libmi_venc.so

- Note
  - This interface is used for getting the output information when the latest frame is encoded. It includes the encoded frame type, the number of blocks and the frame size, etc.
  - If the getting of the output content fails, it returns a failure.
  - This interface must be called after the code channel is created and before the encode channel is destroyed. The allocated memory would be free when the code channel is destroyed or when the memory is released by using the interface below.

- Example

  None.

- Related topic

  None.

2.70. MI_VENC_ReleaseHistoStaticInfo

- Description

  Release the memory used for the output information when the latest frame is encoded.

- Syntax

  MI_S32 MI_VENC_ReleaseHistoStaticInfo (MI_VENC_DEV VeDev, MI_VENC_CHN VeChn);

- Parameters

Parameter	Description	Input/Output
VeDev	Encode device number	Input
VeChn	Encode channel number. Value range: [0, VENC_MAX_CHN_NUM].	Input

- Return value
  - MI_SUCCESS: Successful
  - Not MI_SUCCESS: Failed, see ERROR CODE for details.

- Dependency
  - Header file: mi_venc.h, mi_venc_datatype.h
  - Library file: libmi_venc.a/libmi_venc.so

- Note
  - This interface is not supported currently.
  - This interface is used for releasing memory used for saving the output information when the latest frame is encoded.

- Example

None.

- Related topic

None.

2.71. MI_VENC_SetAdvCustRcAttr

- Description

Set the customer defined advanced RC configuration parameters.

- Syntax

MI_S32 MI_VENC_ SetAdvCustRcAttr (MI_VENC_DEV VeDev, MI_VENC_CHN VeChn, MI_VENC_AdvCustRcAttr_t *pstAdvCustRcAttr);

- Parameters

Parameter	Description	Input/Output
VeDev	Encode device number	Input
VeChn	Encode channel number. Value range: [0, VENC_MAX_CHN_NUM].	Input
pstAdvCustRcAttr	Customer defined advanced RC configuration parameter pointer.	Input

- Return value
  - MI_SUCCESS: Successful
  - Not MI_SUCCESS: Failed, see ERROR CODE for details.

- Dependency
  - Header file: mi_venc.h, mi_venc_datatype.h
  - Library file: libmi_venc.a/libmi_venc.so

- Note
  - This interface is used for setting the customer defined advanced RC configuration parameters.
  - This interface must be called after the code channel is created and before the picture starts to be received.
  - After the switch of customer defined advanced RC configuration is set, the related function could be started by calling the functions MI_VENC_AllocCustomMap and MI_VENC_GetLastHistoStaticInfo.

- Example

None.

- Related topic

None.

2.72.MI_VENC_SetInputSourceConfig

- Description

  Set H.264/H.265 input source info.

- Syntax

  MI_S32 MI_VENC_SetInputSourceConfig(MI_VENC_DEV VeDev, MI_VENC_CHN VeChn, MI_VENC_InputSourceConfig_t *pstInputSourceConfig);

- Parameters

parameter name	Description	Input/Output
VeDev	Encode device number.	Input
VeChn	Encode channel number. Value range: [0, VENC_MAX_CHN_NUM).	input
pstInputSourceConfig	Pointer to input source info.	input

- Return value
  - MI_SUCCESS: Successful
  - Not MI_SUCCESS: Failed, see ERROR CODE for details.

- Dependence
  - Header files: mi_venc.h, mi_venc_datatype.h
  - Library file: libmi_venc.a/libmi_venc.so

- Note

  When using input ring mode or hw auto sync mode, different chips have different restrictions on whether to call this interface, as shown in the following table:

Chip	Whether to call this interface when using input ring mode or hw auto sync mode?
Pretzel	N
Macaron	N
Pudding	N
Ispahan	Y
Tiramisu	Y
Ikayaki	Y
Muffin	Y
Mochi	Y
Maruko	Y

  - If the channel is not created, it returns a failure.
  - This interface must be called after the code channel is created and before the picture starts to be received.
  - It allows to be called for at most one channel to set input ring info or input ring hw auto sync info.
  - If set E_MI_VENC_INPUT_MODE_RING_ONE_FRM or

E_MI_VENC_INPUT_MODE_RING_HALF_FRM or E_MI_VENC_INPUT_MODE_HW_AUTO_SYNC, then APP should set E_MI_SYS_BIND_TYPE_HW_RING or E_MI_SYS_BIND_TYPE_HW_AUTOSYNC and the corresponding ring buffer height when call MI_SYS_BindChnPort2. For example, if the resolution is 1920x1080 and set E_MI_VENC_INPUT_MODE_RING_ONE_FRM, then the ring buffer height should set 1080; if set E_MI_VENC_INPUT_MODE_RING_HALF_FRM, then the ring buffer height should set 540.

- The Maruko platform supports E_MI_VENC_INPUT_MODE_RING_UNIFIED_DMA mode, and the front level is SCL module, configured by the VENC main channel. App needs to set E_MI_SYS_BIND_TYPE_HW_RING when calling MI_SYS_BindChnPort2, and u32BindParam can be set to 0.
- The Maruko platform supports the binding between different channels of VENC module. The APP needs to set E_MI_SYS_BIND_TYPE_HW_RING when calling MI_SYS_BindChnPort2. Under this usage, we cannot call MI_VENC_SetInputSourceConfig to set the buf mode as E_MI_VENC_INPUT_MODE_RING_UNIFIED_DMA. The correct way is to set the buf mode as E_MI_VENC_INPUT_MODE_NORMAL_FRMBASE or not to call this API.

➢ Example

None.

➢ related topic

None.

2.73. MI_VENC_SetSmartDetInfo

➢ Description

For third-party intelligent detection algorithms to provide VENC with statistics needed for intelligent coding.

➢ Syntax

MI_S32 MI_VENC_ SetSmartDetInfo(MI_VENC_DEV VeDev, MI_VENC_CHN VeChn, MI_VENC_SmartDetInfo_t *pstSmartDetInfo);

➢ Parameters

Parameter	Description	Input/Output
VeDev	Encode device number.	Input
VeChn	Encode channel number. Value range: [0, VENC_MAX_CHN_NUM].	Input
pstSmartDetInfo	Pointer to related statistics of intelligent detection algorithm.	Input

➢ Return value

- MI_SUCCESS: Successful
- Not MI_SUCCESS: Failed, see ERROR CODE for details.

- Dependency
  - Header file: mi_venc.h, mi_venc_datatype.h
  - Library file: libmi_venc.a/libmi_venc.so

- Note
  - It will return failure if the channel is not created
  - This interface must be called after the code channel is created and before the picture starts to be received.
  - This interface parameter setting takes effect on Pudding, Ispahan, Tiramisu and Muffin.

- Example

  None.

- Related topic

  None.

2.74. MI_VENC_SetIntraRefresh

- Description

  Set H.264/H.265 P-frame brush Islice. When IntraResearch mode is turned on, only one IDR is present at the beginning of the encoding, and then there will be P-frame only. The advantage of enabling IntraRefresh mode is that, by scattering the original IDR coded IntraLCU/Macroblock in a number of P-frames, the size of the frames will be relatively similar, without affecting the IDR frame quality.

- Syntax

  MI_S32 MI_VENC_SetIntraRefresh(MI_VENC_DEV VeDev, MI_VENC_CHN VeChn, MI_VENC_IntraRefresh_t *pstIntraAttr);

- Parameters

Parameter	Description	Input/Output
VeDev	Encode device number.	Input
VeChn	Encode channel number. Value range: [0, VENC_MAX_CHN_NUM].	Input
pstIntraAttr	Input Intra configuration information.	Input

- Return value
  - MI_SUCCESS: Successful
  - Not MI_SUCCESS: Failed, see ERROR CODE for details.

- Dependency
  - Header file: mi_venc.h, mi_venc_datatype.h
  - Library file: libmi_venc.a/libmi_venc.so

- Chip Difference

  The following table lists chips that support H.264 and H.265 encoder intra refresh feature:

Chip	H264 and H265 intra refresh supported or not?
Pudding	Y
Ispahan	Y
Tiramisu	Y
Muffin	Y
Mochi	Y
Maruko	Y

➢ Note

- If the channel is not created, it returns a failure.
- This interface must be called after the code channel is created and before the picture starts to be received.
- u32RefreshLineNum value range: total_mb_count/gop <u32RefreshLineNum <= total_mb_count, for example: 1080P, gop = 30, mb_height = 16, value range: total_mb_count = 1088/16 = 68, total_mb_count/gop = 68/30 = 2.26, then 2.26 <u32RefreshLineNum<=68. In order to reduce the bit rate, it is recommended that the value of u32RefreshLineNum be smaller, otherwise the P frame of the entire gop will be refreshed to Islice.

➢ Example

```
MI_S32 SetIntraRefresh()
{
    MI_S32 s32Ret;
    MI_VENC_DEV VeDev = MI_VENC_DEV_ID_H264_H265_0;
  MI_VENC_CHN VeChn = 0;
    MI_VENC_IntraRefresh_t stIntraAttr;

  //...omit other thing
  s32Ret = MI_VENC_GetIntraRefresh(VeDev, VeChnId, &stIntraAttr);
  if (MI_SUCCESS != s32Ret)
  {
     printf("MI_VENC_GetIntraRefresh err0x%x\n", s32Ret);
     return E_MI_ERR_FAILED;
  }
    stIntraAttr.bEnable = TRUE;
    stIntraAttr.u32RefreshLineNum = 2;
    stIntraAttr.u32ReqIQp = FALSE;
    s32Ret = MI_VENC_SetIntraRefresh(VeDev, VeChn, &stIntraAttr);
  if (MI_SUCCESS != s32Ret)
  {
       printf("MI_VENC_SetIntraRefresh err 0x%x\n", s32Ret);
     return E_MI_ERR_FAILED;
```

```
    }

    return s32Ret;
}
```

- Related topic

  None.

2.75. MI_VENC_GetIntraRefresh

- Description

  Get H.264/H.265 Intra Refresh Parameter.

- Syntax

  MI_S32 MI_VENC_GetIntraRefresh(MI_VENC_DEV VeDev, MI_VENC_CHN VeChn, MI_VENC_IntraRefresh_t *pstIntraAttr);

- Parameters

Parameter	Description	Input/Output
VeDev	Encode device number.	Input
VeChn	Encode channel number. Value range: [0, VENC_MAX_CHN_NUM].	Input
pstIntraAttr	Output Intra configuration information pointer.	Output

- Return value
  - MI_SUCCESS: Successful
  - Not MI_SUCCESS: Failed, see ERROR CODE for details.

- Dependency
  - Header file: mi_venc.h, mi_venc_datatype.h
  - Library file: libmi_venc.a/libmi_venc.so

- Chip Difference

  The following table lists chips that support H.264 and H.265 encoder intra refresh feature:

Chip	Intra refresh supported or not?
Pudding	Y
Ispahan	Y
Tiramisu	Y
Muffin	Y
Mochi	Y
Maruko	Y

- Note
  - If the channel is not created, it returns a failure.

- This interface must be called after the code channel is created and before the picture starts to be received.

➢ Example

Please refer to the example of MI_VENC_SetIntraRefresh.

➢ Related topic

None.

2.76. MI_VENC_DupChn

➢ Description

Synchronize status of VENC channel.

➢ Syntax

MI_S32 MI_VENC_DupChn(MI_VENC_DEV VeDev, MI_VENC_CHN VeChn);

➢ Parameters

Parameter	Description	Input/Output
VeDev	Encode device number	Input
VeChn	Encode channel number. Value range: [0, VENC_MAX_CHN_NUM).	Input

➢ Return value

- MI_SUCCESS: Successful
- Not MI_SUCCESS: Failed, see ERROR CODE for details.

➢ Dependency

- Header file: mi_venc.h, mi_venc_datatype.h
- Library file: libmi_venc.a/libmi_venc.so

➢ Note

- This interface applies only to dual OS environments.The venc channel is initialized in the RTK environment and used to synchronize the status of the venc channel when switching to the Linux environment.
- After Linux dup channel, the RTK app can no longer operate the venc channel.

➢ Example

None.

➢ Related topic

None.

3. VENC DATA TYPE

The relevant data types and data structures are defined as follows:

data type	define
MI_VENC_DEV_ID_H264_H265_0	Defines H264 & h265 coding device 0
MI_VENC_DEV_ID_H264_H265_1	Defines H264 & h265 coding device 1
MI_VENC_DEV_ID_JPEG_0	Defines JPEG coding device 0
MI_VENC_DEV_ID_JPEG_1	Defines JPEG coding device 1
RC_TEXTURE_THR_SIZE	Define the threshold number of texture level code control
MI_VENC_H264eNaluType_e	Define H.264 code stream NALU type
MI_VENC_H264eRefSliceType_e	Defining which reference frame in the frame skip reference mode the acquired H.264 code stream belongs to
M I_VENC_H264eRefType_e	Define the frame type and reference attribute of the H.264 frame skip reference stream
MI_VENC_JpegePackType_e	Define the PACK type of the JPEG stream
MI_VENC_H265eNaluType_e	Define H.265 code stream NALU type
MI_VENC_DataType_t	Defining code stream structure union
MI_VENC_PackInfo_t	A structure that defines other types of codestream data contained in the current stream packet data
MI_VENC_Pack_t	Define a frame code stream packet structure
MI_VENC_StreamInfoH264_t	Define H.264 protocol code stream feature information
MI_VENC_StreamInfoJpeg_t	Define JPEG protocol stream feature information
MI_VENC_StreamInfoH265_t	Define H.265 protocol code stream feature information
MI_VENC_Stream_t	Define the frame code stream type structure
MI_VENC_StreamBufInfo_t	Structure for defining code stream buffer information
MI_VENC_AttrH264_t	Define the H.264 encoder attribute structure
MI_VENC_AttrJpeg_t	Define JPEG capture encoder attribute structure
MI_VENC_AttrH265_t	Define the H.265 encoder attribute structure
MI_VENC_Attr_t	Define the encoder attribute structure
MI_VENC_ChnAttr t	Define the encoding channel attribute structure
MI_VENC_ChnStat_t	Define the state structure of the code channel
MI_VENC_ParamH264SliceSplit_t	Define the H.264 encoding channel slice segmentation attribute
MI_VENC_ParamH264InterPred_t	Defining H.264 encoding channel inter prediction properties
MI_VENC_ParamH264Trans_t	Define H.264 encoding channel transform, quantization properties
MI_VENC_ParamH264Entropy_t	Define H.264 encoding channel entropy encoding properties
MI_VENC_ParamH265InterPred_t	Defining H.265 encoding channel inter prediction properties

data type	define
MI_VENC_ParamH265IntraPred_t	Define H.265 encoding channel intra prediction properties
MI_VENC_ParamH265Trans_t	Define H.265 encoding channel transform, quantization properties
MI_VENC_ParamH264Dblk_t	Define the H.264 encoding channel Deblocking property
MI_VENC_ParamH264Vui_t	Define H.264 encoding channel VUI properties
MI_VENC_ParamH264VuiAspectRatio_t	A structure defining the AspectRatio information in the H.264 protocol encoding channel VUI
MI_VENC_ParamH264Vui TimeInfo_t	A structure defining TIME_INFO information in the H.264 protocol encoding channel VUI
MI_VENC_Param H264 VuiVideoSignal_t	A structure defining the VIDEO_SIGNAL information in the H.264 protocol encoding channel VUI
MI_VENC_ParamJpeg_t	Define JPEG encoding parameter sets
MI_VENC_RoiCfg_t	Define the coding channel interest area coding attribute
MI_VENC_RoiBgFrameRate_t	Define the frame rate attribute of a non-ROI area
MI_VENC_RcAttr_t	Define the encoding channel rate controller attribute
MI_VENC_RcMode_e	Define the code channel rate controller mode
MI_VENC_AttrH264Cbr_t	Define the HBR encoding channel CBR attribute structure
MI_VENC_AttrH264Vbr_t	Define the HBR encoding channel VBR attribute structure
MI_VENC_AttrH264FixQp_t	Define the H.264 encoding channel Fixqp attribute structure
MI_VENC_AttrH264Abr t	Define the HBR encoding channel ABR attribute structure
MI_VENC_SuperFrmMode_e	Define the jumbo frame processing mode in rate control
MI_VENC_AttrH265Cbr t	Define the H.265 encoding channel CBR attribute structure
MI_VENC_AttrH265Vbr t	Define the HBR attribute channel VBR attribute structure
MI_VENC_AttrH265FixQp_t	Define the H.265 encoding channel Fixqp attribute structure
MI_VENC_ParamH264Vbr_t	Define H264 protocol encoding channel VBR rate control mode advanced parameter configuration
MI_VENC_ParamH264Cbr_t	Define H264 protocol encoding channel CBR new version rate control mode advanced parameter configuration
MI_VENC_ParamH265Vbr_t	Define H265 protocol encoding channel VBR rate control mode advanced parameter configuration
MI_VENC_ParamH265Cbr_t	Define H265 protocol encoding channel CBR new version rate control mode advanced parameter configuration
MI_VENC_RcParam_t	Define the rate control advanced parameters of the coding channel
MI_VENC_CropCfg_t	Define channel crop parameters
MI_VENC_RecvPicParam_t	Receive the specified frame number image encoding

data type	define
MI_VENC_H264eIdrPicIdMode_e	Set the mode of IDR frame or Idr_pic_id of I frame
MI_VENC_H264IdrPicIdCfg_t	Idr_pic_id parameter of IDR frame or I frame
MI_VENC_FrameLostMode_e	Defining the frame loss mode when the code channel instantaneous code rate exceeds the threshold
MI_VENC_ParamFrameLost_t	Defining the frame loss strategy when the code channel instantaneous code rate exceeds the threshold
MI_VENC_SuperFrameCfg_t	Large frame processing strategy parameters
MI_VENC_RcPriority_e	Rate control priority enumeration
MI_VENC_ModParam_t	Define encoding module parameters
MI_VENC_ModType_e	Define module parameter types
MI_VENC_ParamModH265e_t	Define mi_h265.ko module parameters
MI_VENC_ParamModJpege_t	Define mi_jpege.ko module parameters
MI_VENC_AdvCustRcAttr_t	Define customer defined advanced RC configuration parameters
MI_VENC_FrameHistoStaticInfo_t	Define attributes of the encoded frame
MI_VENC_InputSourceConfig_t	Define H.264/H.265 input buffer config info structure
MI_VENC_InputSrcBufferMode_e	Define H.264/H.265 input buffer mode
MI_VENC_AttrH264Avbr_t	Define the H.264 encoding channel AVBR attribute structure
MI_VENC_AttrH265Avbr_t	Define the H.265 encoding channel AVBR attribute structure
MI_VENC_ParamH264Avbr_t	Define H264 protocol encoding channel AVBR rate control mode advanced parameter configuration
MI_VENC_ParamH265Avbr_t	Define H265 protocol encoding channel AVBR rate control mode advanced parameter configuration
MI_VENC_SmartDetInfo_t	Define the statistics required by the intelligent detection algorithm
MI_VENC_IntraRefresh_t	Support P-frame containing Islice
MI_VENC_InitParam_t	Venc device initialization parameter
MI_VENC_H265eRefType_e	Define the frame type and reference attribute of the H.265 frame skip reference stream

3.1. MI_VENC_DEV_ID_H264_H265_0

- Description

  Defines H264 & h265 coding device 0.

- Definition

  #define MI_VENC_DEV_ID_H264_H265_0 0

- Note

  None.

- Related data types and interfaces

  MI_VENC_CreateDev

3.2. MI_VENC_DEV_ID_H264_H265_1

- Description

  Defines H264 & h265 coding device 1.

- Definition

  #define MI_VENC_DEV_ID_H264_H265_1 (MI_VENC_DEV_ID_H264_H265_0 + 1)

- Note

  None.

- Related data types and interfaces

  MI_VENC_CreateDev

3.3. MI_VENC_DEV_ID_JPEG_0

- Description

  Defines JPEG coding device 0.

- Definition

  #define MI_VENC_DEV_ID_JPEG_0 (8)

- Note

  None.

- Related data types and interfaces

  MI_VENC_CreateDev

3.4. MI_VENC_DEV_ID_JPEG_1

- Description

  Defines JPEG coding device 1.

- Definition

  #define MI_VENC_DEV_ID_JPEG_1 (MI_VENC_DEV_ID_JPEG_0 + 1)

- Note

  None.

- Related data types and interfaces

  MI_VENC_CreateDev

3.5. RC_TEXTURE_THR_SIZE

- Description

  Defines the number of thresholds for RC macroblock complexity.

- Definition

  ```
  #define  RC_TEXTURE_THR_SIZE  1
  ```

- Note

  None.

- Related data types and interfaces

  None.

3.6. MI_VENC_H264eNaluType_e

- Description

  Define the H.264 code stream NALU type.

- Definition

  ```
  typedef enum
  {
      E_MI_VENC_H264E_NALU_PSLICE =  1,
      E_MI_VENC_H264E_NALU_ISLICE =  5,
      E_MI_VENC_H264E_NALU_SEI    =  6,
      E_MI_VENC_H264E_NALU_SPS    =  7,
      E_MI_VENC_H264E_NALU_PPS    =  8,
      E_MI_VENC_H264E_NALU_IPSLICE = 9,
      E_MI_VENC_H264E_NALU_PREFIX = 14,
      E_MI_VENC_H264E_NALU_MAX
  }MI_VENC_H264eNaluType_e;
  ```

- Members

Member	Description
E_MI_VENC_H264E_NALU_PSLICE	PSLICE type
E_MI_VENC_H264E_NALU_ISLICE	ISLICE type
E_MI_VENC_H264E_NALU_SEI	SEI type
E_MI_VENC_H264E_NALU_SPS	SPS type
E_MI_VENC_H264E_NALU_PPS	PPS type

Member	Description
E_MI_VENC_H264E_NALU_PREFIX	PREFIX type
E_MI_VENC_H264E_NALU_IPSLICE	P frame bruch ISLICE type (not supported at this time)

- Note

  None.

- Related data types and interfaces

  None.

3.7. MI_VENC_H264eRefSliceType_e

- Description

  Defines the reference frame in which the acquired H.264 code stream belongs in the frame skip reference mode.

- Definition

```
typedef enum
{
    E_MI_VENC_H264E_REFSLICE_FOR_1X = 1,
    E_MI_VENC_H264E_REFSLICE_FOR_2X = 2,
    E_MI_VENC_H264E_REFSLICE_FOR_4X,
    E_MI_VENC_H264E_REFSLICE_FOR_MAX = 5
}MI_VENC_H264eRefSliceType_e;
```

- Members

Member Name	Description
E_MI_VENC_H264E_REFSLICE_FOR_1X	Reference frame at 1x frame skip reference.
E_MI_VENC_H264E_REFSLICE_FOR_2X	2x frame skip reference frame or 4x frame skip reference for 2 The reference frame of the double-jump frame reference.
E_MI_VENC_H264E_REFSLICE_FOR_4X	Reference frame for 4x frame skip reference.
E_MI_VENC_H264E_REFSLICE_ MAX	Non-reference frame.

- Note

  None.

- Related data types and interfaces

  None.

3.8. MI_VENC_H264eRefType_e

- Description

Defines the frame type and reference attribute of the H.264 frame skip reference stream.

- Definition

```
typedef enum
{
        E_MI_VENC_BASE_IDR = 0,
        E_MI_VENC_BASE_P_REFTOIDR,
        E_MI_VENC_BASE_P_REFBYBASE,
        E_MI_VENC_BASE_P_REFBYENHANCE,
        E_MI_VENC_ENHANCE_P_REFBYENHANCE,
        E_MI_VENC_ENHANCE_P_NOTFORREF,
        E_MI_VENC_REF_TYPE_MAX
}MI_VENC_H264eRefType_e;
```

- Members

Member Name	Description
E_MI_VENC_BASE_IDR	IDR frame in the base layer.
E_MI_VENC_BASE_P_REFTOIDR	A P frame in the base layer for reference to IDR frame.
E_MI_VENC_BASE_P_REFBYBASE	A P frame in the base layer for reference by other frames in the base layer.
E_MI_VENC_BASE_P_REFBYENHANCE	P frame in the base layer, used for reference of frames in the enhancement layer.
E_MI_VENC_ENHANCE_P_REFBYENHANCE	P frame in the enhance layer, used in the enhance layer The reference of his frame.
E_MI_VENC_ENHANCE_P_NOTFORREF	P frame in the enhance layer, not used for reference

- Note

None.

- Related data types and interfaces

None.

3.9. MI_VENC_JpegePackType_e

- Description

Define the PACK type of the JPEG stream.

- Definition

```
typedef enum
{
    E_MI_VENC_JPEGE_PACK_ECS = 5,
    E_MI_VENC_JPEGE_PACK_APP = 6,
```

```
    E_MI_VENC_JPEGE_PACK_VDO = 7,
    E_MI_VENC_JPEGE_PACK_PIC = 8,
    E_MI_VENC_JPEGE_PACK_MAX
}MI_VENC_JpegePackType_e;
```

- Members

Member	Description
E_MI_VENC_JPEGE_PACK_ECS	ECS type
E_MI_VENC_JPEGE_PACK_APP	APP type
E_MI_VENC_JPEGE_PACK_VDO	VDO type
E_MI_VENC_JPEGE_PACK_PIC	PIC type

- Note

None.

- Related data types and interfaces

None.

3.10. MI_VENC_H265eNaluType_e

- Description

Define the H.265 code stream NALU type.

- Definition

```
typedef enum
{
   E_MI_VENC_H265E_NALU_PSLICE = 1,
   E_MI_VENC_H265E_NALU_ISLICE = 19,
   E_MI_VENC_H265E_NALU_VPS = 32,
   E_MI_VENC_H265E_NALU_SPS = 33,
   E_MI_VENC_H265E_NALU_PPS = 34,
   E_MI_VENC_H265E_NALU_SEI = 39,
   E_MI_VENC_H265E_NALU_MAX
} MI_VENC_H265eNaluType_e;
```

- Members

Member	Description
E_MI_VENC_H265E_NALU_PSLICE	PSLICE type
E_MI_VENC_H265E_NALU_ISLICE	ISLICE type
E_MI_VENC_H265E_NALU_VPS	VPS type
E_MI_VENC_H265E_NALU_SPS	SPS type
E_MI_VENC_H265E_NALU_PPS	PPS type
E_MI_VENC_H265E_NALU_SEI	SEI type

- Note

None.

- Related data types and interfaces

None.

3.11. MI_VENC_Rect_t

- Description

A rectangular box that defines the encoding Description.

- Definition

```
typedef struct MI_VENC_Rect_s
{
    MI_U32 u32Left;
    MI_U32 u32Top;
    MI_U32 u32Width;
    MI_U32 u32Height;
}MI_VENC_Rect_t;
```

- Members

Member Name	Description
u32Left	The distance between the left side of the rectangle and the left side of the actual screen, in pixel.
u32Top	The distance between the top edge of the rectangle and the top edge of the actual screen, in pixel
u32Width	The width of the rectangle in pixel
u32Height	The height of the rectangle in pixel

- Note

The rectangular frame cannot exceed the range of the actual image being encoded.

- Related data types and interfaces

MI_VENC_SetRoiCfg
MI_VENC_SetCrop
MI_VENC_SetRoiBgFrameRate

3.12. MI_VENC_DataType_t

- Description

Define code stream structure type.

- Definition

```
typedef union MI_VENC_DataType_s
{
    MI_VENC_H264eNaluType_e     eH264EType;
```

```
    MI_VENC_JpegePackType_e     eJPEGEType;
    MI_VENC_H265eNaluType_e     eH265EType;
}MI_VENC_DataType_t;
```

- Members

Member	Description
enH264EType	H.264 code stream packet type
enJPEGEType	JPEG code stream packet type
enMPEG4EType	MPEG-4 code stream packet type
enH265EType	H.265 code stream packet type

- Note

None.

- Related data types and interfaces

MI_VENC_H264eNaluType_e
MI_VENC_JpegePackType_e

3.13. MI_VENC_PackInfo_t

- Description

A structure that defines other types of codestream data contained in current stream packet data.

- Definition

```
typedef struct MI_VENC_PackInfo_s
{
    MI_VENC_DataType_t  stPackType;
    MI_U32  u32PackOffset;
    MI_U32  u32PackLength;
    MI_U32  u32SliceId;
}MI_VENC_PackInfo_t;
```

- Members

Member	Description
u32PackType	The current stream packet data contains the types of other stream packets.
u32PackOffset	The current stream packet data contains offsets of other stream packet data.
u32PackLength	The current stream packet data contains the size of other stream packet data.
u32SliceId	The current stream packet data contains the slice id of other stream packet data.

3.14.MI_VENC_Pack_t

- Description

Define the frame code stream packet structure.

- Definition

```
typedef struct MI_VENC_Pack_s
{
    MI_PHY phyAddr;
    MI_U8 *pu8Addr;
    MI_U32 u32Len;
    MI_U64 u64PTS;
    MI_BOOL bFrameEnd;
    MI_VENC_DataType_t stDataType;
    MI_U32 u32Offset;
    MI_U32 u32DataNum;
    MI_U8 u8FrameQP;
    MI_S32 s32PocNum;
    MI_U32 u32Gradient;
    MI_VENC_PackInfo_t asackInfo[8];
} MI_VENC_Pack_t;
```

- Members

Member Name	Description
pu8Addr	The first address of the stream packet.
phyAddr	The code stream packet physical address.
u32Len	The length of the stream packet.
DataType	The stream type supports data packets of the H.264/JPEG/H.265 protocol type.
u64PTS	Timestamp. Unit: us.
bFrameEnd	End of frame identification. Ranges: MI_TRUE: This stream packet is the last packet of the frame. MI_FALSE: The stream packet is not the last packet of the frame.
u32Offset	The offset of the valid data in the stream packet from the first address of the stream packet, pu8Addr.
u32DataNum	The current stream packet (the type of the current packet is specified by the DataType) is included in the data. The number of other types of stream packets.
u8FrameQP	The frame qp of the current code stream packet.
s32PocNum	The poc number of the current stream packet.
u32Gradient	The gradient information of the current code stream packet (used for code rate control).
stPackInfo[8]	The current stream packet data contains other types of stream packet data information.

- Note

  None.

- Related data types and interfaces

  MI_VENC_DataType_t

3.15. MI_VENC_StreamInfoH264_t

- Description

  Define H.264 protocol code stream feature information.

- Definition

```
typedef struct MI_VENC_StreamInfoH264_s
{
  MI_U32  u32PicBytesNum;
  MI_U32  u32PSkipMbNum;
  MI_U32  u32IpcmMbNum;
  MI_U32  u32Inter16x8MbNum;
  MI_U32  u32Inter16x16MbNum;
  MI_U32  u32Inter8x16MbNum;
  MI_U32  u32Inter8x8MbNum;
  MI_U32  u32Intra16MbNum;
  MI_U32  u32Intra8MbNum;
  MI_U32  u32Intra4MbNum;
  MI_VENC_H264eRefSliceType_e  eRefSliceType;
  MI_VENC_H264eRefType_e       eRefType;
  MI_U32  u32UpdateAttrCnt;
  MI_U32  u32StartQp;
}MI_VENC_StreamInfoH264_t;
```

- Members

Member Name	Description
u32PicBytesNum	Encode the number of bytes (BYTE) of the current frame
u32PSkipMbNum	Encoding the number of macroblocks in the current frame using the skip (SKIP) encoding mode
u32IpcmMbNum	Encoding the number of macroblocks in the current frame using the IPCM encoding mode
u32Inter16x8MbNum	Encoding the number of macroblocks in the current frame using the Inter16x8 prediction mode
u32Inter16x16MbNum	Encoding the number of macroblocks in the current frame using the Inter16x16 prediction mode
u32Inter8x16MbNum	Encoding the number of macroblocks in the current frame using the Inter8x16 prediction mode
u32Inter8x8MbNum	Encoding the number of macroblocks in the current frame using the

Member Name	Description
	Inter8x8 prediction mode
u32Intra16MbNum	Encoding the number of macroblocks in the current frame using the Intra16 prediction mode
u32Intra8MbNum	Encoding the number of macroblocks in the current frame using the Intra8 prediction mode
u32Intra4MbNum	Encoding the number of macroblocks in the current frame using the Intra4 prediction mode
enRefSliceType	Coding with the reference frame in the frame skip reference mode of the current frame
enRefType	Coded frame type under advanced frame skip reference
u32UpdateAttrCnt	The number of times the channel attribute or parameter (including the RC parameter) is set
u32StartQp	Initial Qp value used for encoding

- Note
  - When saving the H.264 code stream, only the reference frame in the corresponding frame skip reference mode can be selected:
  - When the frame skip reference mode is the 1x frame skip reference mode, only the code stream whose enRefSliceType is equal to E_MI_VENC_H264E_REFSLICE_FOR_1X can be saved.
  - When the frame skip reference mode is the 2x frame skip reference mode, only the code stream whose enRefSliceType is equal to E_MI_VENC_H264E_REFSLICE_FOR_2X can be saved.
  - When the frame skip reference mode is the 4x frame skip reference mode, only the code stream whose enRefSliceType is equal to _MI_VENC_H264E_REFSLICE_FOR_4X can be saved.
  - Or save the code stream with enRefSliceType equal to E_MI_VENC_H264E_REFSLICE_FOR_2X and E_MI_VENC_H264E_REFSLICE_FOR_4X.

- Related data types and interfaces

  MI_VENC_DataType_t
  MI_VENC_H264eRefSliceType_e

3.16.MI_VENC_StreamInfoJpeg_t

- Description

  Define JPEG protocol code stream feature information.

- Definition

```
typedef struct MI_VENC_StreamInfoJpeg_s
{
  MI_U32  u32PicBytesNum;
  MI_U32  u32UpdateAttrCnt;
  MI_U32  u32Qfactor;
```

}MI_VENC_StreamInfoJpeg_t

- Members

Member Name	Description
u32PicBytesNum	The size of a frame of jpeg code stream, in bytes.
u32Updat eAttrCnt	The number of times the channel attribute or parameter (including the RC parameter) is set.
u32Qfactor	Encodes the Qfactor of the current frame.

- Note

None.

- Related data types and interfaces

MI_VENC_DataType_t

3.17.MI_VENC_StreamInfoH265_t

- Description

Define the H.265 protocol code stream feature information.

- Definition

```
typedef struct MI_VENC_StreamInfoH265_s
{
  MI_U32  u32PicBytesNum;
  MI_U32  u32Inter64x64CuNum;
  MI_U32  u32Inter32x32CuNum;
  MI_U32  u32Inter16x16CuNum;
  MI_U32  u32Inter8x8CuNum;
  MI_U32  u32Intra32x32CuNum;
  MI_U32  u32Intra16x16CuNum;
  MI_U32  u32Intra8x8CuNum;
  MI_U32  u32Intra4x4CuNum;
  MI_VENC_H265eRefType_e eRefType;
  MI_U32  u32UpdateAttrCnt;
  MI_U32  u32StartQp;
}MI_VENC_StreamInfoH265_t;
```

- Members

Member Name	Description
u32PicBytesNum	Encode the number of bytes (BYTE) of the current frame
u32Inter64x64CuNum	Encode the number of CU blocks in the current frame using Inter64x64 prediction mode
u32Inter32x32CuNum	Encoding the number of CU blocks in the current frame using the Inter32x32 prediction mode

Member Name	Description
u32Inter16x16CuNum	Encoding the number of CU blocks in the current frame using Inter16x16 prediction mode
u32Inter8x8CuNum	Encoding the number of CU blocks in the current frame using the Inter8x8 prediction mode
u32Intra32x32CuNum	Encoding the number of CU blocks in the current frame using the Intra32x32 prediction mode
u32Intra16x16CuNum	Encoding the number of CU blocks in the current frame using the Intra16x16 prediction mode
u32Intra8x8CuNum	Encoding the number of CU blocks in the current frame using the Intra8x8 prediction mode
u32Intra4x4CuNum	Encoding the number of CU blocks in the current frame using the Intra4x4 prediction mode
e RefType	Coded frame type under advanced frame skip reference
u32UpdateAttrCnt	The number of times the channel attribute or parameter (including the RC parameter) is set
u32StartQp	Initial Qp value used for encoding

- Note

  See the Description of the enRefType variable on MI_VENC_StreamInfoH264_t.

- Related data types and interfaces

  MI_VENC_DataType_t
  MI_VENC_H264eRefSliceType_e
  MI_VENC_H264eRefType_e

3.18. MI_VENC_Stream_t

- Description

  Define the framestream type structure.

- Definition

```
typedef struct MI_VENC_Stream_s
{
    MI_VENC_Pack_t *pstPack;
    MI_U32 u32PackCount;
    MI_U32 u32Seq;
    MI_SYS_BUF_HANDLE hMiSys;
    union
    {
        MI_VENC_StreamInfoH264_t stH264Info;
        MI_VENC_StreamInfoJpeg_t stJpegInfo;
        MI_VENC_StreamInfoH265_t stH265Info;
    };
```

```
} MI_VENC_Stream_t;
```

- Members

Member Name	Description
pstPack	Frame code stream packet structure.
u32PackCount	The number of all packets of a frame stream.
u32Seq	Code stream serial number.The frame number is obtained by frame; the packet sequence number is obtained by packet.
hMiSys	Buffer handle.
stH624Info/stJpegInfo/ stMpeg4Info/stH265Info	Code stream feature information.

- Note

None.

- Related data types and interfaces

MI_VENC_Pack_t
MI_VENC_GetStream

3.19.MI_VENC_StreamBufInfo_t

- Description

A structure that defines the stream buffer information.

- Definition

```
typedef struct MI_VENC_StreamBufInfo_s
{
  MI_PHY  phyAddr;
  void*     pUserAddr;
  MI_U32        u32BufSize;
}MI_VENC_StreamBufInfo_t;
```

- Members

Member Name	Description
u32PhyAddr	The starting physical address of the code stream buffer.
pUserAddr	The virtual address of the stream buffer.
u32BufSize	The size of the stream buffer.

- Note

None.

- Related data types and interfaces

None.

3.20.MI_VENC_AttrH264_t

- Description

Define the H.264 encoding attribute structure.

- Definition

```
typedef struct MI_VENC_AttrH264_s
{
  MI_U32  u32MaxPicWidth;
  MI_U32  u32MaxPicHeight;
  MI_U32  u32BufSize;
  MI_U32  u32Profile;
  MI_BOOL bByFrame;
  MI_U32  u32PicWidth;
  MI_U32  u32PicHeight;
  MI_U32  u32BFrameNum;
  MI_U32  u32RefNum;
}MI_VENC_AttrH264_t;
```

- Members

Member Name	Description
u32MaxPicWidth	The maximum width of the encoded image. Value range: [MIN_WIDTH, MAX_WIDTH], in pixels. Must be an integer multiple of MIN_ALIGN. Static attribute.
u32PicWidth	Encode image width. Value range: [MIN_WIDTH, u32MaxPicWidth], in pixels. Must be an integer multiple of MIN_ALIGN. Dynamic attribute.
u32MaxPicHeight	The maximum height of the encoded image. Value range: [MIN_HEIGHT, MAX_HEIGHT], in pixels. Must be an integer multiple of MIN_ALIGN. Static attribute.
u32PicHeight	Encode image height. Value range: [MIN_HEIGHT, u32MaxPicHeight], in pixels Bit. Must be an integer multiple of MIN_ALIGN. Dynamic attribute.
u32BufSize	Code stream buffer size. Value range: [Min, Max], in bytes. Recommended value: One maximum encoded image size. The recommended value is u32MaxPicWidth x u32MaxPicHeight x 1.5 byte. Min: 1/2 of the maximum encoded image size. Max: No limit, but it will consume more memory. Static attribute.

Member Name	Description
bByFrame	The frame/packet mode acquires the code stream. Value range: {MI_TRUE, MI_FALSE}. • MI_TRUE: Get by frame. • MI_FALSE: Get by package. Static attribute.
u32Profile	The level of coding. Value range: [0, 2]. 0: Baseline. 1: MainProfile. 2: HighProfile. Dynamic attribute.
u32BFrameNum	Encoding supports the number of B frames. Value range: [0, Max]. Max: Unlimited. Static attribute, reserved field, not supported at this time.
u32RefNum	Encoding supports the number of reference frames. Value range: [1, 2]. Static attribute, reserved field, not supported at this time.

- Note

None.

- Related data types and interfaces

None.

3.21.MI_VENC_AttrJpeg_t

- Description

Define the JPEG capture attribute structure.

- Definition

```
typedef struct MI_VENC_AttrJpeg_s
{
  MI_U32  u32MaxPicWidth;
  MI_U32  u32MaxPicHeight;
  MI_U32  u32BufSize;
  MI_BOOL bByFrame;
  MI_U32  u32PicWidth;
  MI_U32  u32PicHeight;
  MI_BOOL bSupportDCF;
  MI_U32  u32RestartMakerPerRowCnt;
}MI_VENC_AttrJpeg_t;
```

- Members

Member Name	Description
u32MaxPicWidth	The maximum width of the encoded image. Value range: [MIN_WIDTH, MAX_WIDTH], in pixels. Must be an integer multiple of MIN_ALIGN.

Member Name	Description
	Static attribute.
u32PicWidth	Encode image width. Value range: [MIN_WIDTH, u32MaxPicWidth], in pixels. Must be an integer multiple of MI_ALIGN. Dynamic attribute.
u32MaxPicHeight	The maximum height of the encoded image. Value range: [MIN_HEIGHT, MAX_HEIGHT], in pixels. Must be an integer multiple of MIN_ALIGN Static attribute.
u32PicHeigh	Encode image height. Value range: [MIN_HEIGHT, u32MaxPicHeight], in pixels bit. Must be an integer multiple of MIN_ALIGN Dynamic attribute.
u32BufSize	Code stream buffer size. Value range: [Min, Max], in bytes. Recommended value: One maximum encoded image size. The recommended value is u32MaxPicWidth x u32MaxPicHeight x 1.5 byte. Min: 1/2 of the maximum encoded image size. Max: No limit, but it will consume more memory. Static attribute.
bByFrame	Get the stream mode, frame or package. Value range: {MI_TRUE, MI_FALSE}. • MI_TRUE: Get by frame. • MI_FALSE: Get by package. Static attribute.
bSupportDCF	Whether to support Jpeg thumbnails. Static attribute.
u32RestartMakerPerRowCnt	Restart marker after designated rows.

- Note

  None.

- Related data types and interfaces

  None.

3.22. MI_VENC_AttrH265_t

- Description

  Define the H.265 encoding attribute structure.

- Definition

  typedef struct MI_VENC_AttrH265_s

```
{
  MI_U32  u32MaxPicWidth;
  MI_U32  u32MaxPicHeight;
  MI_U32  u32BufSize;
  MI_U32  u32Profile;
  MI_BOOL bByFrame;
  MI_U32  u32PicWidth;
  MI_U32  u32PicHeight;
  MI_U32  u32BFrameNum;
  MI_U32  u32RefNum;
}MI_VENC_AttrH265_t;
```

- Members

Member Name	Description
u32MaxPicWidth	The maximum width of the encoded image. Value range: [MIN_WIDTH, MAX_WIDTH], in pixels. Must be an integer multiple of MIN_ALIGN. Static attribute.
u32PicWidth	Encode image width. Value range: [MIN_WIDTH, u32MaxPicWidth], in pixels. Must be an integer multiple of MIN_ALIGN. Dynamic properties.
u32MaxPicHeight	The maximum height of the encoded image. Value range: [MIN_HEIGHT, MAX_HEIGHT], in pixels. Must be an integer multiple of MIN_ALIGN. Static attribute.
u32PicHeight	Encode image height. Value range: [MIN_HEIGHT, u32MaxPicHeight], in pixels bit. Must be an integer multiple of MIN_ALIGN Dynamic attribute.
u32BufSize	Code stream buffer size. Value range: [Min, Max], in bytes. Recommended value: One maximum encoded image size. The recommended value is u32MaxPicWidth x u32MaxPicHeight x 1.5 byte. Min: 1/2 of the maximum encoded image size. Max: No limit, but it will consume more memory. Static attribute.
bByFrame	The frame/packet mode acquires the code stream. Value range: {MI_TRUE, MI_FALSE}. MI_TRUE: Get by frame. MI_FALSE: Get by package. Static attribute.

Member Name	Description
u32Profile	The level of coding. Value range: 0. 0: MainProfile. Static attribute.
u32BFrameNum	Encoding supports the number of B frames. Value range: [0, Max]. Max: Unlimited. Static attribute, reserved field, not supported at this time.
u32RefNum	Encoding supports the number of reference frames. Value range: [1, 2]. Static attribute, reserved field, not supported at this time.

- Note

  None.

- Related data types and interfaces

  None.

3.23. MI_VENC_Attr_t

- Description

  Define the encoder attribute structure.

- Definition

```
typedef struct MI_VENC_Attr_s
{
   MI_VENC_ModType_e          eType;
   union
   {
      MI_VENC_AttrH264_t    stAttrH264e;
      MI_VENC_AttrJpeg_t         stAttrJpeg;
      MI_VENC_AttrH265_t    stAttrH265e;
   };
}MI_VENC_Attr_t;
```

- Members

Member Name	Description
eType	The encoding protocol type.
stAttrH264e/ stAttrJpeg/stAttrH265e	The encoder property of a protocol.

- Note

  None.

- Related data types and interfaces

  MI_VENC_CreateChn

3.24.MI_VENC_ChnAttr_t

- Description

  Define the encoding channel attribute structure.

- Definition

```
typedef struct MI_VENC_ChnAttr_s
{
  MI_VENC_Attr_t  stVeAttr;
  MI_VENC_RcAttr_t  stRcAttr;
}MI_VENC_ChnAttr_t;
```

- Members

Member Name	Description
stVeAttr	Encoder properties.
stRcAttr	Rate controller properties.

- Note

  None.

- Related data types and interfaces

  MI_VENC_CreateChn

3.25.MI_VENC_ChnStat_t

- Description

  Define the state structure of the code channel.

- Definition

```
typedef struct MI_VENC_ChnStat_s
{
  MI_U32  u32LeftPics;
  MI_U32  u32LeftStreamBytes;
  MI_U32  u32LeftStreamFrames;
  MI_U32  u32LeftStreamMillisec;
  MI_U32  u32CurPacks;
  MI_U32  u32LeftRecvPics;
  MI_U32  u32LeftEncPics;
  MI_U32  u32FrmRateNum;
  MI_U32  u32FrmRateDen;
  MI_U32  u32Bitrate;
}MI_VENC_ChnStat_t;
```

- Members

Member Name	Description
u32LeftPics	The number of images to be encoded.
u32LeftStreamBytes	The number of bytes remaining in the code stream buffer.
u32LeftStreamFrames	The number of frames remaining in the stream buffer.
u32LeftStreamMillisec	The number of time remaining in the stream, in ms
u32CurPacks	The number of streams of the current frame.
u32LeftRecvPics	The number of remaining frames to be received, valid after setting MI_VENC_StartRecvPicEx.
u32LeftEncPics	The number of frames remaining to be encoded, valid after setting MI_VENC_StartRecvPicEx.
u32FrmRateNum	Encoder frame rate molecule, in integer units
u32FrmRateDen	Encoder frame rate denominator, in integer units.
u32Bitrate	Stream bitrate, in kbps.

- Note

  None.

- Related data types and interfaces

  MI_VENC_Query

3.26. MI_VENC_ParamH264SliceSplit_t

- Description

  Define the H.264 protocol encoding channel SLICE partition structure.

- Definition

```
typedef struct MI_VENC_ParamH264SliceSplit_s
{
   MI_BOOL bSplitEnable;
   MI_U32  u32SliceRowCount;
}MI_VENC_ParamH264SliceSplit_t;
```

- Members

Member Name	Description
bSplitEnable	Whether slice split is enabled.
u32SliceRowCount	Represents the number of macroblock lines occupied by each slice. The minimum value is 1: and the maximum value is: (image height +15)/16.

- Note

  None.

- Related data types and interfaces

  MI_VENC_SetH264SliceSplit
  MI_VENC_GetH264SliceSplit

3.27. MI_VENC_ParamH265SliceSplit_t

- Description

  Define the H.265 protocol coding channel SLICE partition structure.

- Definition

```
typedef struct MI_VENC_ParamH265SliceSplit_s
{
   MI_BOOL bSplitEnable;
   MI_U32  u32SliceRowCount;
}MI_VENC_ParamH265SliceSplit_t;
```

- Members

Member Name	Description
bSplitEnable	Whether slice split is enabled.
u32SliceRowCount	Represents the number of macroblock lines occupied by each slice. The minimum value is 1: and the maximum value is: (image height +31)/32.

- Note

  None.

- Related data types and interfaces

  MI_VENC_SetH265SliceSplit
  MI_VENC_GetH265SliceSplit

3.28. MI_VENC_ParamH264InterPred_t

- Description

  Define the H.264 protocol coding channel inter-frame prediction structure.

- Definition

```
typedef struct MI_VENC_ParamH264InterPred_s
{
    /* search window */
    MI_U32 u32HWSize;
    MI_U32 u32VWSize;
    MI_BOOL bInter16x16PredEn;
    MI_BOOL bInter16x8PredEn;
    MI_BOOL bInter8x16PredEn;
    MI_BOOL bInter8x8PredEn;
    MI_BOOL bInter8x4PredEn;
    MI_BOOL bInter4x8PredEn;
    MI_BOOL bInter4x4PredEn;
```

```
    MI_BOOL bExtedgeEn;
} MI_VENC_ParamH264InterPred_t;
```

- Members

Member Name	Description
u32HWSize	Horizontal search window size.
u32VWSize	Vertical search window size.
bInter16x16PredEn	16x16 interframe prediction enable switch, enabled by default.
bInter16x8PredEn	16x8 interframe prediction enable switch, enabled by default.
bInter8x16PredEn	8x16 interframe prediction enable switch, enabled by default.
bInter8x8PredEn	8x8 interframe prediction enable switch, enabled by default.
bInter8x4PredEn	8x4 interframe prediction enable switch, enabled by default.
bInter4x8PredEn	4x8 interframe prediction enable switch, enabled by default.
bInter4x4PredEn	4x4 interframe prediction enable switch, enabled by default.
bExtedgeEn	When the search encounters the boundary of the image, it is beyond the scope of the image, and whether the enable switch for the search is enabled is enabled by default.

- Note

None.

- Related data types and interfaces

MI_VENC_SetH264InterPred
MI_VENC_GetH264InterPred

3.29. MI_VENC_ParamH264IntraPred_t

- Description

Define the H.264 protocol encoding channel intra prediction structure.

- Definition

```
typedef struct MI_VENC_ParamH264IntraPred_s
{
   MI_BOOL bIntra16x16PredEn;
   MI_BOOL bIntraNxNPredEn;
   MI_BOOL bConstrainedIntraPredFlag;
   MI_BOOL bIpcmEn;
   MI_U32  u32Intra16x16Penalty;
   MI_U32  u32Intra4x4Penalty;
   MI_BOOL bIntraPlanarPenalty;
}MI_VENC_ParamH264IntraPred_t;
```

- Members

Member Name	Description
bIntra16x16PredEn	16x16 intra prediction enable, enabled by default. Value range: 0 or 1. 0: not enabled; 1: Enable.
bIntraNxNPredEn	NxN intra prediction is enabled, enabled by default. Value range: 0 or 1. 0: Not enabled; 1: Enable.
bConstrainedIntraPredFlag	The default is 0. Value range: 0 or 1.For details, please refer to the constrained_intra_pred_flag of H.264 protocal.
bIpcmEn	IPCM prediction is enabled, and the default values vary according to different chips. Value range: 0 or 1.
u32Intra16x16Penalty	The default is 0.
u32Intra4x4Penalty	The default is 0.
bIntraPlanarPenalty	The default is 0.

- Note

  None.

- Related data types and interfaces

  MI_VENC_SetH264IntraPred
  MI_VENC_GetH264IntraPred

3.30. MI_VENC_ParamH264Trans_t

- Description

  Define the H.264 protocol encoding channel transform and quantize the structure.

- Definition

```
typedef struct MI_VENC_ParamH264Trans_s
{
   MI_U32  u32IntraTransMode;
   MI_U32  u32InterTransMode;
   MI_S32 s32ChromaQpIndexOffset;
}MI_VENC_ParamH264Trans_t;
```

- Members

Member Name	Description
u32IntraTransMode	Intra prediction conversion mode: This is a reserved interface that is currently not supported
u32InterTransMode	Transformation mode of inter prediction: This is a reserved interface that is currently not supported
s32ChromaQpIndexOffset	For details, see the slice_qp_offset_index of H.264 protocol.

Member Name	Description
	The system default is 0. Value range: [-12, 12].

- Note

  None.

- Related data types and interfaces

  MI_VENC_SetH264Trans
  MI_VENC_GetH264Trans

3.31. MI_VENC_ParamH264Entropy_t

- Description

  Define the H.264 protocol coding channel entropy coding structure.

- Definition

```
typedef struct MI_VENC_ParamH264Entropy_s
{
  MI_U32  u32EntropyEncModeI;
  MI_U32  u32EntropyEncModeP;
}MI_VENC_ParamH264Entropy_t;
```

- Members

Member Name	Description
u32EntropyEncModeI	I frame entropy coding mode. • 0: cavlc • 1: cabac >=2 has no meaning. Baseline does not support cabac.
u32EntropyEncModeP	P frame entropy coding mode. • 0: cavlc • 1: cabac >=2 has no meaning. Baseline does not support cabac.

- Note

  None.

- Related data types and interfaces

  MI_VENC_SetH264Entropy
  MI_VENC_GetH264Entropy

3.32. MI_VENC_ParamH265InterPred_t

- Description

Define the H.265 protocol coding channel inter-frame prediction structure.

- Definition

```
typedef struct MI_VENC_ParamH265InterPred_s
{
/*search window*/
  MI_U32  u32HWSize;
  MI_U32  u32VWSize;
  MI_BOOL bInter16x16PredEn;
  MI_BOOL bInter16x8PredEn;
  MI_BOOL bInter8x16PredEn;
  MI_BOOL bInter8x8PredEn;
  MI_BOOL bInter8x4PredEn;
  MI_BOOL bInter4x8PredEn;
  MI_BOOL bInter4x4PredEn;
  MI_U32  u32Inter32x32Penalty;
  MI_U32  u32Inter16x16Penalty;
  MI_U32  u32Inter8x8Penalty;
  MI_BOOL bExtedgeEn;
}MI_VENC_ParamH265InterPred_t;
```

- Members

Member Name	Description
u32HWSize	Horizontal search window size.
u32VWSize	Vertical search window size.
bInter16x16PredEn	16x16 interframe prediction enable switch, enabled by default.
bInter16x8PredEn	16x8 interframe prediction enable switch, enabled by default.
bInter8x16PredEn	8x16 interframe prediction enable switch, enabled by default.
bInter8x8PredEn	8x8 interframe prediction enable switch, enabled by default.
bInter8x4PredEn	8x4 interframe prediction enable switch, enabled by default.
bInter4x8PredEn	4x8 interframe prediction enable switch, enabled by default.
bInter4x4PredEn	4x4 interframe prediction enable switch, enabled by default.
u32Inter32x32Penalty	32x32 block inter prediction is selected probability, the larger the value, the lower the probability. The default is 0
u32Inter16x16Penalty	16x16 block inter prediction is selected probability, the larger the value, the lower the probability. The default is 0
u32Inter8x8Penalty	8x8 block inter prediction is selected probability, the larger the value, the lower the probability. The default is 0
bExtedgeEn	When the search encounters the boundary of the image, it is beyond the scope of the image, and whether the enable switch for the search is enabled is enabled by default.

- Note

  None.

- Related data types and interfaces

  MI_VENC_SetH265InterPred
  MI_VENC_GetH265InterPred

3.33. MI_VENC_ParamH265IntraPred_t

- Description

  Define the H.265 protocol coding channel intra prediction structure.

- Definition

```
typedef struct MI_VENC_ParamH265IntraPred_s
{
  MI_BOOL bIntra32x32PredEn;
  MI_BOOL bIntra16x16PredEn;
  MI_BOOL bIntra8x8PredEn;
  MI_BOOL bConstrainedIntraPredFlag;
   MI_U32  u32Intra32x32Penalty;
   MI_U32  u32Intra16x16Penalty;
   MI_U32  u32Intra8x8Penalty;
}MI_VENC_ParamH265IntraPred_t;
```

- Members

Member Name	Description
bIntra32x32PredEn	32x32 intra prediction enable, enabled by default. Value range: 0 or 1. 0: not enabled; 1: Enable.
bIntra16x16PredEn	16x16 intra prediction enable, enabled by default. Value range: 0 or 1. 0: not enabled; 1: Enable.
bIntra8x8PredEn	8x8 intra prediction enable, enabled by default. Value range: 0 or 1. 0: not enabled; 1: Enable.
bConstrainedIntraPredFlag	The default is 0. Value range: 0 or 1.For details, please refer to the constrained_intra_pred_flag of H.265 protocol.
u32Intra32x32Penalty	32x32 block intra prediction is selected probability, the larger the value, the lower the probability. The default is 0.
u32Intra16x16Penalty	16 x 16 block intra prediction is selected probability, the larger

Member Name	Description
	the value, the lower the probability The default is 0.
u32Intra 8 x 8 Penalty	8 x 8 block intra prediction is selected probability, the larger the value, the lower the probability The default is 0.

- Note

  None.

- Related data types and interfaces

  MI_VENC_SetH265IntraPred
  MI_VENC_GetH265IntraPred

3.34. MI_VENC_ParamH265Trans_t

- Description

  Define the H.265 protocol coding channel transform and quantize the structure.

- Definition

```
typedef struct MI_VENC_ParamH265Trans_s
{
   MI_U32  u32IntraTransMode;
   MI_U32  u32InterTransMode;
   MI_S32 s32ChromaQpIndexOffset;
}MI_VENC_ParamH265Trans_t;
```

- Members

Member Name	Description
u32IntraTransMode	Intra prediction conversion mode: This is a reserved interface that is currently not supported
u32InterTransMode	Transformation mode of inter prediction: This is a reserved interface that is currently not supported
s32ChromaQpIndexOffset	For details, please refer to the slice_cb_qp_offset and slice_cr_qp_offset of H.265 protocol. The system default is 0. Value range: [-12, 12].

- Note

  None.

- Related data types and interfaces

  MI_VENC_SetH265Trans
  MI_VENC_GetH265Trans

3.35.MI_VENC_ParamH264Dblk_t

- Description

  Define the H.264 protocol encoding channel Dblk structure.

- Definition

```
typedef struct MI_VENC_ParamH264Dblk_s
{
   MI_U32 disable_deblocking_filter_idc;
   MI_S32 slice_alpha_c0_offset_div2;
   MI_S32 slice_beta_offset_div2;
}MI_VENC_ParamH264Dblk_t;
```

- Members

Member Name	Description
Disable_deblocking_filter_idc	The value range is [0, 2], and the default value is 0. For details, see the H.264 protocol.
Slice_alpha_c0_offset_div2	The value range [-6,6], the default value is 0. For details, see the H.264 protocol.
Slice_beta_offset_div2	The value range [-6,6], the default value is 0. For details, see the H.264 protocol.

- Note

  None.

- Related data types and interfaces

  MI_VENC_SetH264Dblk
  MI_VENC_GetH264Dblk

3.36.MI_VENC_ParamH265Dblk_t

- Description

  Define the H.265 protocol encoding channel Dblk structure.

- Definition

```
typedef struct MI_VENC_ParamH265Dblk_s
{
   MI_U32 disable_deblocking_filter_idc;     //special naming for CODEC ISO SPEC.
   MI_S32 slice_tc_offset_div2;     //special naming for CODEC ISO SPEC.
   MI_S32 slice_beta_offset_div2;     //special naming for CODEC ISO SPEC.
} MI_VENC_ParamH265Dblk_t;
```

- Members

Member Name	Description
disable_deblocking_filter_idc	The value range is [0, 2], and the default value is 0. For details, see the slice_deblocking_filter_disabled_flag of

Member Name	Description
	H.265 Protocol.
slice_tc_offset_div2	The value range is [-6,6], and the default value is 0. For details, see H.265 Protocol.
slice_beta_offset_div2	The value range is [-6,6], and the default value is 0. For details, see H.265 Protocol.

- Note

  None.

- Related data types and interfaces

  MI_VENC_SetH265Dblk
  MI_VENC_GetH265Dblk

3.37. MI_VENC_ParamH264Vui_t

- Description

  Define the VUI structure of the H.264 protocol encoding channel.

- Definition

```
typedef struct MI_VENC_ParamH264Vui_s
{
  MI_VENC_ParamH264VuiAspectRatio_t  stVuiAspectRatio;
  MI_VENC_ParamH264VuiTimeInfo_t     stVuiTimeInfo;
  MI_VENC_ParamH264VuiVideoSignal_t      stVuiVdeoSignal;
}MI_VENC_ParamH264Vui_t;
```

- Member

Member Name	Description
stVuiAspectRatio	For details, see the Annex E Video usability information of H.264 protocol.
stVuiTimeInfo	For details, see the Annex E Video usability information of H.264 protocol.
stVuiVideoSignal	For details, see the Annex E Video usability information of H.264 protocol.

- Note

  None.

- Related data types and interfaces

  MI_VENC_SetH264Vui
  MI_VENC_GetH264Vui

3.38. MI_VENC_ParamH264VuiAspectRatio_t

- Description

  A structure that defines the AspectRatio information in the H.264 protocol encoding channel VUI.

- Definition

```
typedef struct MI_VENC_ParamH264VuiAspectRatio_s
{
  MI_U8     u8AspectRatioInfoPresentFlag;
  MI_U8     u8AspectRatioIdc;
  MI_U8     u8OverscanInfoPresentFlag;
  MI_U8     u8OverscanAppropriateFlag;
  MI_U16    u16SarWidth;
  MI_U16    u16SarHeight;
}MI_VENC_ParamH264VuiAspectRatio_t;
```

- Member

Member Name	Description
u8AspectRatioInfoPresentFlag	For details, see the H.264 protocol. The system defaults to 0. Value range: 0 or 1.
u8AspectRatioIdc	For details, see the Annex E Video usability information of H.264 Protocol. The system defaults to 1. Value range: [0, 255], 17~254 reserved.
u8OverscanInfoPresentFlag	For details, see the Annex E Video usability information of H.264 protocol. The system defaults to 0. Value range: 0 or 1.
u8OverscanAppropriateFlag	For details, see the Annex E Video usability information of H.264 protocol. The system defaults to 0. Value range: 0 or 1.
u16SarWidth	For details, see the Annex E Video usability information of H.264 Protocol. The system defaults to 1. The value range is (0,65535) and is relatively prime to u16SarHeight.
u16SarHeight	For details, see the Annex E Video usability information of H.264 Protocol. The system defaults to 1. The value range is (0,65535) and is mutually prime with u16SarWidth.

- Note

  None.

- Related data types and interfaces

  MI_VENC_SetH264Vui

3.39. MI_VENC_ParamH264VuiTimeInfo_t

- Description

  A structure that defines TIME_INFO information in the H.264 protocol encoding channel VUI.

- Definition

```
typedef struct MI_VENC_ParamH264VuiTimeInfo_s
{
  MI_U8       u8TimingInfoPresentFlag;
  MI_U8       u8FixedFrameRateFlag;
  MI_U32          u32NumUnitsInTick;
  MI_U32          u32TimeScale;
}MI_VENC_ParamH264VuiTimeInfo_t;
```

- Member

Member Name	Description
u8TimingInfoPresentFlag	For details, see the Annex E Video usability information of H.264 protocol. The system defaults to 0. Value range: 0 or 1.
u32NumUnitsInTick	For details, see the Annex E Video usability information of H.264 Protocol. The system defaults to 1. Value range: greater than 0.
u32TimeScale	For details, see the Annex E Video usability information of H.264 Protocol. The system defaults to 60. Value range: greater than 0.
u8FixedFrameRateFlag;	For details, see the Annex E Video usability information of H.264 Protocol. The system defaults to 1. Value range: 0 or 1.

- Note

  None.

- Related data types and interfaces

  MI_VENC_SetH264Vui
  MI_VENC_GetH264Vui

3.40. MI_VENC_ParamH264VuiVideoSignal_t

- Description

  A structure that defines VIDEO_SIGNAL information in the H.264 protocol encoding channel VUI.

- Definition

```
typedef struct MI_VENC_ParamVuiVideoSignal_s
{
  MI_U8   u8VideoSignalTypePresentFlag;
  MI_U8   u8VideoFormat;
  MI_U8   u8VideoFullRangeFlag;
  MI_U8   u8ColourDescriptionPresentFlag;
```

```
  MI_U8   u8ColourPrimaries;
  MI_U8   u8TransferCharacteristics;
  MI_U8   u8MatrixCoefficients;
}MI_VENC_ParamH264VuiVideoSignal_t;
```

- Member

Member Name	Description
u8VideoSignalTypePresentFlag	For details, see the Annex E Video usability information of H.264 Protocol. The system defaults to 1. Value range: 0 or 1.
u8VideoFormat	For details, see the Annex E Video usability information of H.264 protocol. The system defaults to 5.Value range: [0,7].
u8VideoFullRangeFlag	For details, see the Annex E Video usability information of H.264 Protocol. The system defaults to 1. Value range: 0 or 1.
u8ColourDescriptionPresentFlag;	For details, see the Annex E Video usability information of H.264 Protocol. The system defaults to 1. Value range: 0 or 1.
u8ColourPrimaries	For details, see the Annex E Video usability information of H.264 Protocol. The system defaults to 1. Value range: [0, 255].
u8TransferCharacteristics	For details, see the Annex E Video usability information of H.264 Protocol. The system defaults to 1. Value range: [0, 255].
u8MatrixCoefficients	For details, see the Annex E Video usability information of H.264 Protocol. The system defaults to 1. Value range: [0, 255].

- Note

None.

- Related data types and interfaces

MI_VENC_SetH264Vui

3.41.MI_VENC_ParamH265Vui_t

- Description

Define the H.265 protocol encoding channel VUI structure.

- Definition

```
typedef struct MI_VENC_ParamH265Vui_s
{
  MI_VENC_ParamH265VuiAspectRatio_t   stVuiAspectRatio;
```

```
    MI_VENC_ParamH265VuiTimeInfo_t      stVuiTimeInfo;
    MI_VENC_ParamH265VuiVideoSignal_t   stVuiVdeoSignal;
}MI_VENC_ParamH265Vui_t;
```

- Member

Member Name	Description
stVuiAspectRatio	For details, please refer to the Annex E Video usability information of H.265 protocol.
stVuiTimeInfo	For details, please refer to the Annex E Video usability information of H.265 protocol.
stVuiVideoSignal	For details, please refer to the Annex E Video usability information of H.265 protocol.

- Note

None.

- Related data types and interfaces

MI_VENC_SetH265Vui
MI_VENC_GetH265Vui

3.42. MI_VENC_ParamH265VuiAspectRatio_t

- Description

A structure that defines the AspectRatio information in the H.265 protocol encoding channel VUI.

- Definition

```
typedef struct MI_VENC_ParamH265VuiAspectRatio_s
{
    MI_U8       u8AspectRatioInfoPresentFlag;
    MI_U8       u8AspectRatioIdc;
    MI_U8       u8OverscanInfoPresentFlag;
    MI_U8       u8OverscanAppropriateFlag;
    MI_U16      u16SarWidth;
    MI_U16      u16SarHeight;
}MI_VENC_ParamH265VuiAspectRatio_t;
```

- Member

Member Name	Description
u8AspectRatioInfoPresentFlag	For details, see the Annex E Video usability information of H.265 Protocol. The system defaults to 0. Value range: 0 or 1.
u8AspectRatioIdc	For details, see the Annex E Video usability information of H.265 Protocol. The system defaults to 1. Value range: [0, 255], 17~254 reserved.

Member Name	Description
u8OverscanInfoPresentFlag	For details, see the Annex E Video usability information of H.265 Protocol. The system defaults to 0. Value range: 0 or 1.
u8OverscanAppropriateFlag	For details, see the Annex E Video usability information of H.265 Protocol. The system defaults to 0. Value range: 0 or 1.
u16SarWidth	For details, see the Annex E Video usability information of H.265 Protocol. The system defaults to 1. The value range is (0, 65535) and is relatively prime to u16SarHeight.
u16SarHeight	For details, see the Annex E Video usability information of H.265 Protocol. The system defaults to 1. The value range is (0, 65535) and is mutually prime with u16SarWidth.

- Note

  None.

- Related data types and interfaces

  MI_VENC_SetH265Vui

3.43. MI_VENC_ParamH265VuiTimeInfo_t

- Description

  TIME_INFO Definition structure information encoding channel VUI H.265 of protocol

- Definition

```
typedef struct MI_VENC_ParamH265VuiTimeInfo_s
{
  MI_U8       u8TimingInfoPresentFlag;
  //MI_U8          u8FixedFrameRateFlag;
  MI_U32         u32NumUnitsInTick;
  MI_U32         u32TimeScale;
}MI_VENC_ParamH265VuiTimeInfo_t;
```

- Member

Member Name	Description
u8TimingInfoPresentFlag	For details, see the Annex E Video usability information of H.265 Protocol. The system defaults to 0. Value range: 0 or 1.
u32NumUnitsInTick	For details, see the Annex E Video usability information of H.265 Protocol. The system defaults to 1. Value range: greater than 0.
u32TimeScale	For details, see the Annex E Video usability information of H.265 Protocol. The system default is 60. Value range: greater than 0.
u8FixedFrameRateFlag;	For details, see the Annex E Video usability information of H.265 Protocol. The system defaults to 1. Value range: 0 or 1.

- Note

  None.

- Related data types and interfaces

  MI_VENC_SetH265Vui
  MI_VENC_GetH265Vui

3.44. MI_VENC_ParamH265VuiVideoSignal_t

- Description

  H.265 protocol defined in VIDEO_SIGNAL VUI channel coding structure information.

- Definition

```
typedef struct MI_VENC_ParamVuiVideoSignal_s
{
   MI_U8   u8VideoSignalTypePresentFlag;
   MI_U8   u8VideoFormat;
   MI_U8   u8VideoFullRangeFlag;
   MI_U8   u8ColourDescriptionPresentFlag;
   MI_U8   u8ColourPrimaries;
   MI_U8   u8TransferCharacteristics;
   MI_U8   u8MatrixCoefficients;
}MI_VENC_ParamH265VuiVideoSignal_t;
```

- Member

Member Name	Description
u8VideoSignalTypePresentFlag	For details, see the Annex E Video usability information of H.265 Protocol. The system defaults to 1. Value range: 0 or 1.
u8VideoFormat	For details, see the Annex E Video usability information of H.265 Protocol. The system defaults to 5. Value range: [0,7].
u8VideoFullRangeFlag	For details, see the Annex E Video usability information of H.265 Protocol. The system defaults to 1. Value range: 0 or 1.
u8ColourDescriptionPresentFlag;	For details, see the Annex E Video usability information of H.265 Protocol. The system defaults to 1. Value range: 0 or 1.
u8ColourPrimaries	For details, see the Annex E Video usability information of H.265 Protocol. The system defaults to 1. Value range: [0, 255].
u8TransferCharacteristics	For details, see the Annex E Video usability information of H.265 Protocol. The system defaults to 1. Value range: [0, 255].

Member Name	Description
u8MatrixCoefficients	For details, see the Annex E Video usability information of H.265 Protocol. The system defaults to 1. Value range: [0, 255].

- Note

  None.

- Related data types and interfaces

  MI_VENC_SetH265Vui

3.45. MI_VENC_ParamJpeg_t

- Description

  Define the JPEG protocol encoding channel advanced parameter structure.

- Definition

```
typedef struct MI_VENC_ParamJpeg_s
{
   MI_U32 u32Qfactor;
   MI_U8 au8YQt[64];
   MI_U8 au8CbCrQt[64];
   MI_U32 u32McuPerEcs;
} MI_VENC_ParamJpeg_t;
```

- Member

Member Name	Description
u32Qfactor	For details, see RFC2435. The system defaults to 70. Value range: [1, 90].
au8YQt	Y quantization table. Value range: [1,255].
au8CbCrQt	Cb Cr quantization table. Value range: [1,255].
u32MCUPerEcs	How many MCUs are included in each ECS, the system defaults to 0, indicating no Divide Ecs. u32MCUPerECS:[0,(picwidth+15)>>4x(picheight+15)>>4x2] Not supported yet.

- Note

  None.

- Related data types and interfaces

  MI_VENC_SetJpegParam
  MI_VENC_GetJpegParam

3.46.MI_VENC_RoiCfg_t

- Description

  Define the information of the region of interest of the code.

- Definition

```
typedef struct MI_VENC_RoiCfg_s
{
   MI_U32  u32Index;
   MI_BOOL bEnable;
   MI_BOOL bAbsQp;
   MI_S32 s32Qp;
   MI_VENC_Rect_t stRect;
}MI_VENC_RoiCfg_t;
```

- Member

Member Name	Description
u32Index	The index of the ROI area, the index range supported by the system is [0,7], not supported. An index beyond this range.
bEnable	Whether to enable this ROI area.
bAbsQp	ROI area QP mode. • MI_FALSE: Relative QP • MI_TURE: Absolute QP
s32Qp	QP value. When QP mode is MI_FALSE, s32Qp is QP offset; when QP mode is MI_TRUE, s32Qp is macroblock QP value.
stRect	ROI area.

- Note

  The QP mode of the ROI area could only support the relative QP (bAbsQp = MI_FALSE) from Pudding, Ispahan and Tiramisu. As a result, the value range of s32Qp is modified to [-32, 31]. See MI_VENC_SetRoiCfg for details.

- Related data types and interfaces

  MI_VENC_SetRoiCfg
  MI_VENC_GetRoiCfg

3.47.MI_VENC_RoiBgFrameRate_t

- Description

  Define the frame rate of the non-coded region of interest.

- Definition

```
typedef struct MI_VENC_RoiBgFrameRate_s
{
   MI_S32 s32SrcFrmRate;
```

```
  MI_S32 s32DstFrmRate;
}MI_VENC_RoiBgFrameRate_t;
```

- Member

Member Name	Description
s32SrcFrmRate	Source frame rate for non-ROI regions.
s32DstFrmRate	The target frame rate of the non-ROI area.

- Note

None.

- Related data types and interfaces

MI_VENC_SetRoiBgFrameRate
MI_VENC_GetRoiBgFrameRate

3.48.MI_VENC_ParamRef_t

- Description

Define advanced frame skipping reference parameters for H.264/H.265 encoding.

- Definition

```
typedef struct MI_VENC_ParamRef_s
{
  MI_U32  u32Base;
  MI_U32  u32Enhance;
  MI_BOOL bEnablePred;
}MI_VENC_ParamRef_t;
```

- Member

Member Name	Description
u32Base	The period of the base layer.Value range: (0, +∞).
u32Enhance	The period of the enhance layer.Value range: [0, 255].
bEnablePred	Whether the frame representing the base layer is used as a reference by other frames in the base layer. When When MI_FALSE, all frames of the base layer refer to the IDR frame.

- Note

None.

- Related data types and interfaces

MI_VENC_GetRefParam
MI_VENC_SetRefParam

3.49. MI_VENC_RcAttr_t

- Description

Define the encoding channel rate controller attribute.

- Definition

```
typedef struct MI_VENC_RcAttr_s
{
    MI_VENC_RcMode_e       eRcMode;
    union
    {
     MI_VENC_AttrH264Cbr_t          stAttrH264Cbr;
     MI_VENC_AttrH264Vbr_t          stAttrH264Vbr;
     MI_VENC_AttrH264FixQp_t   stAttrH264FixQp;
     MI_VENC_AttrH264Abr_t          stAttrH264Abr;
     MI_VENC_AttrH264Avbr_t stAttrH264Avbr;
     MI_VENC_AttrMjpegCbr_t     stAttrMjpegCbr;
     MI_VENC_AttrMjpegFixQp_t  stAttrMjpegFixQp;
     MI_VENC_AttrH265Cbr_t          stAttrH265Cbr;
     MI_VENC_AttrH265Vbr_t          stAttrH265Vbr;
     MI_VENC_AttrH265FixQp_t   stAttrH265FixQp;
     MI_VENC_AttrH265Avbr_t stAttrH265Avbr;
    };
    MI_VOID* pRcAttr;
}MI_VENC_RcAttr_t;
```

- Member

Member Name	Description
enRcMode	RC mode.
stAttrH264Cbr	H.264 protocol encoding channel Cbr mode attribute.
stAttrH264Vbr	H.264 protocol encoding channel Vbr mode attribute.
stAttrH264FixQp	H.264 protocol encoding channel Fixqp mode attribute.
stAttrH264Abr	H.264 protocol encoding channel Abr mode attribute (not supported at this time).
stAttrH264Avbr	H.264 protocol encoding channel Avbr mode attribute.
stAttrMjpegCbr	MJPEG protocol encoding channel Cbr mode attribute.
stAttrMjpegFixQp	MJPEG protocol encoding channel FixQp mode attribute.
stAttrH265Cbr	H.265 protocol encoding channel Cbr mode attribute.
stAttrH265Vbr	H.265 protocol encoding channel Vbr mode attribute.
stAttrH265FixQp	H.265 protocol encoding channel Fixqp mode attribute.
stAttrH265Avbr	H.265 protocol encoding channel Avbr mode attribute.

- Note

None.

- Related data types and interfaces
  MI_VENC_CreateChn

3.50. MI_VENC_RcMode_e

- Description
  Define the code channel rate controller mode.

- Definition

```
typedef enum
{
    E_MI_VENC_RC_MODE_H264CBR = 1,
    E_MI_VENC_RC_MODE_H264VBR,
    E_MI_VENC_RC_MODE_H264ABR,
    E_MI_VENC_RC_MODE_H264FIXQP,
    E_MI_VENC_RC_MODE_H264AVBR,
    E_MI_VENC_RC_MODE_MJPEGCBR,
    E_MI_VENC_RC_MODE_H265CBR,
    E_MI_VENC_RC_MODE_H265VBR,
    E_MI_VENC_RC_MODE_H265FIXQP,
    E_MI_VENC_RC_MODE_H265AVBR,
    E_MI_VENC_RC_MODE_MAX,
}MI_VENC_RcMode_e;
```

- Member

Member Name	Description
E_MI_VENC_RC_MODE_H264CBR	H.264 CBR mode.
E_MI_VENC_RC_MODE_H264VBR	H.264 VBR mode.
E_MI_VENC_RC_MODE_H264ABR	H.264 ABR mode (not available at the moment)
E_MI_VENC_RC_MODE_H264FIXQP	H.264 FixQp mode.
E_MI_VENC_RC_MODE_H264AVBR	H.264 AVBR mode
E_MI_VENC_RC_MODE_MJPEGCBR	MJPEG CBR mode
E_MI_VENC_RC_MODE_H265CBR	H.265 CBR mode.
E_MI_VENC_RC_MODE_H265VBR	H.265 VBR mode.
E_MI_VENC_RC_MODE_H265FIXQP	H.265 FixQp mode
E_MI_VENC_RC_MODE_H265AVBR	H.265 AVBR mode

- Note
  None.

- Related data types and interfaces
  MI_VENC_CreateChn

3.51.MI_VENC_AttrH264Cbr_t

- Description

  Define the CBR attribute structure of the H.264 encoding channel.

- Definition

```
typedef struct MI_VENC_AttrH264Cbr_s
{
   MI_U32 u32Gop;
   MI_U32 u32StatTime;
   MI_U32 u32SrcFrmRateNum;
   MI_U32 u32SrcFrmRateDen;
   MI_U32      u32BitRate;
   MI_U32      u32FluctuateLevel;
}MI_VENC_AttrH264Cbr_t;
```

- Member

Member Name	Description
u32Gop	H.264 gop value. Value range: [1, 65536].
u32StatTime	CBR code rate statistics time in seconds. Value range: [1, 60]. Not supported at the moment.
u32SrcFrmRateNum	The encoder frame rate molecule, in integer units.
u32SrcFrmRateDen	Encoder frame rate denominator, in integer units.
u32BitRate	Average bitrate in bps. Value range: [2000, 102400000].
u32FluctuateLevel	The maximum code rate relative to the average bit rate fluctuation level. A volatility level of 0 is recommended. Not supported at the moment.

- Note

  Src F rmRate should be set to the actual frame rate of the input encoder, and RC needs to be rate controlled according to SrcFrmRate.u32SrcFrmRateNum / u32SrcFrmRateDen range between (0, 65535].

- Related data types and interfaces

  MI_VENC_CreateChn

3.52.MI_VENC_AttrH264Vbr_t

- Description

  Define the VBR attribute structure of the H.264 encoding channel.

- Definition

```
typedef struct MI_VENC_AttrH264Vbr_s
```

```
{
  MI_U32  u32Gop;
  MI_U32  u32StatTime;
  MI_U32  u32SrcFrmRateNum;
  MI_U32  u32SrcFrmRateDen;
  MI_U32  u32MaxBitRate;
  MI_U32  u32MaxQp;
  MI_U32  u32MinQp;
}MI_VENC_AttrH264Vbr_t;
```

- Member

Member Name	Description
u32Gop	H.264 gop value. Value range: [1, 65535].
u32StatTime	VBR code rate statistics time in seconds. Value range: [1, 60]. Not supported at the moment.
u32SrcFrmRateNum	Encoder frame rate numerator, in integer units.
u32SrcFrmRateDen	Encoder denominator framerate, in units of integers.
u32MaxBitRate	The encoder outputs the maximum code rate in bps. Value range: [2000,102400000].
u32MaxQp	The encoder supports image maximum QP. Value range: (u32MinQp, 48].
u32MinQp	The encoder supports image minimum QP. Value range: [12, u32MaxQp).

- Note

  See MI_VENC_AttrH264Cbr_t for a Description of u32SrcFrmRateNum and u32SrcFrmRateDen.

- Related data types and interfaces

  MI_VENC_CreateChn

3.53. MI_VENC_AttrH264FixQp_t

- Description

  Define the Fixqp attribute structure of the H.264 encoding channel.

- Definition

```
typedef struct MI_VENC_AttrH264FixQp_s
{
  MI_U32  u32Gop;
  MI_U32  u32SrcFrmRateNum;
  MI_U32  u32SrcFrmRateDen;
  MI_U32  u32IQp;
```

```
    MI_U32  u32PQp;
}MI_VENC_AttrH264FixQp_t;
```

- Member

Member Name	Description
u32Gop	H.264 gop value. Value range: [1, 65535].
u32SrcFrmRateNum	Encoder frame rate numerator, in integer units.
u32SrcFrmRateDen	Encoder denominator framerate, in units of integers.
u32IQp	I frame all macroblock Qp values. Value range: [12, 48].
u32PQp	P frame all macroblock Qp values. Value range: [12, 48].

- Note

See MI_VENC_AttrH264Cbr_t for a Description of u32SrcFrmRateNum and u32SrcFrmRateDen.

- Related data types and interfaces

MI_VENC_CreateChn

3.54. MI_VENC_AttrH264Abr_t

- Description

Define the ABR attribute structure of the H.264 encoding channel.

- Definition

```
typedef struct MI_VENC_AttrH264Abr_s
{
    MI_U32      u32Gop;
    MI_U32      u32StatTime;
    MI_U32      u32SrcFrmRateNum;
    MI_U32    u32SrcFrmRateDen;
    MI_U32      u32AvgBitRate;
    MI_U32      u32MaxBitRate;
}MI_VENC_AttrH264Abr_t;
```

- Member

Member Name	Description
u32Gop	H.264 gop value. Value range: [1, 65535]
u32StatTime	ABR code rate statistics time in seconds. Value range: [1, 60]. Not supported at the moment.
u32SrcFrmRateNum	Encoder frame rate numerator, in integer units.
u32SrcFrmRateDen	Encoder denominator framerate, in units of integers.
u32AvgBitRate	Average code rate in bps. Value range [2000, u32MaxBitRate)

Member Name	Description
u32MaxBitRate	Maximum code rate. Value range [2000,102400000]

- Note

  See MI_VENC_AttrH264Cbr_t for a Description of u32SrcFrmRateNum and u32SrcFrmRateDen.

- Related data types and interfaces

  MI_VENC_CreateChn

3.55. MI_VENC_AttrH264Avbr_t

- Description

  Define the AVBR attribute structure of the H.264 encoding channel.

- Definition

```
typedef struct MI_VENC_AttrH264Avbr_s
{
    MI_U32 u32Gop;
    MI_U32 u32StatTime;
    MI_U32 u32SrcFrmRateNum;
    MI_U32 u32SrcFrmRateDen;
    MI_U32 u32MaxBitRate;
    MI_U32 u32MaxQp;
    MI_U32 u32MinQp;
} MI_VENC_AttrH264Avbr_t;
```

- Member

Member Name	Description
u32Gop	gop value. Value range: [1, 65535].
u32StatTime	Code rate statistics time in seconds. Value range: [1, 60].
u32SrcFrmRateNum	Encoder frame rate numerator, in integer units.
u32SrcFrmRateDen	Encoder denominator framerate, in units of integers.
u32MaxBitRate	The encoder outputs the maximum code rate in bps. Value range: [2000, 102400000].
u32MaxQp	The encoder supports image maximum QP. Value range: (u32MinQp, 48].
u32MinQp	The encoder supports image minimum QP. Value range: [12, u32MaxQp).

- Note

  See MI_VENC_AttrH264Cbr_t for a Description of u32SrcFrmRateNum and u32SrcFrmRateDen.

- Related data types and interfaces
  MI_VENC_CreateChn

3.56. MI_VENC_AttrMjpegCbr_t

- Description
  Define the CBR attribute structure of the MJPEG encoding channel.

- Definition

```
typedef struct MI_VENC_AttrMjpegCbr_s
{
   MI_U32       u32BitRate;
   MI_U32       u32SrcFrmRateNum;
   MI_U32 u32SrcFrmRateDen;
} MI_VENC_AttrMjpegCbr_t;
```

- Member

Member Name	Description
u32BitRate	Stream bit rate in bps. Value range: [2000,102400000]
u32SrcFrmRateNum	Encoder frame rate numerator, in integer units.
u32SrcFrmRateDen	Encoder denominator framerate, in units of integers.

- Note
  See MI_VENC_AttrH264Cbr_t for a Description of u32SrcFrmRateNum and u32SrcFrmRateDen.

- Related data types and interfaces
  MI_VENC_CreateChn

3.57. MI_VENC_AttrMjpegFixQp_t

- Description
  Define the CBR attribute structure of the MJPEG encoding channel.

- Definition

```
typedef struct MI_VENC_AttrMjpegFixQp_s
{
   MI_U32 u32SrcFrmRateNum;
   MI_U32 u32SrcFrmRateDen;
   MI_U32 u32Qfactor;
} MI_VENC_AttrMjpegFixQp_t;
```

- Member

Member Name	Description
u32Qfactor	Qfactor. Value range: [1,90]

u32SrcFrmRateNum	Encoder frame rate numerator, in integer units.
u32SrcFrmRateDen	Encoder denominator framerate, in units of integers.

- Note

  See MI_VENC_AttrH264Cbr_t for a Description of u32SrcFrmRateNum and u32SrcFrmRateDen.

- Related data types and interfaces

  MI_VENC_CreateChn

3.58. MI_VENC_AttrH265Cbr_t

- Description

  Define the HBR 265 code channel CBR attribute structure.

- Definition

```
typedef struct MI_VENC_AttrH265Cbr_s
{
  MI_U32      u32Gop;
  MI_U32      u32StatTime;
  MI_U32      u32SrcFrmRateNum;
  MI_U32 u32SrcFrmRateDen;
  MI_U32      u32BitRate;
  MI_U32      u32FluctuateLevel;
}MI_VENC_AttrH265Cbr_t;
```

- Member

Member Name	Description
u32Gop	H.265 gop value. Value range: [1, 65535].
u32StatTime	CBR code rate statistics time in seconds. Value range: [1, 60].
u32SrcFrmRateNum	Encoder frame rate numerator, in integer units.
u32SrcFrmRateDen	Encoder denominator framerate, in units of integers.
u32BitRate	Average bitrate in bps. Value range: [2000, 102400000].
u32FluctuateLevel	The maximum code rate relative to the average bit rate fluctuation level. A volatility level of 0 is recommended. Not supported at the moment.

- Note

  See MI_VENC_AttrH264Cbr_t for a Description of u32SrcFrmRateNum and u32SrcFrmRateDen.

- Related data types and interfaces

  MI_VENC_CreateChn

3.59.MI_VENC_AttrH265Vbr_t

- Description

  Define the HBR attribute channel VBR attribute structure.

- Definition

```
typedef struct MI_VENC_AttrH265Vbr_s
{
   MI_U32      u32Gop;
   MI_U32      u32StatTime;
   MI_U32      u32SrcFrmRateNum;
   MI_U32 u32SrcFrmRateDen;
   MI_U32      u32MaxBitRate;
   MI_U32      u32MaxQp;
   MI_U32      u32MinQp;
}MI_VENC_AttrH265Vbr_t;
```

- Member

Member Name	Description
u32Gop	H.265gop value. Value range: [1,65535]
u32StatTime	VBR code rate statistics time in seconds. Value range: [1, 60].
u32SrcFrmRateNum	Encoder frame rate numerator, in integer units.
u32SrcFrmRateDen	Encoder denominator frame rate, in units of integers.
u32MaxBitRate	The encoder outputs the maximum code rate in bps. Value range: [2000, 102400000].
u32MaxQp	The encoder supports image maximum QP. Value range: (u32MinQp, 48].
u32MinQp	The encoder supports image minimum QP. Value range: [12, u32MaxQp).

- Note

  See MI_VENC_AttrH264Cbr_t for a Description of u32SrcFrmRateNum and u32SrcFrmRateDen.

- Related data types and interfaces

  MI_VENC_CreateChn

3.60.MI_VENC_AttrH265FixQp_t

- Description

  Define the H.265 encoding channel Fixqp attribute structure.

- Definition

```
typedef struct MI_VENC_AttrH265FixQp_s
{
  MI_U32      u32Gop;
  MI_U32      u32SrcFrmRateNum;
  MI_U32 u32SrcFrmRateDen;
  MI_U32      u32IQp;
  MI_U32      u32PQp;
}MI_VENC_AttrH265FixQp_t;
```

- Member

Member Name	Description
u32Gop	H.265 gop value. Value range: [1, 65535].
u32SrcFrmRateNum	Encoder frame rate numerator, in integer units.
u32SrcFrmRateDen	Encoder frame rate denominator, in integer units.
u32IQp	I frame all macroblock Qp values. Value range: [12, 48].
u32PQp	P frame all macroblock Qp values. Value range: [12, 48].

- Note

See MI_VENC_AttrH264Cbr_t for a Description of u32SrcFrmRateNum and u32SrcFrmRateDen.

- Related data types and interfaces

MI_VENC_CreateChn

3.61. MI_VENC_AttrH265Avbr_t

- Description

Define the AVBR attribute structure of the H.265 encoding channel.

- Definition

```
typedef struct MI_VENC_AttrH265Avbr_s
{
   MI_U32 u32Gop;
   MI_U32 u32StatTime;
   MI_U32 u32SrcFrmRateNum;
   MI_U32 u32SrcFrmRateDen;
   MI_U32 u32MaxBitRate;
   MI_U32 u32MaxQp;
   MI_U32 u32MinQp;
} MI_VENC_AttrH265Avbr_t;
```

- Member

Member Name	Description
u32Gop	gop value. Value range: [1, 65536].
u32StatTime	Code rate statistics time in seconds. Value range: [1, 60]. Not supported at the moment.
u32SrcFrmRateNum	Encoder frame rate numerator, in integer units.
u32SrcFrmRateDen	Encoder denominator framerate, in units of integers.
u32MaxBitRate	The encoder outputs the maximum code rate in bps. Value range: [2000, 102400000].
u32MaxQp	The encoder supports image maximum QP. Value range: (u32MinQp, 48].
u32MinQp	The encoder supports image minimum QP. Value range: [12, u32MaxQp).

- Note

  See MI_VENC_AttrH264Cbr_t for a Description of u32SrcFrmRateNum and u32SrcFrmRateDen.

- Related data types and interfaces

  MI_VENC_CreateChn

3.62. MI_VENC_SuperFrmMode_e

- Description

  Define the superframe processing mode in rate control.

- Definition

```
typedef enum
{
  E_MI_VENC_SUPERFRM_NONE,
  E_MI_VENC_SUPERFRM_DISCARD,
  E_MI_VENC_SUPERFRM_REENCODE,
  E_MI_VENC_SUPERFRM_MAX
}MI_VENC_SuperFrmMode_e;
```

- Member

Member Name	Description
E_MI_VENC_SUPERFRM_NONE	No special strategy.
E_MI_VENC_SUPERFRM_DISCARD	Discard jumbo frames.
E_MI_VENC_SUPERFRM_REENCODE	Reprogram oversized frames.

- Note

  None.

- Related data types and interfaces

MI_VENC_GetSuperFrameCfg
MI_VENC_SetSuperFrameCfg

3.63. MI_VENC_ParamH264Vbr_t

- Description

Define the H264 protocol encoding channel VBR rate control mode advanced parameter configuration.

- Definition

```
typedef struct MI_VENC_ParamH264Vbr_s
{
   MI_S32 s32IPQPDelta;
   MI_S32 s32ChangePos;
   MI_U32       u32MaxIQp;
   MI_U32     u32MinIQP;
   MI_U32       u32MaxIPProp;
   MI_U32    u32MaxISize;
   MI_U32    u32MaxPSize;
}MI_VENC_ParamH264Vbr_t;
```

- Member

Member Name	Description
s32IPQPDelta	IPQP change value. Value range: [-12, 12].
s32ChangePos	The ratio of the code rate at the time when VBR starts to adjust Qp with respect to the maximum code rate. Value range: [50,100].
u32MaxIQp	The maximum QP of the I frame. The minimum number of bits used to control the I frame. Value range: (MinQp, MaxQp].
u32MinIQP	The minimum QP of the I frame. The maximum number of bits used to control the I frame. Value range: [MinQp, MaxQp).
u32MaxIPProp	Maximum ratio of I/P frame size, range of value [5,100].
u32MaxISize	The maximum size of the I frame. Rate control will try to let the target size not exceed the max size. If value is 0, it means that rate control will not consider the parameter. Value range:[0, 0xFFFFFFFF] default value:0
u32MaxPSize	The maximum size of the P frame. Rate control will try to let the target size not exceed the max size. If value is 0, it means that rate control will not consider the parameter. Value range:[0, 0xFFFFFFFF] default value:0

- Note

  None.

- Related data types and interfaces

  MI_VENC_GetRcParam
  MI_VENC_SetRcParam

3.64. MI_VENC_ParamH264Cbr_t

- Description

  Define the H264 protocol encoding channel CBR new version rate control mode advanced parameter configuration.

- Definition

```
typedef struct MI_VENC_ParamH264Cbr_s
{
  MI_U32 u32MaxQp;
  MI_U32 u32MinQp;
  MI_S32 s32IPQPDelta;
  MI_U32 u32MaxIQp;
  MI_U32 u32MinIQp;
  MI_U32 u32MaxIPProp;
  MI_U32  u32MaxISize;
  MI_U32  u32MaxPSize
}MI_VENC_ParamH264Cbr_t;
```

- Member

Member Name	Description
u32MaxQp	Frame maximum QP for clamp quality. Value range: (u32MinQp, 48].
u32MinQp	Frame minimum QP for clamp rate fluctuations. Value range: [12, u32MaxQp).
s32IPQPDelta	IPQP change value. Value range: [-12,12].
u32MaxIQp	The maximum QP of the I frame. The minimum number of bits used to control the I frame. Value range: [u32MinIQp, 48].
u32MinIQp	The minimum QP of the I frame. The maximum number of bits used to control the I frame. Value range: [12, u32MaxIQp).
u32MaxIPProp	Maximum ratio of I/P frame size, Value range: [5,100].
u32MaxISize	The maximum size of the I frame. Rate control will try to let the target size not exceed the max size. If value is 0, it means that rate control will not consider the parameter.

Member Name	Description
	Value range:[0, 0xFFFFFFFF] default value:0
u32MaxPSize	The maximum size of the P frame. Rate control will try to let the target size not exceed the max size. If value is 0, it means that rate control will not consider the parameter. Value range:[0, 0xFFFFFFFF] default value:0

- Note

None.

- Related data types and interfaces

MI_VENC_GetRcParam
MI_VENC_SetRcParam

3.65.MI_VENC_ParamH264Avbr_t

- Description

Define the H264 protocol encoding channel AVBR rate control mode advanced parameter configuration.

- Definition

```
typedef struct MI_VENC_ParamH264Avbr_s
{
    MI_S32 s32IPQPDelta;
    MI_S32 s32ChangePos;
    MI_U32 u32MinIQp;
    MI_U32 u32MaxIPProp;
    MI_U32 u32MaxIQp;
    MI_U32 u32MaxISize;
    MI_U32 u32MaxPSize;
    MI_U32 u32MinStillPercent;
    MI_U32 u32MaxStillQp;
    MI_U32 u32MotionSensitivity;
} MI_VENC_ParamH264Avbr_t;
```

- Member

Member Name	Description
s32IPQPDelta	IPQP change value. Value range: [-12, 12]. default value:0
s32ChangePos	The ratio of the code rate at the time when VBR starts to adjust Qp with respect to the maximum code rate. Value range: [50,100]. default value:80

Member Name	Description
u32MinIQP	The minimum QP of the I frame. The maximum number of bits used to control the I frame. Value range: [MinQp, MaxQp). default value:12
u32MaxIPProp	Maximum ratio of I/P frame size, Value range: [5,100].
u32MaxIQp	The maximum QP of the I frame. The minimum number of bits used to control the I frame. Value range: (MinQp, MaxQp]. default value:48
u32MaxISize	The maximum size of the I frame. Rate control will try to let the target size not exceed the max size. If value is 0, it means that rate control will not consider the parameter. Value range:[0, 0xFFFFFFFF] default value:0
u32MaxPSize	The maximum size of the P frame. Rate control will try to let the target size not exceed the max size. If value is 0, it means that rate control will not consider the parameter. Value range:[0, 0xFFFFFFFF] default value:0
u32MinStillPercent	The minimum percent of target bitrate in still state. If the value set to 100,then AVBR will not actively decrease target rate in still state, so the behavior of AVBR will be same with VBR. Value range:[5,100]
u32MaxStillQp	The maximum QP of the I frame in still state. Value range:[MinIQp, MaxIQp] Not supported yet.
u32MotionSensitivity	The sensitivity of adjusting bitrate depending on the motion of picture. Value range: [0,100]. Default value: 100.

- Note

  None.

- Related data types and interfaces

  MI_VENC_GetRcParam
  MI_VENC_SetRcParam

3.66. MI_VENC_ParamMjpegCbr_t

- Description

Define the MJPEG protocol encoding channel CBR new version rate control mode advanced parameter configuration.

- Definition

```
typedef struct MI_VENC_ParamMjpegCbr_s
{
  MI_U32 u32MaxQfactor;
  MI_U32 u32MinQfactor;
} MI_VENC_ParamMjpegCbr_t;
```

- Member

Member Name	Description
u32MaxQfactor	Maximum Qfactor for clamp quality. Value range: (u32MinQfactor, 90].
u32MinQfactor	Minimum Qfactor for clamp quality. Value range: [1, u32MaxQfactor).

- Note

None.

- Related data types and interfaces

MI_VENC_GetRcParam
MI_VENC_SetRcParam

3.67.MI_VENC_ParamH265Vbr_t

- Description

Define the advanced parameter configuration of the VBR rate control mode of the H265 protocol code channel.

- Definition

```
typedef struct MI_VENC_ParamH265Vbr_s
{
  MI_S32 s32IPQPDelta;
  MI_S32 s32ChangePos;
  MI_U32      u32MaxIQp;
  MI_U32    u32MinIQP;
  MI_U32      u32MaxIPProp;
  MI_U32  u32MaxISize;
  MI_U32  u32MaxPSize;
}MI_VENC_ParamH265Vbr_t;
```

- Member

Member Name	Description
s32IPQPDelta	IPQP change value. Value range: [-12, 12].

Member Name	Description
s32ChangePos	The ratio of the code rate at the time when VBR starts to adjust Qp with respect to the maximum code rate. Value range: [50,100].
u32MaxIQp	The maximum QP of the I frame. The minimum number of bits used to control the I frame. Value range: (MinQp, MaxQp].
u32MinIQP	The minimum QP of the I frame. The maximum number of bits used to control the I frame. Value range: [MinQp, MaxQp).
u32MaxIPProp	Maximum ratio of I/P frame size, Value range: [5,100].
u32MaxISize	The maximum size of the I frame. Rate control will try to let the target size not exceed the max size. If value is 0, it means that rate control will not consider the parameter. Value range:[0, 0xFFFFFFFF] default value:0
u32MaxPSize	The maximum size of the P frame. Rate control will try to let the target size not exceed the max size. If value is 0, it means that rate control will not consider the parameter. Value range:[0, 0xFFFFFFFF] default value:0

- Note

  None.

- Related data types and interfaces

  MI_VENC_GetRcParam
  MI_VENC_SetRcParam

3.68. MI_VENC_ParamH265Cbr_t

- Description

  Define the H265 protocol encoding channel CBR new version rate control mode advanced parameter configuration.

- Definition

```
typedef struct MI_VENC_ParamH265Cbr_s
{
  MI_U32  u32MaxQp;
  MI_U32  u32MinQp;
  MI_S32  s32IPQPDelta;
  MI_U32  u32MaxIQp;
  MI_U32  u32MinIQp;
```

```
    MI_U32  u32MaxIPProp;
    MI_U32  u32MaxISize;
    MI_U32  u32MaxPSize;
}MI_VENC_ParamH265Cbr_t;
```

- Member

Member Name	Description
u32MaxQp	Frame maximum QP for clamp quality. Value range: (u32MinQp, 48].
u32MinQp	Frame minimum QP for clamp rate fluctuations. Value range: [12, u32MaxQp).
s32IPQPDelta	IPQP change value. Value range: [-12, 12].
u32MaxIQp	The maximum QP of the I frame. The minimum number of bits used to control the I frame. Value range: [u32MinIQp, u32MaxQp).
u32MinIQp	The minimum QP of the I frame. The maximum number of bits used to control the I frame. Value range: [u32MinQp, u32MaxIQp).
u32MaxIPProp	Maximum ratio of I/P frame size, Value range: [5,100].
u32MaxISize	The maximum size of the I frame. Rate control will try to let the target size not exceed the max size. If value is 0, it means that rate control will not consider the parameter. Value range:[0, 0xFFFFFFFF] default value:0
u32MaxPSize	The maximum size of the P frame. Rate control will try to let the target size not exceed the max size. If value is 0, it means that rate control will not consider the parameter. Value range:[0, 0xFFFFFFFF] default value:0

- Note

None.

- Related data types and interfaces

MI_VENC_GetRcParam
MI_VENC_SetRcParam

3.69.MI_VENC_ParamH265Avbr_t

- Description

Define the H265 protocol encoding channel AVBR rate control mode advanced parameter configuration.

- Definition

```
typedef struct MI_VENC_ParamH265Avbr_s
{
    MI_S32 s32IPQPDelta;
    MI_S32 s32ChangePos;
    MI_U32 u32MinIQp;
    MI_U32 u32MaxIPProp;
    MI_U32 u32MaxIQp;
    MI_U32 u32MaxISize;
    MI_U32 u32MaxPSize;
    MI_U32 u32MinStillPercent;
    MI_U32 u32MaxStillQp;
    MI_U32 u32MotionSensitivity;
} MI_VENC_ParamH265Avbr_t;
```

- Member

Member Name	Description
s32IPQPDelta	IPQP change value. Value range: [-12, 12]. default value:0
s32ChangePos	The ratio of the code rate at the time when VBR starts to adjust Qp with respect to the maximum code rate. Value range: [50,100]. default value:80
u32MinIQP	The minimum QP of the I frame. The maximum number of bits used to control the I frame. Value range: [MinQp, MaxQp). default value:12
u32MaxIPProp	Maximum ratio of I/P frame size, Value range: [5,100].
u32MaxIQp	The maximum QP of the I frame. The minimum number of bits used to control the I frame. Value range: (MinQp, MaxQp]. default value:48
u32MaxISize	The maximum size of the I frame. Rate control will try to let the target size not exceed the max size. If value is 0, it means that rate control will not consider the parameter. Value range:[0, 0xFFFFFFFF] default value:0
u32MaxPSize	The maximum size of the P frame. Rate control will try to let the target size not exceed the max size. If value is 0, it

Member Name	Description
	means that rate control will not consider the parameter. Value range:[0, 0xFFFFFFFF] default value:0
u32MinStillPercent	The minimum percent of target bitrate in still state. If the value set to 100,then AVBR will not actively decrease target rate in still state, so the behavior of AVBR will be same with VBR. Value range:[5,100]
u32MaxStillQp	The maximum QP of the I frame in still state. Value range:[MinIQp, MaxIQp] Not supported yet.
u32MotionSensitivity	The sensitivity of adjusting bitrate depending on the motion of picture. Value range: [0,100] Default value: 100.

- Note

  None.

- Related data types and interfaces

  MI_VENC_GetRcParam
  MI_VENC_SetRcParam

3.70. MI_VENC_RcParam_t

- Description

  Define the rate control advanced parameters of the code channel.

- Definition

```
typedef struct MI_VENC_RcParam_s
{
   MI_U32 au32ThrdI[RC_TEXTURE_THR_SIZE];
   MI_U32 au32ThrdP[RC_TEXTURE_THR_SIZE];
   MI_U32 u32RowQpDelta;
   union
   {
      MI_VENC_ParamH264Cbr_t    stParamH264Cbr;
      MI_VENC_ParamH264Vbr_t    stParamH264VBR;
      MI_VENC_ParamH264Avbr_t stParamH264Avbr;
      MI_VENC_ParamMjpegCbr_t   stParamMjpegCbr;
      MI_VENC_ParamH265Cbr_t    stParamH265Cbr;
      MI_VENC_ParamH265Vbr_t    stParamH265Vbr;
      MI_VENC_ParamH265Avbr_t stParamH265Avbr;
```

```
    };
    MI_VOID*pRcParam;
}MI_VENC_RcParam_t;
```

- Member

Member Name	Description
au32ThrdI	The mad threshold of the I frame macroblock level code rate control. Value range: [0, 255]. Not supported yet.
au32ThrdP	The mad threshold of the P frame macroblock level rate control. Value range: [0, 255]. Not supported yet.
u32RowQpDelta	Row level rate control. Value range: [0,10]. Not supported yet.
stParamH264Cbr	H.264 channel CBR (ConstantBitRate) rate control mode advanced parameters.
stParamH264Vbr	H.264 channel VBR (VariableBitRate) rate control mode advanced parameters.
stParamH264Avbr	H.264 channel AVBR (AdaptiveVariableBitRate) rate control mode advanced parameters.
stParamH265Cbr	H.265 channel CBR (ConstantBitRate) rate control mode advanced parameters.
stParamH265Vbr	H.265 channel VBR (VariableBitRate) rate control mode advanced parameters.
stParamH265Avbr	H.265 channel AVBR (AdaptiveVariableBitRate) rate control mode advanced parameters.
pRcParam	The parameters are reserved and are not currently used.

- Note

  None.

- Related data types and interfaces

  MI_VENC_SetRcParam
  MI_VENC_GetRcParam

3.71. MI_VENC_CropCfg_t

- Description

  Define the channel crop parameter.

- Definition

  typedef struct MI_VENC_CropCfg_s

```
{
    MI_BOOL bEnable; /* Crop region enable */
    MI_VENC_Rect_t stRect; /* Crop region, note: s32X must be multi of 16 */
} MI_VENC_CropCfg_t;
```

- Member

Member Name	Description
bEnable	Whether to cut. MI_TRUE: Enable cropping. MI_FALSE: Cropping is not enabled.
stRect	Cropped area. stRect.s32X: H.264 must be 16 pixels aligned and H.265 must be 32 pixels aligned. stRect.s32Y: H.264/H.265 must be 2 pixels aligned. stRect.u32Width, s32Rect.u32Height: H.264 must be 16 pixels aligned, H.265 must be 32 pixels aligned.

- Note

None.

- Related data types and interfaces

MI_VENC_SetCrop
MI_VENC_GetCrop

3.72.MI_VENC_RecvPicParam_t

- Description

Receives the specified frame number image encoding.

- Definition

```
typedef struct MI_VENC_RecvPicParam_s
{
    MI_S32 s32RecvPicNum;
}MI_VENC_RecvPicParam_t;
```

- Member

Member Name	Description
s32RecvPicNum	The number of frames that the encoding channel continuously receives and encodes.

- Note

None.

- Related data types and interfaces

MI_VENC_StartRecvPicEx

3.73. MI_VENC_H264eIdrPicIdMode_e

- Description

  Set the mode of the IDR frame or the idr_pic_id of the I frame.

- Definition

```
typedef enum
{
    E_MI_VENC_H264E_IDR_PIC_ID_MODE_USR,
}MI_VENC_H264eIdrPicIdMode_e;
```

- Member

Member Name	Description
E_MI_VENC_H264E_IDR_PIC_ID_MODE_USR	User mode; ie idr_pic_id is set by the user.

- Note

  None.

- Related data types and interfaces

  MI_VENC_SetH264IdrPicId

3.74. MI_VENC_H264IdrPicIdCfg_t

- Description

  The idr_pic_id parameter of the IDR frame or I frame.

- Definition

```
typedef struct MI_VENC_H264IdrPicIdCfg_s
{
   MI_VENC_H264eIdrPicIdMode_e  eH264eIdrPicIdMode;
   MI_U32  u32H264eIdrPicId;
}MI_VENC_H264IdrPicIdCfg_t;
```

- Member

Member Name	Description
eH264eIdrPicIdMode	Set the mode of idr_pic_id.
u32H264eIdrPicId	The value of idr_pic_id. The value range is: [0, 65535].

- Note

  None.

- Related data types and interfaces

  MI_VENC_SetH264IdrPicId

3.75.MI_VENC_FrameLostMode_e

- Description

  Drop frame mode when the instantaneous code rate exceeds the threshold.

- Definition

```
typedef enum
{
  E_MI_VENC_FRMLOST_NORMAL,
  E_MI_VENC_FRMLOST_PSKIP,
  E_MI_VENC_FRMLOST_MAX,
}MI_VENC_FrameLostMode_e;
```

- Member

Member Name	Description
E_MI_VENC_FRMLOST_NORMAL	Normal frame loss when the instantaneous code rate exceeds the threshold.
E_MI_VENC_FRMLOST_PSKIP	The pskip frame is encoded when the instantaneous code rate exceeds the threshold.

- Note

  None.

- Related data types and interfaces

  MI_VENC_SetFrameLostStrategy
  MI_VENC_GetFrameLostStrategy

3.76.MI_VENC_ParamFrameLost_t

- Description

  The frame loss policy parameter when the instantaneous code rate exceeds the threshold.

- Definition

```
typedef struct MI_VENC_ParamFrameLost_s
{
   MI_BOOL bFrmLostOpen;
   MI_U32 u32FrmLostBpsThr;
   MI_VENC_FrameLostMode_e eFrmLostMode;
   MI_U32 u32EncFrmGaps;
} MI_VENC_ParamFrameLost_t;
```

- Member

Member Name	Description
bFrmLostOpen	The frame loss switch when the instantaneous code rate

Member Name	Description
	exceeds the threshold.
u32FrmLostBpsThr	Drop frame threshold. (in bits/s)
enFrmLostMode	Frame loss mode when the instantaneous code rate exceeds the threshold.
u32EncFrmGaps	Frame loss interval, the default is 0.Value range: [0,65535]

- Note

  None.

- Related data types and interfaces

  MI_VENC_SetFrameLostStrategy

3.77.MI_VENC_SuperFrameCfg_t

- Description

  Very large frame processing strategy parameters.

- Definition

```
typedef struct MI_VENC_SuperFrameCfg_s
{
  MI_VENC_SuperFrmMode_e  eSuperFrmMode;
  MI_U32  u32SuperIFrmBitsThr;
  MI_U32  u32SuperPFrmBitsThr;
  MI_U32  u32SuperBFrmBitsThr;
}MI_VENC_SuperFrameCfg_t;
```

- Member

Member Name	Description
eSuperFrmMode	Large frame processing mode.
u32SuperIFrmBitsThr	The I frame has a large threshold and defaults to 0. Value range: greater than or equal to 0. (unit: bits)
u32SuperPFrmBitsThr	The P frame has a large threshold and defaults to 0. Value range: greater than or equal to 0. (unit: bits)
u32SuperBFrmBitsThr	The B frame has a large threshold and defaults to 0. Value range: greater than or equal to 0. (unit: bits)

- Note

  If the threshold is set to 0, the corresponding setting will not take effect. For example, the I frame threshold ≠ 0 and the P frame threshold = 0, then the jumbo frame setting will only take effect for I frame.

- Related data types and interfaces

  MI_VENC_SetSuperFrameCfg

3.78.MI_VENC_RcPriority_e

- Description

  Rate control priority enumeration.

- Definition

```
typedef enum
{
  E_MI_VENC_RC_PRIORITY_BITRATE_FIRST=1,
  E_MI_VENC_RC_PRIORITY_FRAMEBITS_FIRST,
  E_MI_VENC_RC_PRIORITY_MAX,
}MI_VENC_RcPriority_e;
```

- Member

Member Name	Description
E_MI_VENC_RC_PRIORITY_BITRATE_FIRST	The target bit rate is prioritized.
E_MI_VENC_RC_PRIORITY_FRAMEBITS_FIRST	The oversized frame threshold is prioritized.

- Note

  None.

- Related data types and interfaces

  None.

3.79.MI_VENC_ModParam_t

- Description

  Encode related module parameters.

- Definition

```
typedef struct MI_VENC_ModParam_s
{
  MI_VENC_ModType_e eVencModType;
  union
  {
    //MI_VENC_ParamModVenc_t stVencModParam; //not defined yet
    //MI_VENC_ParamModH264e_t stH264eModParam; //not defined yet
    MI_VENC_ParamModH265e_t stH265eModParam;
    MI_VENC_ParamModJpege_t stJpegeModParam;
  };
} MI_VENC_ModParam_t;
```

- Member

Member Name	Description

eVencModType	Set or get the type of module parameter.
stH265eModParam/ stJpegeModParam	mi_venc.ko module parameter structure.

- Note

  None.

- Related data types and interfaces

  No

3.80. MI_VENC_ModType_e

- Description

  Encoding related module parameter types.

- Definition

```
typedef enum
{
  E_MI_VENC_MODTYPE_VENC=1,
  E_MI_VENC_MODTYPE_H264E,
  E_MI_VENC_MODTYPE_H265E,
  E_MI_VENC_MODTYPE_JPEGE,
  E_MI_VENC_MODTYPE_MAX
}MI_VENC_ModType_e;
```

- Member

Member Name	Description
E_MI_VENC_MODTYPE_VENC	VENC module parameter type, not support.
E_MI_VENC_MODTYPE_H264E	h264 module parameter type
E_MI_VENC_MODTYPE_H265E	h265 module parameter type
E_MI_VENC_MODTYPE_JPEGE	jpeg module parameter type

- Note

  None.

- Related data types and interfaces

  MI_VENC_ModParam_t

3.81. MI_VENC_ParamModH265e_t

- Description

  mi_h265.ko module parameters.

- Definition

  typedef struct MI_VENC_ParamModH265e_s

```
{
    MI_U32  u32OneStreamBuffer;
    MI_U32  u32H265eMiniBufMode;
}MI_VENC_ParamModH265e_t;
```

- Member

Member Name	Description
u32OneStreamBuffer	The u32OneStreamBuffer parameter in the mi_h265.ko module. 0: Multi-package mode 1: single package mode
u32H265eMiniBufMode	u32H265eMiniBufMode parameter in the mi_h265.ko module. 0: code stream buffer is allocated according to resolution 1: The minimum buffer of the stream buffer is 32k, which is guaranteed by the user.

- Note

None.

- Related data types and interfaces

MI_VENC_ModParam_t

3.82. MI_VENC_ParamModJpege_t

- Description

Parameters in the mi_jpeg.ko module.

- Definition

```
typedef struct MI_VENC_ParamModJpege_s
{
    MI_U32  u32OneStreamBuffer;
    MI_U32  u32JpegeMiniBufMode;
}MI_VENC_ParamModJpege_t;
```

- Member

Member Name	Description
u32OneStreamBuffer	The u32OneStreamBuffer parameter in the mi_jpeg.ko module. 0: Multi-package mode 1: single package mode
u32JpegeMiniBufMode	u32JpegeMiniBufMode parameter in the mi_jpeg.ko module. 0: code stream buffer is allocated according to resolution 1: The minimum buffer of the stream buffer is 32k, which

Member Name	Description
	is guaranteed by the user.

- Note

  None.

- Related data types and interfaces

  MI_VENC_ModParam_t

3.83. MI_VENC_AdvCustRcAttr_t

- Description

  Customer defined advanced RC configuration parameters.

- Definition

```
typedef struct MI_VENC_AdvCustRcAttr_s
{
    MI_BOOL bEnableQPMap;
    MI_BOOL bAbsQP;
    MI_BOOL bEnableModeMap;
    MI_BOOL bEnabelHistoStaticInfo;
}MI_VENC_AdvCustRcAttr_t;
```

- Member

Member Name	Description
bEnableQPMap	Whether to enable QPMap function. MI_TRUE: ON. MI_FALSE: OFF.
bAbsQP	Whether to use the absolute QP when QP Map is configured. MI_TRUE: YES. MI_FALSE: NONE. The relative QP value is used.
bEnableModeMap	Whether to enable ModeMap function. MI_TRUE: ON. MI_FALSE: OFF.
bEnabelHistoStaticInfo	Whether to enable the lastest information acquisition function of the encoded frame. MI_TRUE: ON. MI_FALSE: OFF.

- Note

  None.

- Related data types and interfaces

  MI_VENC_SetAdvCustRcAttr

3.84. MI_VENC_FrameHistoStaticInfo_t

- Description

  Attributes of the encoded frame.

- Definition

```
typedef struct MI_VENC_FrameHistoStaticInfo_s
{
    MI_U8   u8PicSkip;
    MI_U16  u16PicType;
    MI_U32  u32PicPoc;
    MI_U32  u32PicSliNum;
    MI_U32  u32PicNumIntra;
    MI_U32  u32PicNumMerge;
    MI_U32  u32PicNumSkip;
    MI_U32  u32PicAvgCtuQp;
    MI_U32  u32PicByte;
    MI_U32  u32GopPicIdx;
    MI_U32  u32PicNum;
    MI_U32  u32PicDistLow;
    MI_U32  u32PicDistHigh;
} MI_VENC_FrameHistoStaticInfo_t;
```

- Member

Member Name	Description
u8PicSkip	The frame skip flag.
u16PicType	Encoded frame type: 0: I 1: P 2: B
u32PicPoc	A POC value of the currently encoded picture.
u32PicSliNum	The number of slice segments.
u32PicNumIntra	The number of intra blocks in the encoded frame (8x8 unit).
u32PicNumMerge	The number of merge blocks in the encoded frame (8x8 unit).
u32PicNumSkip	The number of skip blocks in the encoded frame (8x8 unit).
u32PicAvgCtuQp	An average value of CTU QPs.
u32PicByte	The size of encoded picture in byte.
u32GopPicIdx	A picture index in GOP.
u32PicNum	The encoded picture number.
u32PicDistLow	Low 32-bit SSD between source Y picture and reconstructed Y picture.
u32PicDistHigh	Low 32-bit SSD between source Y picture and reconstructed Y picture.

- Note

  None.

- Related data types and interfaces

  MI_VENC_GetLastHistoStaticInfo

3.85. MI_VENC_InputSourceConfig_t

- Description

  Input source config parameters.

- Definition

```
typedef struct MI_VENC_InputSourceConfig_s
{
    MI_VENC_InputSrcBufferMode_e eInputSrcBufferMode;
} MI_VENC_InputSourceConfig_t;
```

- Member

Member Name	Description
eInputSrcBufferMode	Input buffer mode

- Note

  None.

- Related data types and interfaces

  MI_VENC_InputSrcBufferMode_e, MI_VENC_SetInputSourceConfig

3.86. MI_VENC_InputSrcBufferMode_e

- Description

  Define H.264/H.265 input buffer mode enumeration.

- Definition

```
typedef enum
{
    E_MI_VENC_INPUT_MODE_NORMAL_FRMBASE = 0, /*Handshake with input by about 3
                                               buffers in frame mode*/
    E_MI_VENC_INPUT_MODE_RING_ONE_FRM, /*Handshake with input by one buffer in ring
                                         mode*/
    E_MI_VENC_INPUT_MODE_RING_HALF_FRM, /*Handshake with input by half buffer in ring
                                          mode*/
    E_MI_VENC_INPUT_MODE_HW_AUTO_SYNC, /*Handshake with input by hw auto sync in
                                         ring mod*/
    E_MI_VENC_INPUT_MODE_RING_UNIFIED_DMA, /*Multiple venc main channel
                                             handshake with input by one unified
```

```
                                                    buffer in ring mode*/
    E_MI_VENC_INPUT_MODE_MAX
} MI_VENC_InputSrcBufferMode_e;
```

- Member

Member Name	Description
E_MI_VENC_INPUT_MODE_NORMAL_FRMBASE	Handshake with input by about 3 buffers in frame mode.
E_MI_VENC_INPUT_MODE_RING_ONE_FRM	Handshake with input by one buffer in ring mode.
E_MI_VENC_INPUT_MODE_RING_HALF_FRM	Handshake with input by half buffer in ring mode.
E_MI_VENC_INPUT_MODE_HW_AUTO_SYNC	Use this to distinguish ring mode and hw auto sync mode
E_MI_VENC_INPUT_MODE_RING_UNIFIED_DMA	When the front stage is connected to SCL, the main channel of Venc multiplexes the same ring buf.

- Note

  None.

- Related data types and interfaces

  MI_VENC_InputSourceConfig_t
  MI_VENC_SetInputSourceConfig

3.87. VENC_MAX_SAD_RANGE_NUM

- Description

  define the maximum number of statistical SAD values that fall within the range of the intelligent detection.

- Definition

```
#define  VENC_MAX_SAD_RANGE_NUM          16
```

- Note

  None.

- Related data types and interfaces

  None.

3.88. MI_VENC_SmartDetType_e

- Description

  define the intelligent detection type.

- Definition

```
typedef enum
{
  E_MI_VENC_MD_DET=1,
  E_MI_VENC_ROI_DET,
  E_MI_VENC_SMART_DET_MAX,
} MI_VENC_SmartDetType_e;
```

- Member

Member Name	Description
E_MI_VENC_MD_DET	type of motion detection
E_MI_VENC_ROI_DET	type of ROI detection

- Note

None.

- Related data types and interfaces

None.

3.89. MI_VENC_MdInfo_t

- Description

define the statistics of motion detection.

- Definition

```
typedef struct MI_VENC_MdInfo_s
{
  MI_U16  u16SadRangeRatio[VENC_MAX_SAD_RANGE_NUM];
} MI_VENC_MdInfo_t;
```

- Member

Member Name	Description
U16SadRangeRatio	The percentage of SAD of all MB in each frame that falls into each interval.

- Note

The value range of SAD is [0, 255]. All SAD values are divided into 16 ranges:
[0,10), [10,15), [15,20), [20,25), [25,30), [30,40), [40,50), [50,60), [60,70), [70,80), [80,90), [90,100), [100,120), [120,140), [140,160), [160, 255]。
Here is the percentage of all MB(8*8) in a frame that falls into the above range.

- Related data types and interfaces

VENC_MAX_SAD_RANGE_NUM

3.90.MI_VENC_SmartDetInfo_t

- Description

  Define the statistics required by the intelligent detection algorithm.

- Definition

```
typedef struct MI_VENC_SmartDetInfo_s
{
 MI_VENC_SmartDetType_e eSmartDetType;
 union
 {
      MI_VENC_MdInfo_t  stMdInfo;
      MI_BOOL           bRoiExist;
 };
 MI_U8                  u8ProtectFrmNum;
} MI_VENC_SmartDetInfo_t;
```

- Member

Member Name	Description
eSmartDetType	type of intelligent detection
stMdInfo	statistics of motion detection
bRoiExist	information in the ROI region MI_TRUE: the current frame has ROI MI_FALSE: no ROI exists for the current frame
u8ProtectFrmNum	number of protected frames

- Note

  None.

3.91.MI_VENC_IntraRefresh_t

- Description

  Support P-frame containing Islice.

- Definition

```
typedef struct MI_VENC_IntraRefresh_s
{
MI_BOOL bEnable;
MI_U32 u32RefreshLineNum;
MI_U32 u32ReqIQp;
}MI_VENC_IntraRefresh_t;
```

- Member

Member Name	Description
bEnable	Enable venc Intra Refresh

u32RefreshLineNum	Total number of Islices that need to be refreshed in each gop sequence
u32ReqIQp	Whether or not to encode I frames

- Note

  Only H.264 and H.265 are supported.

- Related data types and interfaces

  MI_VENC_SetIntraRefresh

3.92.MI_VENC_InitParam_t

- Description

  Venc device initialization parameter.

- Definition

```
typedef struct MI_VENC_InitParam_s
{
    MI_U32 u32MaxWidth;
    MI_U32 u32MaxHeight;
}MI_VENC_InitParam_t;
```

- Member

Member Name	Description
u32MaxWidth	Max width used by venc device
u32MaxHeight	Max height used by venc device

- Note

  Pudding, Ispahan, Tiramisu, Ikayaki, Muffin, Mochi and Maruko series should set max width and max height case by case according to actual need.

- Related data types and interfaces

  MI_VENC_CreateDev

3.93.MI_VENC_H265eRefType_e

- Description

  Defines the frame type and reference attribute of the H.265 frame skip reference stream.

- Definition

```
typedef MI_VENC_H264eRefType_e MI_VENC_H265eRefType_e;
```

- Members

  None.

- Note

Please refer to MI_VENC_H264eRefType_e for details.

- Related data types and interface
  MI_VENC_H264eRefType_e

4. ERROR CODE

The video coding API error codes are shown in the table below:

Return Code	Macro Definition	Description
0x0	MI_SUCCESS	success
0xa0022001	MI_ERR_VENC_INVALID_DEVID	invalid device ID
0xa0022002	MI_ERR_VENC_INVALID_CHNID	invalid channel ID
0xa0022003	MI_ERR_VENC_ILLEGAL_PARAM	at least one parameter is illegal
0xa0022004	MI_ERR_VENC_EXIST	channel exists
0xa0022005	MI_ERR_VENC_UNEXIST	channel unexist
0xa0022006	MI_ERR_VENC_NULL_PTR	using a NULL point
0xa0022007	MI_ERR_VENC_NOT_CONFIG	try to enable or initialize device or channel, before configuring attribute
0xa0022008	MI_ERR_VENC_NOT_SUPPORT	operation is not supported by NOW
0xa0022009	MI_ERR_VENC_NOT_PERM	operation is not permitted
0xa002200C	MI_ERR_VENC_NOMEM	failure caused by malloc memory
0xa002200D	MI_ERR_VENC_NOBUF	failure caused by malloc buffer
0xa002200E	MI_ERR_VENC_BUF_EMPTY	no data in buffer
0xa002200F	MI_ERR_VENC_BUF_FULL	no buffer for new data
0xa0022010	MI_ERR_VENC_NOTREADY	System is not ready
0xa0022011	MI_ERR_VENC_BADADDR	bad address
0xa0022012	MI_ERR_VENC_BUSY	resource is busy
0xa0022013	MI_ERR_VENC_CHN_NOT_STARTED	channel not started
0xa0022014	MI_ERR_VENC_CHN_NOT_STOPPED	channel not stopped
0xa002201F	MI_ERR_VENC_UNDEFINED	unexpected error

5. PROCFS INTRODUCTION

5.1. cat

- Debug info

#cat /proc/mi_modules/mi_venc/mi_vencN

N represents different devices, and the values of N for different chips are as follows:

Table 4-1 N values of different chips

Chip	N value	
	H.264/H.265	JPEG
Pretzel	0	1
Macaron	0	1
Pudding	0	1
Ispahan	0	1
Tiramisu	0	8
Ikayaki	0	1
Muffin	0,1	8,9
Mochi	0	8,9
Maruko	0	8

```
----------------------------- All VENC Dev info -----------------------------
------------------------------ VENC 0 Dev info ------------------------------
  DevId  IrqNum IsrTotalCnt  IsrFrmDoneCnt  IsrBufFullCnt IsrSliceDoneCnt IsrRingFullCnt  IsrTimeoutCnt IsrOtherCnt
      0  26/  0        1049           1049          0,   0              0              0             0           0
  DevId  MaxTaskCnt    WorkingTaskCnt  UtilHw  UtilMi  PeakHw  PeakMi     FPS      MbRate   %
      0           1                 0       0       0       0       0   29.99      972000 146
  DevId SupportRing  SupportImi NeedPadding  CmdqHandle CmdqBufSize  ResetTime MaxW MaxH AlignMaxW AlignMaxH
      0           0           0           0      (null)           0          0 3840 2160      3840      2176

------------------------------ VENC 0 CHN info ------------------------------
ChnId  DevId  bStart  RefMemPA  RefMemSize  AlMemPA  AlMemVA  AlMemSize
    0      0       1         0           0        0   (null)          0
ChnId  State  EnPred  base  enhance MaxStreamCnt  FrameIdx  Gardient  Fps_1s    kbps  Fps10s    kbps
    0      0       0     1        0           20         1       929   30.18    8456   29.99    7917
ChnId  RingBufStartLine  RingBufHeight QueryCnt   GetStreamCnt   RlsStreamCnt   PollReadyCnt    PollFailCnt
    0                 0              0     1049           1049           1049           1049           1049
ChnId  BufWidth BufHeight ContentWidth  ContentHeight AlignMaxW AlignMaxH
    0      3840      2176         3840           2176      3840      2176
--------------------------- InputPort of dev: 0 ---------------------------
ChnId  Width  Height  SrcFrmRate  MaxW  MaxH  FrameCnt  DropCnt  BlockCnt
    0   3840    2160     30/1      3840  2160      1049        1         0
--------------------------- OutputPort of dev: 0 ---------------------------
ChnId  CODEC  Profile   BufSize MinAllocSize  RefNum  bByFrame  FrameCnt  DropCnt  ReEncCnt  RingUnreadCnt  RingTotalCnt   UsrLockedCnt
    0   H264        2   4147200        33528       0         1      1049        0         0              0             0              0
ChnId RateCtl  GOP MaxBitrate  IPQPDelta  MaxQp  MinQp  MaxIQp  MinIQp  QpMap  AbsQp  ModeMap
    0     CBR   60    8000000         -2     48     15      48      15      0      0        0
```

- Debug info analysis

  Record the current usage status of a VENC device, as well as device attributes, layer attributes, and inputport attributes, which can be dynamically obtained for debugging and testing.

- Parameter Description

Parameter		Description
Dev info	DevId	Device number, refer to Table 4-1
	IrqNum	Interrupt number
	IsrTotalCnt	Total number of various types of interrupt
	IsrFrmDoneCnt	Total number of interrupt of frame done type
	IsrBufFullCnt	The total number of interrupt of buffer full type
	IsrSliceDoneCnt	Total number of interrupt of slice done
	IsrRingFullCnt	The total number of interrupt of ring buffer full type
	IsrReEncCnt	The total number of interrupt of re-encoding type
	IsrTimeoutCnt	The total number of interrupt timed out

Parameter		Description
	IsrOtherCnt	The total number of other types of interrupt
	MaxTaskCnt	Support the maximum number of tasks that can be processed at the same time
	WorkingTaskCnt	Number of tasks being processed
	FPS	Statistics the output frame rate of the device
	SupportRing	Whether to support HW RING
	SupportImi	Whether to support HW REALTIME
	NeedPadding	Need MI SYS clear padding or not
	CmdqHandle	cmdq handle value
	CmdqBufSize	cmdq buffer size
	ResetTime	Device reset time
	MaxW	The width of max resolution
	MaxH	The height of max resolution
	AlignMaxW	The aligned width of max resolution
	AlignMaxH	The aligned height of max resolution
CHN info	ChnId	Channel ID
	bStart	Whether channel encoding has been started
	bStartReceive	Whether the channel has started receiving image data
	ReceiveFrameCnt	The number of frames received by the last call to the start interface. If the value is 0, it means that the image coding is received until the user stops receiving the image coding
	RefMemPA	The hardware physical address required by the underlying driver
	RefMemSize	The hardware memory size required by the underlying driver
	AlMemPA	The CPU physical address required by underlying driver
	AlMemVA	The CPU virtual address required by underlying driver
	AlMemSize	The CPU memory size required by underlying driver
	State	Underlying processing state: typedef enum { E_MI_VENC_BUFSTATE_IDLE = 0, E_MI_VENC_BUFSTATE_ENCODE, E_MI_VENC_BUFSTATE_DONEWAIT_CMDQ, E_MI_VENC_BUFSTATE_CHECK, E_MI_VENC_BUFSTATE_DROP, E_MI_VENC_BUFSTATE_DROP_FORCEBUF_OVERRIDE, E_MI_VENC_BUFSTATE_RECOVER, E_MI_VENC_BUFSTATE_BUF_FULL, E_MI_VENC_BUFSTATE_MAX } MI_VENC_BufferState_e;
	EnPred	Whether the frame of the base layer is used as a reference by other frames of the base layer

Parameter		Description
	base	Period of the base layer, value range: (0, +∞)
	enhance	Period of the enhance layer, value range: [0, 255]
	MaxStreamCnt	Maximum output buffer number
	Fps_1s	Count the output frame rate within 1s
	kbps1s	Count the output bit rate within 1s
	Fps10s	Count the output frame rate within 10s
	kbps10s	Statistic output code rate within 10s
	QueryCnt	The number of times that APP calls MI_VENC_Query
	GetStreamCnt	The number of times that APP calls MI_VENC_GetStream
	RlsStreamCnt	The number of times that APP calls MI_VENC_ReleaseStream
	PollReadyCnt	The number of successful APP calls to select/poll
	PollFailCnt	Number of failed select/poll calls by APP
	FrameIdx	The serial number of the current frame: Before Pudding, the value is: 1, 2, 3, 4 After Pudding, the value is: 1, 2, 4
	BufW	The width of input yuv buffer
	BufH	The height of input yuv buffer
	Stride	The stride of input yuv buffer
	ContentW	The effective width of input yuv buffer
	ContentH	The effective height of input yuv buffer
	RingStartLine	ring start line in ring mode
	RingRealTotalHeight	ring real total height in ring mode
	YBufSize	Y buffer size
	BufSize	YUV total buffer size
	Gradient	The gradient value passed by the front-end isp
chn pipeline delay in us	Flow	GetStream by app
	Min	The minimum delay from the Buffer generation to the flow corresponding to the chn
	Max	The maximum delay from the Buffer generation to the flow corresponding to the chn
	Avg	The average delay between the Buffer generation and the corresponding Flow of the chn
	Diff	The current delay from the Buffer generation to the flow corresponding to the chn
Inputport info	ChnId	Channel ID
	Width	Picture width (pixels)
	Height	Picture height (pixels)
	SrcFrmRate	Source frames per second, usually used as a parameter of rate control
	MaxW	Maximum picture width (pixels)

Parameter		Description
	MaxH	Maximum picture height (pixels)
	FrameCnt	Count of frames processed
	DropFrameCnt	Count of dropped frames
	BlockCnt	The total number of input pictures that have been received but rewinded because the previous input picture is still being processed. Assuming that it can process up to two input pictures at the same time, if the two input pictures have not been encoded and executed, the third frame comes, then the third frame will be rewinded, BlockCnt++.
Outputport info	ChnId	Channel ID
	CODEC	Which CODEC to use: H264,H265,JPEG
	Profile	Profile value
	BufSize	Output pool total size (bytes) set by user
	MinAllocSize	Minimum output buffer size allowed to be allocated
	RefNum	Maximum number of reference frames
	bByFrame	Does the subsequent stage take one frame at a time? 0: No, usually means to fetch data according to slice 1: Yes
	FrameCnt	Count of frames that have been encoded and released
	DropCnt	Count of frames that have been encoded but dropped
	ReEncCnt	Count of re-encoded frames
	RingUnreadCnt	Count of frames in the output pool that have not yet been taken
	RingTotalCnt	Count of frames in the output pool
	UsrLockedCnt	Count of frames the user is holding
	RateCtl	Rate Control algorithm: CBR/VBR/AVBR/FixQP, etc.
	GOP	Group of Picture, I frame interval
	MaxBitrate	In CBR/VBR/AVBR, the maximum target bit rate (bits per second)
	IPQPDelta	The Delta between the average QP value and the QP of the current I frame
	MaxQp	In CBR/VBR/AVBR, the maximum allowed QP
	MinQp	In CBR/VBR/AVBR, the minimum allowed QP
	MaxIQP	In CBR/VBR/AVBR, the maximum allowed I frame QP
	MinIQP	In CBR/VBR/AVBR, the minimum allowed I frame QP
	MaxISize	Maximum size of I frame
	MaxPSize	Maximum size of P frame
	ChangePos	The ratio of the bit rate when VBR/AVBR starts to adjust QP to the maximum bit rate
	MinStPct	In AVBR, the minimum percentage of the target bit rate in the static state

Parameter		Description
	MaxStillQp	In AVBR, the maximum I frame QP in static state
	MotionSens	In AVBR, adjust the sensitivity of the bit rate according to the degree of motion of the screen
	QpMap	The QPMAP mode allows users to freely determine the rate control strategy
	AbsQp	Use absolute QP value
	ModeMap	0 decided byhw; 1 force skip; 2 force intra
	Qfactor	Under PEG FixQP, the quantization factor
	MaxQfactor	Under JPEG CBR, the maximum quantization factor
	MinQfactor	Under JPEG CBR, the minimum quantization factor

5.2. echo

Function	Dump input buffer
Command	echo dump_in [ChnId] [num] [dump_path] > /proc/mi_modules/mi_venc/mi_venc[DevID]
Parameter Description	[ChnId]: Channel ID [num]: The total number of sheets that need dump_in [dump_path]: Path to save the file [DevID]: Device ID, refer to Table 4-1
Example	echo dump_in 0 10 /mnt > /proc/mi_modules/mi_venc/mi_venc0

Function	Dump output stream
Command	echo dump_out [ChnId] [num] [dump_path] > /proc/mi_modules/mi_venc/mi_venc[DevID]
Parameter Description	[ChnId]: Channel ID; [num]: The total number of sheets that need dump_out [dump_path]: Path to save the file [DevID]: Device ID, refer to Table 4-1
Example	echo dump_out 0 100 /mnt > /proc/mi_modules/mi_venc/mi_venc0

Function	Show all used device info
Command	echo show_all [Status] > /proc/mi_modules/mi_venc/mi_venc[DevID]
Parameter Description	[Status]: 0-1 0:Show only requested device info 1:Show all used device info. (with created channels) [DevID]: Device ID, refer to Table 4-1
Example	echo show_all 1 > /proc/mi_modules/mi_venc/mi_venc0

Function	Show output frame count
Command	echo frame_cnt [ChnId] > /proc/mi_modules/mi_venc/mi_venc[DevID]
Parameter	[ChnId]: Channel ID

Description	[DevID]: Device ID, refer to Table 4-1
Example	echo frame_cnt 0 > /proc/mi_modules/mi_venc/mi_venc0

Function	Drop/Release the corresponding channel output task
Command	echo drop_out [ChnId] [Status] > /proc/mi_modules/mi_venc/mi_venc[DevID]
Parameter Description	[ChnId]: Channel ID, a means all channels [Status]: r : release the corresponding channel output task d : drop the corresponding channel output task [DevID]: Device ID, refer to Table 4-1
Example	echo drop_out 0 d > /proc/mi_modules/mi_venc/mi_venc0 Illustrate: drop channel 0 output task echo drop_out 0 r > /proc/mi_modules/mi_venc/mi_venc0 Illustrate: release channel 0 output task echo drop_out a d > /proc/mi_modules/mi_venc/mi_venc0 Illustrate: drop the whole channel output task echo drop_out a r > /proc/mi_modules/mi_venc/mi_venc0 Illustrate: release the whole channel output task

Function	Drop/Release the corresponding channel input task
Command	echo drop_in [ChnId] [Status] > /proc/mi_modules/mi_venc/mi_venc[DevID]
Parameter Description	[ChnId]: Channel ID, a means all channels [Status]: r : release the corresponding channel input task d : drop the corresponding channel input task [DevID]: Device ID, refer to Table 4-1
Example	echo drop_in 0 d > /proc/mi_modules/mi_venc/mi_venc0 Illustrate: drop channel 0 input task echo drop_in 0 r > /proc/mi_modules/mi_venc/mi_venc0 Illustrate: release channel 0 input task echo drop_in a d > /proc/mi_modules/mi_venc/mi_venc0 Illustrate: drop the whole channel input task echo drop_in a r > /proc/mi_modules/mi_venc/mi_venc0 Illustrate: release the whole channel input task

Function	Set debug level for super frame flow
Command	echo dbg_supfrm [ChnId] [debug_level] > /proc/mi_modules/mi_venc/mi_venc[DevID]
Parameter Description	[ChnId]: Channel ID [debug_level]: 0.none 1.warn 2.info [DevID]: Device ID, refer to Table 4-1
Example	echo dbg_supfrm 0 2 > /proc/mi_modules/mi_venc/mi_venc0

Function	Set debug level for frame lost flow
Command	echo dbg_frmlost [ChnId] [debug_level] > /proc/mi_modules/mi_venc/mi_venc[DevID]
Parameter Description	[ChnId]: Channel ID [debug_level]: 0.none 1.warn 2.info [DevID]: Device ID, refer to Table 4-1
Example	echo dbg_frmlost 0 2 > /proc/mi_modules/mi_venc/mi_venc0

Function	Set debug level for get frame flow
Command	echo dbg_getout [ChnId] [debug_level] > /proc/mi_modules/mi_venc/mi_venc[DevID]
Parameter Description	[ChnId]: Channel ID [debug_level]: 0.none 1.warn 2.info [DevID]: Device ID, refer to Table 4-1
Example	echo dbg_getout 0 2 > /proc/mi_modules/mi_venc/mi_venc0

Function	Dump dump customer qp map
Command	echo dump_map [ChnId] [dump_times] [dump_path] > /proc/mi_modules/mi_venc/mi_venc0
Parameter Description	[ChnId]: Channel ID [dump_times]: Dump times [dump_path]: Path to save the file [DevID]: Device ID, refer to Table 4-1
Example	echo dump_map 0 1 /mnt > /proc/mi_modules/mi_venc/mi_venc0

Function	Dump the internal buffer
Command	echo dump_dram [ChnId] [dump_path] > /proc/mi_modules/mi_venc/mi_venc[DevID]
Parameter Description	[ChnId]: Channel ID [dump_path]: Path to save the file [DevID]: Device ID, refer to Table 4-1
Example	echo dump_dram 0 /mnt > /proc/mi_modules/mi_venc/mi_venc0

Function	Enable to run shell script when hang
Command	echo run_sh [Status] [path]> /proc/mi_modules/mi_venc/mi_venc[DevID]
Parameter Description	[Status]: 1:enable 0:disable [path]: The path of shell script [DevID]: Device ID, refer to Table 4-1
Example	echo run_sh 1 /mnt/dump_mhal_venc_status_hook.sh > /proc/mi_modules/mi_venc/mi_venc0

Function	Enable to recover when hang
Command	echo en_recover [Status] > /proc/mi_modules/mi_venc/mi_venc[DevID]
Parameter	[Status]: 1:enable 0:disable 2:show

Description	[DevID]: Device ID, refer to Table 4-1
Example	echo en_recover 0 > /proc/mi_modules/mi_venc/mi_venc0

Function	Show output stream info
Command	echo show_out [ChnId] > /proc/mi_modules/mi_venc/mi_venc[DevID]
Parameter Description	[ChnId]: Channel ID [DevID]: Device ID, refer to Table 4-1
Example	echo show_out 0 > /proc/mi_modules/mi_venc/mi_venc0

Function	Dynamic debug sed ROI info or sad info
Command	echo debug_sed [ChnId] [SedType] [Status] > /proc/mi_modules/mi_venc/mi_venc[DevID]
Parameter Description	[ChnId]: Channel ID [SedType]: 0: ive detector algorithm ROI info 1: cnn detector algorithm ROI info 2: motion detector algorithm ROI info [Status]: ture: enable false: disable [DevID]: Device ID, refer to Table 4-1
Example	echo debug_sed 0 0 true > /proc/mi_modules/mi_venc/mi_venc0 Illustrate: show ive detector algorithm ROI info echo debug_sed 0 1 true > /proc/mi_modules/mi_venc/mi_venc0 Illustrate: show cnn detector algorithm ROI info echo debug_sed 0 2 true > /proc/mi_modules/mi_venc/mi_venc0 Illustrate: show motion detector algorithm ROI info

Function	Enable to show sram info
Command	echo debug_sram [ChnId] [Status] > /proc/mi_modules/mi_venc/mi_venc[DevID]
Parameter Description	[ChnId]: Channel ID [Status]: ture: enable false: disable [DevID]: Device ID, refer to Table 4-1
Example	echo debug_sram 0 true > /proc/mi_modules/mi_venc/mi_venc0

Function	Enable to show channel function stat info
Command	echo flow_stat [ChnId] [Status] > /proc/mi_modules/mi_venc/mi_venc[DevID]
Parameter Description	[ChnId]: Channel ID [Status]: 1: enable 0: disable [DevID]: Device ID, refer to Table 4-1
Example	echo flow_stat 0 1 > /proc/mi_modules/mi_venc/mi_venc0